



Benchmarking Microservice Platforms and Applications in the Cloud

vorgelegt von

Martin Grambow, M.Sc.

ORCID: 0000-0001-6866-5461

an der Fakultät IV - Elektrotechnik und Informatik
der Technischen Universität Berlin
zur Erlangung des akademischen Grades

Doktor der Ingenieurwissenschaften
- Dr.-Ing. -

Vorgeschlagener Promotionsausschuss

Vorsitzender Prof. Dr. Henning Sprekeler

Gutachter Prof. Dr. David Bermbach

Gutachter Prof. Dr. Wilhelm Hasselbring

Gutachter Prof. Dr. Odej Kao

Berlin 2024

Abstract

Benchmarking can be used for evaluating software performance metrics within a controlled environment, for example, to ensure the compliance with Service Level Agreements (SLAs). Benchmarking microservice platforms and applications in cloud environments, however, is challenging. First, there are performance variations in cloud environments that make it difficult to accurately determine performance metrics and replicate results. Consequently, there is a trade-off between accuracy on the one hand and effort on the other. Running longer and more frequent benchmarking experiments improves the accuracy when quantifying performance metrics and detecting performance changes. Second, running extensive benchmarking experiments after every code change is unpractical due to the continuous development of microservice applications. Extensive benchmarking experiments are time-consuming, complex, costly and slow down the rapid development cycles of microservice applications. Third, cloud platforms undergo constant updates as well, including new hardware or software implementations that directly impact the performance of microservice applications.

In this thesis, we propose three approaches to simplify benchmarking of microservice platforms and applications in cloud environments. First, we propose an approach to automatically adapt and generate the benchmark workload for a microservice application based on abstract interaction patterns and machine-generated service description files. By applying this concept, benchmark workloads no longer need to be manually adjusted after each change in a microservice, but are automatically adapted. Second, we optimize microbenchmark suites based on function calls during an application benchmark and study their performance change detection capabilities in realistic Continuous Integration/Deployment (CI/CD) pipelines. This opens up a variety of application scenarios for CI/CD pipelines where, e.g., the optimized microbenchmark suite might scan the application for performance issues after every source code change while the more complex application benchmark is run only for major releases. Third, we present BeFaaS, an extensible application-centric benchmarking framework for Function as a Service (FaaS) platforms. We show that BeFaaS can be used to study typical use cases and analyze various performance parameters of FaaS platforms. Although the presented benchmarking approaches are primarily applicable to microservice applications and platforms in the cloud, they can also be used in other domains.

Kurzdarstellung

Benchmarking kann zur Bestimmung von Software-Leistungsmetriken in einer kontrollierten Umgebung verwendet werden, um beispielsweise die Einhaltung von SLAs zu gewährleisten. Das Benchmarking von Microservice-Plattformen und -Anwendungen in Cloud-Umgebungen ist jedoch herausfordernd. Erstens gibt es in Cloud-Umgebungen Leistungsschwankungen, die eine genaue Bestimmung von Leistungsmetriken und die Replizierung von Ergebnissen erschweren. Folglich gibt es einen Kompromiss zwischen der Genauigkeit auf der einen Seite und dem Aufwand auf der anderen Seite. Die Durchführung längerer und mehrmals wiederholter Benchmarking-Experimente verbessert die Genauigkeit bei der Bestimmung von Leistungsmetriken und der Erkennung von Leistungsänderungen. Zweitens ist die Durchführung umfangreicher Benchmarking-Experimente nach jeder Codeänderung aufgrund der ständigen Weiterentwicklung von Microservice-Anwendungen nicht praktikabel. Umfangreiche Benchmarking-Experimente sind zeitaufwändig, komplex, und kostspielig, was die schnellen Entwicklungszyklen von Microservice-Anwendungen bremst. Drittens werden auch die Cloud-Plattformen ständig weiterentwickelt, was unter anderem auch Hardware- oder Softwareaktualisierungen beinhaltet, die sich direkt auf die Leistungen von Microservice-Anwendungen auswirken.

In dieser Arbeit präsentieren wir drei Ansätze, die das Benchmarking von Microservice-Plattformen und -Anwendungen in der Cloud vereinfachen. Erstens, einen Ansatz der die Benchmark-Last für eine Microservice-Anwendung automatisch basierend auf maschinengenerierten Dienstbeschreibungen und abstrakten Interaktionsmustern generiert. Durch die Anwendung dieses Konzepts muss die Last für einen Benchmark nicht mehr manuell nach jeder Änderung in einem Microservice angepasst werden, sondern wird automatisch angepasst. Zweitens optimieren wir Microbenchmark-Suites auf der Grundlage von Funktionsaufrufen während eines Anwendungsbenchmarks und untersuchen deren Erkennungsfähigkeiten von Leistungsänderungen in realistischen CI/CD-Pipelines. Dies eröffnet eine Vielzahl von Anwendungsszenarien für CI/CD-Pipelines, in denen beispielsweise die optimierte Suite die Anwendung auf Leistungsprobleme nach jeder Code-Änderung oder Commit durchsucht, während komplexere Benchmarks nur für größere Entwicklungsschritte ausgeführt werden. Drittens stellen wir BeFaaS vor, ein erweiterbares und anwendungsorientiertes Benchmarking-Framework für FaaS-Plattformen. Wir zeigen, dass BeFaaS verwendet werden kann, um typische Anwendungsfälle zu untersuchen und verschiedene Leistungsparameter von FaaS-Plattformen zu analysieren. Die vorgestellten Ansätze sind hauptsächlich für das Benchmarking von Microservice-Anwendungen und -Plattformen in der Cloud anwendbar, können aber auch in anderen Bereichen angewendet werden.

Acknowledgments

This dissertation is a major milestone in my life. Back in 2012, when I started my Bachelor's degree in Information Systems Management at TU Berlin, I had no idea that another Master's degree and now this PhD thesis would follow. I am deeply grateful for the many people who have accompanied and supported me along this journey.

First of all, I would like to thank my advisor and mentor Prof. Dr. David Bermbach for his academic support, valuable guidance and inspiring discussions over the past six years. Besides teaching and your other duties, you always had an open ear for my questions and gave me constructive feedback on my ideas and drafts. You created a very pleasant working environment where I enjoyed a lot of mutual trust and freedom in my research.

I also want to thank Prof. Dr. Henning Sprekeler for chairing the doctoral committee. In the two years I spent working with you as a teaching assistant during my studies, I learned a lot about supervising courses, which subsequently helped me to supervise various courses alongside my research. I would also like to thank Prof. Dr. Wilhelm Hasselbring and Prof. Dr. Odej Kao for agreeing to review this thesis.

A very special thanks goes to my current and former colleagues in the Scalable Software Systems (or formerly Mobile Cloud Computing) research group, my co-authors, and advised students. In countless meetings, we discussed ideas, shared our expertise, and supported each other. Without your ideas, comments, and contributions, none of our papers would be as great as they are.

Finally, I want to warmly thank my parents, family, and friends. Your absolute support and encouragement constantly motivated and energized me to successfully complete this dissertation and made the last few years an unforgettable and fulfilled experience.

Table of Contents

Abstract	iii
Kurzdarstellung	v
Acknowledgments	vii
List of Acronyms	xiii
1 Introduction	1
1.1 Problem Statement	2
1.2 Contributions	4
1.3 List of Publications	6
1.4 Outline	7
2 Background	9
2.1 Benchmarking	9
2.1.1 Application Benchmarks	9
2.1.2 Microbenchmarks	11
2.2 Continuous Integration and Continuous Deployment	11
2.3 Continuous Benchmarking	12
2.4 Handling Performance Variations	12
2.5 Microservices	14
2.6 Software Call Graphs	15
2.7 Function as a Service Platforms	15
3 Automated Workload Generation for Benchmarking Microservices	17
3.1 Pattern-Based Benchmarking	18
3.1.1 Challenges	18
3.1.2 From Interaction Patterns to Service-Specific Workloads	18
3.1.3 System Design	27
3.2 Evaluation	28
3.2.1 Proof-of-Concept Implementation	29
3.2.2 Sock Shop Microservice Application	29
3.2.3 Experiment	30
3.2.4 Findings	33
3.3 Discussion	34
3.4 Summary	36

4	Deriving Optimized Microbenchmark Suites	37
4.1	Optimizing Microbenchmark Suites	38
4.1.1	Quantifying Practical Relevance and Reference Impact	39
4.1.2	Removing Redundancies in Microbenchmark Suites	41
4.1.3	Recommending Additional Microbenchmark Targets	43
4.2	Evaluation	44
4.2.1	Study Objects	45
4.2.2	Application Benchmark Scenarios	47
4.2.3	Application Benchmark Setup and Experiment Plan	49
4.2.4	Microbenchmarks	49
4.2.5	Quantifying Practical Relevance	49
4.2.6	Removing Redundancies in Microbenchmark Suites	52
4.2.7	Recommending Additional Microbenchmark Targets	53
4.2.8	Findings	55
4.3	Case Study on the Detection Capabilities of Microbenchmark Suites	55
4.3.1	Study Design	56
4.3.2	Analysis	60
4.3.3	Detected Performance Changes Using Application Benchmarks	62
4.3.4	Detected Performance Changes Using Optimized Microbenchmark Suites	64
4.3.5	Detected Performance Changes Using Complete Microbenchmark Suites	69
4.3.6	Implications	71
4.3.7	Findings	74
4.4	Discussion	74
4.5	Summary	81
5	Benchmarking FaaS Platforms using the BeFaaS Framework	83
5.1	The BeFaaS Framework	84
5.1.1	Requirements	84
5.1.2	Design	86
5.1.3	Implementation	89
5.1.4	Benchmark Applications	90
5.2	Evaluation	94
5.2.1	Comparing Major Cloud FaaS Providers in Single Provider Setups	94
5.2.2	Evaluating Hybrid Edge-Cloud Setups Using the Smart City Application	95
5.2.3	Analyzing the Event Pipeline Interplay Within and Across FaaS Providers.	97
5.2.4	Studying the Cold Start Behavior of Different Providers.	99
5.2.5	Discussion of Requirements	99
5.2.6	Findings	101
5.3	Discussion	101
5.4	Summary	102

6	Related Work	105
6.1	Benchmarking in Cloud Environments	105
6.2	Continuous Benchmarking	106
6.3	Detecting and Quantifying Performance Changes	107
6.4	Benchmarking Microservices	108
6.5	Optimizing Microbenchmarks	109
6.6	Benchmarking Function as a Service Platforms	110
6.7	Summary	111
7	Conclusion	113
7.1	Summary	113
7.2	Discussion	115
7.3	Outlook	116
	Bibliography	117

Acronyms

API	Application Programming Interface
AWS	Amazon Web Services
CI	Confidence Interval
CI/CD	Continuous Integration/Deployment
CRUD	Create Read Update Delete
FaaS	Function as a Service
GCP	Google Cloud Platform
Azure	Microsoft Azure
QoS	Quality of Service
REST	Representational State Transfer
RMIT	Randomized Multiple Interleaved Trials
SLA	Service Level Agreement
SUT	System Under Test
TSDB	Time Series Database System
URI	Uniform Resource Identifier
VM	Virtual Machine

Introduction

Both functional and non-functional requirements of software services should be reviewed regularly as part of their software development cycle [91]. While functional requirements ensure the proper execution of service tasks, non-functional requirements such as performance describe the quality attributes and constraints of a service [7, 35, 39, 47]. They define, for example, in Service Level Agreements (SLAs) that requests need to be answered within a specified time period and need to be fulfilled by the provider. However, even if there are no specific agreements, performance changes are relevant as slow services can frustrate service users, negatively impacting their experience and potentially driving them away [142, 191]. Additionally, performance changes also have a direct impact on service developers and providers [79, 142, 143, 190]: On the one hand, performance issues are more complex to solve and slow services have to be compensated with more resources, which causes additional costs. On the other hand, providers can also save infrastructure costs if services run more efficiently. Thus, a periodic evaluation and review of performance metrics is useful for several reasons. Ideally, this should happen after every code change to directly identify the root cause of a performance change, but definitely before a release [44, 45, 61, 95, 155, 183, 185], e.g., to ensure that SLAs are fulfilled. Benchmarking, which evaluates a System Under Test (SUT) in a realistic production-like environment with a realistic but artificial workload, enables this verification [19, 25, 60, 83, 92].

Cloud computing has revolutionized the way businesses operate by providing scalable and cost-efficient infrastructure such as Virtual Machines (VMs), storage solutions, or Function as a Service (FaaS) platforms [119, 193]. Software no longer needs to be operated on in-house hardware but can be hosted on virtual resources in data centers rented from a cloud provider as needed. This offers many advantages such as availability, scalability, and cost efficiency, but there is the disadvantage that shared hardware resources and other factors cause large performance variations in cloud environments [50, 60, 120, 130, 153].

Microservice applications consist of distributed services¹, each focused on a specific task, independent of each other, scalable on demand, and communicating via network [52, 98]. In the context of this thesis, services can be customer management applications or database systems that are deployed on individual VMs or hosted as functions on FaaS platforms. Hosting these services in cloud environments offers the aforementioned benefits, however, benchmarking microservice platforms and applications in cloud environment is challenging [3, 7, 25, 26, 60, 62, 92, 165].

This chapter contains (partially adapted) material from our published research articles [72–78].

¹In the following, we will refer to microservices as either 'services' or 'microservices' interchangeably.

1.1 Problem Statement

Benchmark experiments should always be as realistic as possible to derive reliable and relevant findings [19, 25, 60, 83, 92]. For microservice applications hosted in cloud environments, the benchmark must therefore also run in cloud environments. An evaluation on other hardware would not represent the conditions in cloud environments correctly and could result in incorrect conclusions. Running the benchmark in a cloud environment, on the other hand, not only ensures a realistic environment but also brings the advantage that the resources for the experiment only need to be rented for the duration of the experiment. However, the advantages of the cloud environment also come with disadvantages, as performance variations caused by shared hardware in the data centers of cloud providers, for example, make it difficult to accurately determine performance metrics [50, 60, 120, 130, 153, 171]. From short-term variations in the range of seconds to long term variations depending on the time of day. Every activity of other cloud customers in the same data center and every small change on the cloud provider side may have an impact on the measurement and this makes it very difficult to reproduce results. In addition, cloud providers offer different services, which are also constantly changing, making comparisons between providers difficult [139]. Measurements and findings from a week ago may already be outdated and no longer valid in the current week.

To obtain sound findings, nevertheless, it is important to consider the cloud conditions in the benchmark design and experiment plan [50, 60, 120, 130, 153]. First, short-term variations need to be taken into account by running long experiments. This ensures that the variations are recorded and can be taken into account in the analysis. Second, the experiments need to be repeated several times, either consecutively or simultaneously. This ensures that the SUT is evaluated on different instances and that the findings do not rely on a single experiment run. There is, however, a trade-off between accuracy and effort in terms of complexity or duration [84, 130, 136]. On the one hand, the longer the benchmark runs, the more often it is repeated, and the more scenarios are evaluated, the more accurate the result. On the other hand, longer and more experiments increase the cost of the evaluation accordingly.

Benchmarking microservice applications raises additional challenges as services are constantly evolving [52, 98, 121, 140]. Every source code change can potentially affect the benchmark. For example, the benchmark needs to be adjusted after a parameter is renamed or a functionality is introduced. Moreover, every modification can affect performance metrics such as response time, which can lead to violating SLAs, among other things. This brings a second trade-off with it. While monitoring performance metrics can be very fine granular using benchmarking after every change, it can also be very coarse and only conducted before a release [44, 45, 61, 95, 155, 183, 185]. Additionally, the complexity of the benchmark can be adjusted [19, 25, 27, 60, 83, 92]. There can either be simple benchmark runs in isolation for the modified service, or complex benchmarks that evaluate the modified service in conjunction with the entire microservice application. While sporadic benchmarks for individual services require less effort and are thus less time-consuming and costly, they may not reliably provide relevant findings and

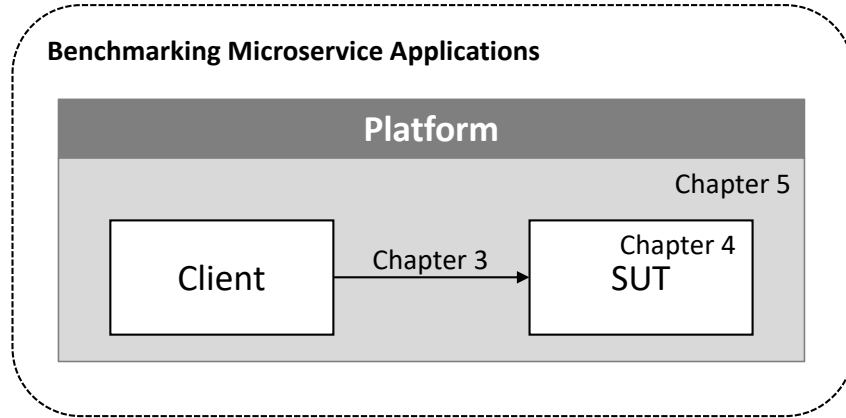


Figure 1.1: Benchmarking a microservice application running on a cloud platform.

it can be difficult to identify the reason for a performance change. On the other hand, frequent extensive benchmarks are very time-consuming and costly, but very accurate, the performance changes can be identified faster, and their causes are easier to identify. Finally, the underlying platform of the cloud provider also needs to undergo a periodic review [27, 118, 139]. On the one hand, benchmarking can be used to compare different cloud providers and thus select the most efficient provider; on the other hand, the performance of the platform needs to be checked periodically, as it is also constantly updated and changes can affect the performance metrics of the microservice applications (even if there are no changes in the source code).

The goal of this thesis is to ease the benchmarking of microservice applications and to enable the most accurate and efficient benchmarking possible. To this end, we address three problems within this general problem context. Figure 1.1 illustrates an abstract benchmarking scheme of a microservice-oriented SUT hosted on a cloud provider platform and evaluated from a client perspective. Here, the client sends an artificial but realistic load of requests to the application and measures performance parameters. This specific load, i.e., the specific parameter values and their syntax, differs for each service and thus these requests had to be created individually and manually for each application: Creating a new customer in a customer management service requires different values than reserving seats on a train. A second issue with manually generated benchmark loads is that they do not evolve with the application code itself. Instead, they need to be adjusted in addition to the code changes, e.g., when a new parameter is introduced for the customer service, making them difficult to maintain (addressed in Chapter 3).

While application benchmarks, which evaluate a complete application in a realistic environment from the client's perspective, are difficult to set up because of the many components involved, microbenchmarks evaluate an application at the function level and repeatably call the respective function while collecting metrics [19, 25, 113, 114]. Thus, a microbenchmark suite, i.e., a set of microbenchmarks, can provide straightforward performance metrics and insights, but does not cover the interaction and integration of different components such as services. In addition, it is hard to estimate the impact of a detected performance change in a microbenchmark on the performance metrics of the overall application. For example, a slower backup

import might raise an alert in a microbenchmark, but this will not affect the running application because this task occurs rarely. Nevertheless, they are easy to embed in Continuous Integration/Deployment (CI/CD) pipelines and provide a quick performance feedback to developers. With larger microbenchmark suites, however, the execution time of the microbenchmarks increases drastically, often taking several hours, which makes them impractical to use and evaluate performance metrics for every single code change in detail; hence, trade-offs have to be made (addressed in Chapter 4).

Hosting services in the cloud means that service owners hand over a certain degree of control to the cloud provider. While the cloud provider’s customer can configure certain parameters and the cloud provider complies with certain general quality metrics, the customer has little control over the specific execution of a service task and its performance metrics [17]. Especially in the FaaS model, where service owners only share the source code and the cloud provider takes care of service execution, scaling, etc., customers have limited options. For a realistic comparison and study of different FaaS systems and their configuration options, FaaS application developers rely on FaaS benchmarking frameworks. Existing frameworks, however, tend to evaluate only single isolated aspects and a more holistic application-centric benchmarking framework is still missing (addressed in Chapter 5).

Overall, measuring the performance of microservice platforms and applications running in cloud environments to ensure Quality of Service (QoS) metrics is essential for modern organizations to avoid unnecessary costs and maintain user satisfaction. However, there are several challenges associated with accurately and efficiently conducting performance evaluations in these complex environments. These challenges include manually creating and maintaining benchmark workloads tailored to individual microservice applications, executing microbenchmark suites in CI/CD pipelines, and comparing FaaS platforms.

1.2 Contributions

In this thesis, we delve into the three challenges mentioned above and propose solutions to enable continuous performance evaluations of microservice-oriented applications in cloud computing environments and their platforms:

Automatic Workload Generation for Benchmarking Microservices:

We enable an automated and realistic benchmark workload generation for microservice applications using machine-generated service description files, i.e., Open API specification files, also known as Swagger files. These service descriptions can be automatically generated based on the source code and thus automatically adapt to changes in the code. We leverage this principle and generate benchmark loads for the respective application, which can also consist of multiple services, based on these description files with minimal manual effort: Assuming a Representational State Transfer (REST) based microservice interface, developers describe the benchmark workload based on abstract interaction patterns. At runtime, our approach uses the correspond-

ing interface description to automatically resolve and bind the workload patterns to the concrete endpoint before executing the benchmark and collecting results. Our approach is not limited to a single service, but is also capable of resolving complex data dependencies across microservice endpoints.

Deriving Practically Relevant Microbenchmark Suites:

We propose an approach to determine the overlap of common function calls between application benchmarks and microbenchmarks, describe a heuristic that identifies redundant microbenchmarks, and present a recommendation algorithm that reveals relevant functions not yet covered by microbenchmarks. A microbenchmark suite optimized in this way only includes a relevant subset of the full microbenchmark suite and can easily evaluate all functions flagged as relevant by application benchmarks after each code change, significantly reducing the risk of undetected performance problems. In an evaluation using two time series database systems, we optimize both microbenchmark suites by decreasing the total number of microbenchmarks and increasing coverage of each suite. It is, however, unclear whether microbenchmarks and application benchmarks detect the same performance problems and if one can be a proxy for the other. To verify our approach, we study whether microbenchmark suites can detect the same application performance changes as an application benchmark in real software development cycles. For this, we run extensive benchmark experiments with both the full and optimized microbenchmark suites of the two time series database systems, for a commit history of 70 code changes for *InfluxDB* and 110 for *VictoriaMetrics*, and compare their results with the results of corresponding application benchmarks. Our results show that it is possible to detect application performance changes using an optimized microbenchmark suite if frequent false positive alarms can be tolerated. By utilizing the differences and synergies between application benchmarks and microbenchmarks, optimized microbenchmark suites are another tool to ensure performance metrics in CI/CD pipelines.

A Benchmarking Framework for FaaS Environments:

We propose BeFaaS, an extensible application-centric benchmarking framework for FaaS environments that focuses on evaluating FaaS platforms using realistic and typical examples of FaaS applications. BeFaaS includes four FaaS application-centric benchmarks that reflect typical FaaS use cases and currently supports commercial cloud FaaS platforms (AWS Lambda, Azure Functions, Google Cloud Functions) and the tinyFaaS edge serverless platform, and is extensible for additional workload profiles and platforms. The framework supports federated benchmark runs in which the benchmark application is distributed over multiple FaaS platforms running on a mix of cloud, edge, and fog nodes. Moreover, BeFaaS implements tracing features that collect fine-grained measurements that can be used for a detailed post-experiment drill-down analysis, e.g., to identify cold starts or other request-level effects. In our study using BeFaaS for comparing different setups and FaaS providers, our experiment results show that (i) network transmission is a major contributor to response latency for function chains, (ii) this effect is exacerbated in hybrid edge-cloud deployments, (iii) the trigger delay between a published event and the start of

the triggered function ranges from about 100ms for AWS Lambda to 800ms for Google Cloud Functions, and (iv) Azure Functions shows the best cold start behavior for our workloads.

1.3 List of Publications

Central parts of this thesis are published in:

- Martin Grambow, Lukas Meusel, Erik Wittern, and David Bermbach. “Benchmarking Microservice Performance: A Pattern-based Approach”. In: *Proc. of the Symposium on Applied Computing (SAC '20)*. ACM, 2020, pp. 232–241. DOI: 10.1145/3341105.3373875
- Martin Grambow, Erik Wittern, and David Bermbach. “Benchmarking the Performance of Microservice Applications”. In: *SIGAPP Applied Computing Review* 20.3 (2020). ACM, pp. 20–34. DOI: 10.1145/3429204.3429206
- Martin Grambow, Christoph Laaber, Philipp Leitner, and David Bermbach. “Using application benchmark call graphs to quantify and improve the practical relevance of microbenchmark suites”. In: *PeerJ Computer Science* (2021). PeerJ. DOI: 10.7717/peerj-cs.548
- Martin Grambow, Denis Kovalev, Christoph Laaber, Philipp Leitner, and David Bermbach. “Using Microbenchmark Suites to Detect Application Performance Changes”. In: *Transactions on Cloud Computing* 11.3 (2023). IEEE, pp. 2575–2590. DOI: 10.1109/TCC.2022.3217947
- Martin Grambow, Tobias Pfandzelter, Luk Burchard, Carsten Schubert, Max Zhao, and David Bermbach. “BeFaaS: An Application-Centric Benchmarking Framework for FaaS Platforms”. In: *Proc. of the International Conference on Cloud Engineering (IC2E '21)*. IEEE, 2021, pp. 1–8. DOI: 10.1109/IC2E52221.2021.00014
- Martin Grambow, Tobias Pfandzelter, and David Bermbach. *Application-Centric Benchmarking of Distributed FaaS Platforms using BeFaaS*. tech. rep. 3S.2024.1. TU Berlin, 2024

Further, related publications are:

- Martin Grambow, Tim Dockenfuß, Trever Schirmer, Nils Japke, and David Bermbach. “Efficiently Detecting Performance Changes in FaaS Application Releases”. In: *Proc. of the International Workshop on Serverless Computing (WoSC '23)*. ACM, 2023, pp. 13–17. DOI: 10.1145/3631295.3631395
- Martin Grambow, Fabian Lehmann, and David Bermbach. “Continuous Benchmarking: Using System Benchmarking in Build Pipelines”. In: *Proc. of the Workshop on Service Quality and Quantitative Evaluation in new Emerging Technologies (SQUEET '19)*. IEEE, 2019, pp. 241–246. DOI: 10.1109/IC2E.2019.00039
- Martin Grambow, Jonathan Hasenburg, Tobias Pfandzelter, and David Bermbach. “Is it Safe to Dockerize my Database Benchmark?” In: *Proc. of the Symposium on Applied*

Computing, Posters Track (SAC '19). ACM, 2019, pp. 341–344. DOI: 10.1145/3297280.3297545

- Martin Grambow, Jonathan Hasenburger, and David Bermbach. “Public Video Surveillance: Using the Fog to Increase Privacy”. In: *Proc. of the Workshop on Middleware and Applications for the Internet of Things (M4IoT '18)*. ACM, 2018, pp. 11–14. DOI: 10.1145/3286719.3286722
- Nils Japke, Christoph Witzko, Martin Grambow, and David Bermbach. “The Early Microbenchmark Catches the Bug – Studying Performance Issues Using Micro- and Application Benchmarks”. In: *Proc. of the International Conference on Utility and Cloud Computing (UCC '23)*. ACM, 2024, pp. 1–10. DOI: 10.1145/3603166.3632128
- Jonathan Hasenburger, Martin Grambow, and David Bermbach. “MockFog 2.0: Automated Execution of Fog Application Experiments in the Cloud”. In: *Transactions on Cloud Computing* 11.1 (2023). IEEE, pp. 58–70. DOI: 10.1109/TCC.2021.3074988
- Jonathan Hasenburger, Martin Grambow, and David Bermbach. “Towards A Replication Service for Data-Intensive Fog Applications”. In: *Proc. of the Symposium on Applied Computing, Posters Track (SAC '20)*. ACM, 2020, pp. 267–270. DOI: 10.1145/3341105.3374060
- Jonathan Hasenburger, Martin Grambow, Elias Grunewald, Sascha Huk, and David Bermbach. “MockFog: Emulating Fog Computing Infrastructure in the Cloud”. In: *Proc. of the International Conference on Fog Computing (ICFC '19)*. IEEE, 2019, pp. 144–152. DOI: 10.1109/ICFC.2019.00026
- Frank Pallas and Martin Grambow. “Three Tales of Disillusion: Benchmarking Property Preserving Encryption Schemes”. In: *Proc. of the International Conference on Trust, Privacy and Security in Digital Business (TrustBus '18)*. Springer, 2018, pp. 39–54. DOI: 10.1007/978-3-319-98385-1_4

1.4 Outline

This thesis is structured as follows. After this introduction in Chapter 1, Chapter 2 outlines background information in the context of this thesis. Besides benchmarking, it covers CI/CD pipelines and benchmarking as part of CI/CD pipelines, and methods to handle performance variations. Moreover, it outlines basic information on microservices, software call graphs, and FaaS platforms.

Chapter 3 presents the automatic generation of benchmark workloads for microservice application based on abstract interaction patterns and service description files. First, Section 3.1 describes the challenges, the approach, and the system design to enable the automatic workload generation. Section 3.2 then evaluates the workload generation using a microservice-oriented application with

several connected services. Last, Section 3.3 discusses the advantages and limitations of the approach, before Section 3.4 summarizes this contribution.

Chapter 4 presents an approach to optimize microbenchmark suites. Section 4.1 describes two algorithms for optimizing microbenchmark suites based on call graph information from application benchmarks. Next, the Sections 4.2 and 4.3 evaluate the optimization: Section 4.2 by applying the optimizations to two open-source time series database systems to determine the potential improvements in execution time and code coverage. Section 4.3 by using the optimized suites in simulated real CI/CD pipelines for multiple code changes to verify that the optimized microbenchmark suites can serve as a proxy for an application benchmark. Chapter 4 ends with Section 4.4 discussing the findings and limitations of optimized microbenchmark suites and Section 4.5 summarizing this contribution.

Chapter 5 presents BeFaaS as an extensible application-centric benchmarking framework for FaaS environments. Section 5.1 begins with a detailed presentation of BeFaaS and its features. Section 5.2 then describes the study design of our experiments using BeFaaS and presents the results. Next, Section 5.3 discusses the framework and addresses limitations before Section 5.4 summarizes this contribution.

Chapter 6 embeds the three contributions in the scientific context. This includes benchmarking in cloud environments, continuous benchmarking, detecting and quantifying performance changes, benchmarking microservices, optimizing microbenchmarks, and benchmarking FaaS platforms.

The final Chapter 7 concludes this thesis with a summary, discussion, and outlook.

Background

This chapter introduces the basic background information necessary for understanding the proposed concepts and their embedding in the scientific context. This chapter contains (partially adapted) material from our published research articles [72–78].

Section 2.1 outlines the basics of benchmarking and the two types of benchmarks we use, namely application benchmarks and microbenchmarks. Next, Section 2.2 deals with CI/CD pipelines and Section 2.3 with continuous benchmarking, which involves integrating benchmarking steps into such pipelines. In Section 2.4, we delve into the execution of benchmark experiments in cloud environments and strategies for handling performance variations. We then outline the approach-specific related background in the following sections in chronological order: Section 2.5 introduces microservice applications, REST interfaces, and their interface descriptions, related to the benchmark workload generation. Section 2.6 outlines software call graphs and their analysis, related to the optimization of microbenchmark suites. Finally, Section 2.7 summarizes the FaaS concept and the basic characteristics of FaaS platforms.

2.1 Benchmarking

Benchmarking aims to determine Quality of Service (QoS) or adherence to SLAs such as a specified maximum latency or processing duration by stressing a SUT in a standardized way while observing its reactions. In contrast to monitoring, which is about non-intrusive and passive observation of a (production) system, benchmarking typically runs in a non-production environment and aims to answer how a system reacts on specific changes or stresses, and is about comparing system alternatives, system versions, configurations, or deployments. During a benchmark run, several scenario-specific metrics are measured and then subsequently evaluated in an offline analysis. In order to derive valid findings, a benchmark design must meet several general requirements such as fairness, portability, repeatability, and reproducibility [19, 25, 60, 92].

2.1.1 Application Benchmarks

Application benchmarking is used for evaluating non-functional properties of an SUT by deploying the respective system and all related components in a production-like test or staging environment and stressing them with an artificial but realistic workload [19, 25]. Thus, application benchmarks are often seen as the gold standard, because they define how to evaluate the respective systems using a realistic load in the actual runtime environment and, depending on the use case, also using specific load scenarios (e.g., increased visits and checkouts during the Christmas season). A well-designed application benchmark can provide answers to many



Figure 2.1: Application benchmarking consists of three phases: initialization, execution, and analysis.

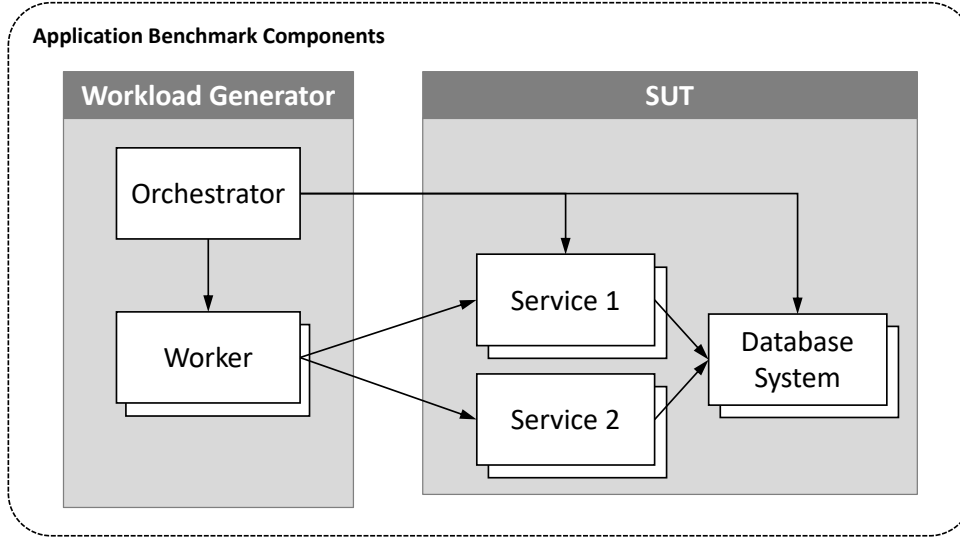


Figure 2.2: Complex application benchmarks require multiple orchestrated workers to generate a realistic and reasonable workload.

performance-related questions and can also be used to compare different versions of an SUT. This is particularly relevant in the context of this thesis, in which a dedicated benchmarking step is envisioned as part of a CI/CD pipeline [74, 183]. On the other hand, however, using application benchmarks for detecting performance regressions is expensive, complex, and time-consuming. Besides the setup and configuration of all relevant components, which can already take a considerable amount of time, all experiments need to run for a certain duration and usually need to be repeated several times to get reliable results, especially in cloud environments [25, 34, 114, 175].

When designing an application benchmark, it is necessary to think in detail about the specific requirements of the application benchmark and its objectives. When defining (and generating) the workload, many aspects need to be addressed to ensure that the requirements of the benchmark are not violated and to guarantee a relevant result later on [19, 25, 60, 92]. This is especially difficult in dynamic cloud environments, where performance variations inherent in cloud systems and other cloud-specific characteristics make it challenging to reproduce results [50, 60, 120, 130, 153].

Figure 2.1 illustrates the three phases of executing an application benchmark and Figure 2.2 shows the involved components during a benchmark run. The setup phase defines and initializes the potentially distributed SUT and workload generator including all components. This can be

done with the assistance of automation tools, such as MockFog [81, 82] or Héctor [14]. However, automation tools still need to be configured first, which further complicates the setup. During the benchmark run, all components must be monitored to ensure that there is no bottleneck within the benchmarking system, e.g., to avoid evaluating the performance of the benchmarking client machine instead of the maximum throughput of the SUT. Finally, the collected data must be transformed into relevant insights, usually in a subsequent offline analysis [25]. Taken together, these factors imply that realistic application benchmarking is usually expensive in terms of both time and money.

2.1.2 Microbenchmarks

Microbenchmarks are typically defined in just a few lines of code and evaluate an SUT on function¹ level by repeatedly calling the respective function under test with artificial parameter values for a specified duration and a specified number of iterations [113, 114]. Thus, a microbenchmark is used for benchmarking individual critical or frequently used functions on a smaller scale (hundreds of invocations) to ensure that there is no performance drop introduced by a code change, or to estimate rough function level metrics, such as average execution duration or throughput. Multiple microbenchmarks together form a microbenchmark suite that can be used to evaluate performance metrics of a single component in a distributed SUT, rather than benchmarking the entire SUT at once. Instead of (possibly) compiling, starting, and configuring various components, it is usually enough to compile the corresponding code files and start the microbenchmark suite with the respective configuration. Similar to unit tests, microbenchmarks can even be run inside the local development environment. Microbenchmarks are thus usually easier to set up and to execute because there is no complex SUT to initialize, and executing a single microbenchmark takes considerably less time than running an application benchmark. However, they cannot cover all aspects of an application benchmark such as the integration of components, and depending on the specific use case, they are typically considered less relevant individually because it is unclear whether they cover relevant parts of the production system or whether detected performance changes will affect the production system [85].

2.2 Continuous Integration and Continuous Deployment

Continuous Integration and Continuous Deployment are two modern paradigms that aim to improve, automate, and accelerate the software development process leading to shorter release cycles [64, 91]. Continuous Integration defines the process of integrating new software changes into the main version, including adapting and running corresponding test cases that ensure the software is extensively tested before it is merged into a production branch of a system. The term Continuous Deployment refers to the automated process of releasing and deploying new software versions. Once a new release has been thoroughly tested in the Continuous Integration

¹We use the term *function* to refer to any form of subroutine, no matter how they are called in the respective programming language.

process, it is automatically rolled out to the production system so that frequent daily releases are possible; this shortens the release cycle. Both processes are designed to run multiple times per day, depending on how many features are implemented per day and the release policy. Thus, 10 minutes is a guiding value for the total duration of both processes so that developers can get early feedback on their software changes. In practice, however, this is not always realistic, so that multi-tiered deployment pipelines are usually used instead. Here, after a first integration stage with integration and component tests, the second level with long-running tests is not always executed. Often, it is run overnight or directly before releasing a new software version.

2.3 Continuous Benchmarking

Continuous Benchmarking aims to not only verify functional properties in CI/CD pipelines, but also to ensure non-functional properties such as performance before releasing a new software version [44, 45, 61, 95, 155, 183, 185]. Because both the execution of application benchmarks and microbenchmark suites can often take several hours, this step is more suitable for CI/CD pipelines that run overnight or for a new release. To avoid maintaining the required hardware and infrastructure for the benchmarks, renting it from cloud providers as needed and executing the benchmarks in cloud environments is a good option, which also provides a realistic environment as software nowadays often runs in cloud environments. Substantial, continuous and random performance variations, which are typical in cloud environments, however, make it difficult to execute relevant benchmarks and derive reliable findings.

2.4 Handling Performance Variations

Performance variations in cloud environments are caused by, e.g., sharing hardware resources with other clients, the way resources are provided by the cloud provider, and both hardware and software updates [50, 60, 120, 130, 139, 153]. In order to obtain meaningful results nonetheless, benchmarking experiments should run for a sufficient duration to compensate for short-term performance variations during the experiment run and should be repeated several times to ensure the reproducibility and reliability of the measurement results. When comparing two alternatives such as two successive software versions, both should be evaluated concurrently or right after each other, as results from a week ago may already be outdated, e.g., due to software or hardware updates on the cloud provider side.

The concurrent benchmarking of two alternatives/versions in the same environment is called **Duet Benchmarking** [32, 33]. Here, both alternatives are deployed using the same hardware, virtual machines, network infrastructure, etc., and both benchmarking experiments then run simultaneously. This ensures that both alternatives are exposed to the same performance variations at the same time and consequently enables a more reliable comparison of the measurement values recorded for both SUTs. Executing application benchmarks for two versions of a single microservice, for example, involves deploying both SUT versions in separate containers on the

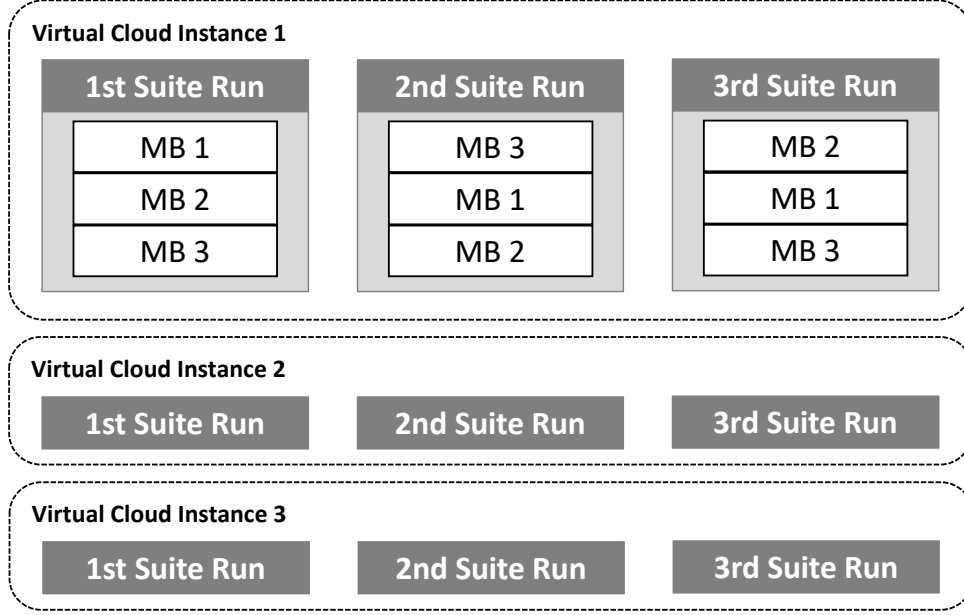


Figure 2.3: Performance variations in cloud environments are addressed by running the microbenchmarks in RMIT execution order, repeating the suite runs multiple times, and on multiple virtual cloud instances.

same VM. During the execution phase, two workload generators deployed on a second VM then run the same workload simultaneously. For the evaluation of the results, it is important to ensure that only measurement values for which both services were actually under load at the same time are included. For example, if one version is considerably faster, the application benchmark run ends earlier and releases resources for the slower running version, which skews the values.

Although duet benchmarking can also be used for the execution of microbenchmark suites, the simultaneous execution of short-term running microbenchmarks is complex, partially because the two executions must synchronize after each run of a single microbenchmark. Thus, microbenchmark suites usually follow the **Randomized Multiple Interleaved Trials (RMIT)** execution order and are executed on multiple virtual instances [1, 2, 26, 113, 114, 117]. Figure 2.3 illustrates this execution of microbenchmark suites in cloud environments: The microbenchmarks in a suite are executed in a random order to mitigate caching effects, the suite is executed several times in a row to evaluate each microbenchmark at different times, and each experiment is repeated on different VMs to average out the effects of performance variations.

Ultimately, it is important to consider performance variations in the analysis phase and express them. This can be done, among other things, by specifying Confidence Intervals (CIs). Computing CIs allows assessing the quality of the measurement results, since a 3% to 5% slowdown in response time between two software versions is more accurate than a range of 1% to 9%. However, it is important to be careful when choosing the calculation method, as many distributions of measurement values in benchmark experiments do not follow a normal distribution. In this thesis, we use a bootstrapping methodology for analyzing the experiment results that implements hierarchical random re-sampling with replacement [102].

2.5 Microservices

Microservices are independently deployable and scalable components [52, 98, 121, 140]. In contrast to a monolithic system which combines all application logic in a single artifact, the microservice architecture splits the logic into a suite of services that communicate over the network. This separation enables different parts of an application (i.e., individual services) to evolve and operate independently of each other (e.g., horizontally scaled). Within an application, individual services can be written in different programming languages or use different storage technologies, resulting in a heterogeneous environment. Being separate deployment units, individual services can independently be shut down, replaced or updated at will, or new service instances can be deployed at runtime to counteract performance bottlenecks. Given these characteristics, all services must be designed to tolerate failures, as no service can expect correctly typed data or assume that a required service is always available.

Microservices communicate over the network, relying on a networked Application Programming Interface (API). APIs can differ in the communication protocols (e.g., TCP, HTTP) and data formats (e.g., JSON, binary data, XML) they rely on. In this thesis, we focus on APIs following the REST architectural style. REST APIs evolve around resources identified by hierarchical Uniform Resource Identifiers (URIs), and use HTTP methods to interact with these resources (e.g., `POST` to create one or `GET` to receive one). REST APIs do not rely on client state (stateless), and evolve around the communication of resource representations (typically in JSON or XML) between clients and servers [157].

Richardson’s maturity model [63] categorizes REST APIs into three levels: While level 0 APIs use HTTP only to tunnel requests to an endpoint, level 1 introduces resources which can be addressed following hierarchical URIs. Level 2 additionally demands that APIs use HTTP verbs to indicate whether to create (`POST`), get (`GET`), update (`PUT` or `PATCH`), or delete (`DELETE`) a resource. Finally, level 3 inserts links (URIs) to corresponding services and or resources into the server responses at runtime, realizing *RESTful* APIs. In this thesis, we assume APIs to comply at least with level 2 of this maturity model – specifically, we rely on the use of HTTP methods for defining abstract operations.

In addition to human-readable API documentation targeting (client) developers, REST APIs are often described in a machine-understandable way using description files such as OpenAPI² or RAML³. For the sake of simplicity, we decided to only consider OpenAPI in our work as possible interface contract.⁴ OpenAPI files are written in YAML or JSON and describe where to reach an API, available operations, its expected inputs, and possible outputs. Although the current version, OpenAPI 3.0, supports so-called link definitions to express relationships between two requests, they are not designed to describe complex interaction patterns which we develop and present in this thesis.

²<https://swagger.io/docs/specification/about/>

³<https://github.com/raml-org/raml-spec/>

⁴Translations between formats are possible, using for example <https://apimatic.io/transformer>.

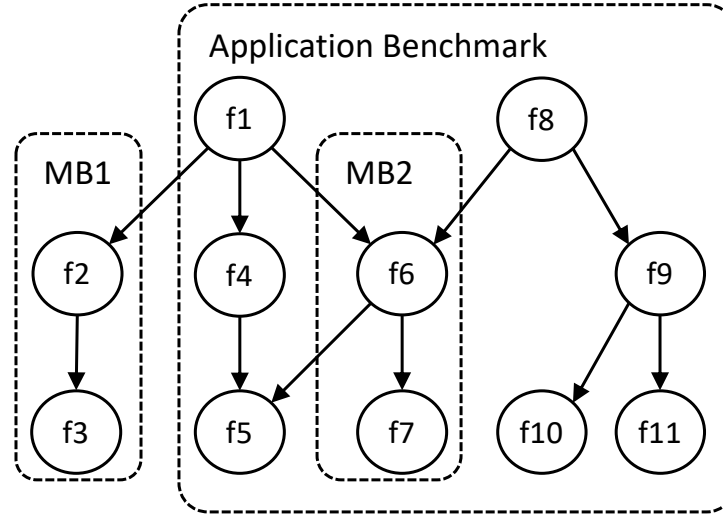


Figure 2.4: A software call graph illustrates internal function calls.

2.6 Software Call Graphs

Software source code is organized into classes and functions. At runtime, executed functions call other classes and functions, resulting in a program flow that can be represented as a call graph (see Figure 2.4) [138, 160]. This graph shows a small example code with eleven functions and illustrates which functions call which other functions. A call graph also provides additional meta information such as the duration of the executed function (not visualized in Figure 2.4). Regardless of whether software is evaluated by an application benchmark or microbenchmark, both types evaluate the same source code. Since an application benchmark is designed to simulate realistic operations in a production-near environment, it can reasonably be assumed that it can serve as a baseline or reference execution to quantify relevance in the absence of a real production trace. On the other hand, microbenchmarks are written to check the performance of individual functions and multiple microbenchmarks are bundled as a microbenchmark suite. If these graphs are available for both an application benchmark and the respective microbenchmark suite, it is possible to compare and analyze the flow of both graphs. In this thesis, we use this relation between both benchmark types to optimize microbenchmark suites, among other things.

2.7 Function as a Service Platforms

All major cloud providers such as AWS⁵, Google Cloud⁶, or Azure⁷ offer FaaS solutions where users only have to take care of their source code (functions) while the underlying infrastructure and environment are abstracted away by the provider [17, 56, 173]. FaaS applications are composed of microservices implemented as single functions and are deployed on a FaaS platform that handles almost every aspect of the function life cycle, such as deployment, execution, and

⁵<https://aws.amazon.com/>

⁶<https://cloud.google.com/>

⁷<https://azure.microsoft.com/>

automatic scaling. Developers do not have direct control of the infrastructure and can only define high-level parameters, such as the region in which the function should run or memory size. Due to this, FaaS platforms are easy to use but comparing cloud platform performance is challenging, as the cloud variability is further compounded by an additional, unknown infrastructure component [15, 118].

Automated Workload Generation for Benchmarking Microservices

Benchmarking microservice applications is hard as there are complex interactions and frequent updates. First, designing and implementing a relevant benchmark for a microservice application with multiple services is difficult. While there are a variety of tools in domains such as database benchmarking which leverage the common interface of database systems to achieve repeatability and portability (e.g., YCSB [19, 40, 50]), interfaces and supported operations vary with every microservice, making it impossible to find a general interface for microservice benchmarking. Second, as microservices evolve over time, interface changes will break ad-hoc benchmarks that a developer might have implemented.

In this chapter, we propose an approach to benchmark REST-based microservice applications with multiple services in which developers can specify their benchmark workload through abstract interaction patterns. We do this based on the machine-readable interface descriptions of the respective microservices and resolve the actual workload at benchmark runtime. Our algorithm identifies links between microservice operations and resolves the specified abstract patterns into application-specific interaction sequences which can span multiple microservices. This significantly reduces the effort for developers while still ensuring that benchmarks fulfill important characteristics, namely that they are relevant, repeatable, portable, verifiable, and economical [25].

This chapter presents the following contributions:

- We propose an approach for benchmarking of REST-based microservice applications based on abstract and reusable interaction patterns.
- We present a proof-of-concept prototype implementing our pattern-based benchmarking approach.
- We evaluate our approach by benchmarking an open-source microservice application to demonstrate how little manual effort is needed for this.

Our evaluation shows that our approach can be applied to typical RESTful microservices and abstract interaction patterns can be mapped to interaction sequences with the services. Please, note that providing a complete pattern catalog is beyond the scope of this thesis, but applying our approach in practice obviously requires a comprehensive set of interaction patterns.

This chapter contains (partially adapted) material from our published research articles [75, 78] and is structured as follows: First, we present our pattern-based benchmarking approach using

service description files in Section 3.1. Section 3.2 then evaluates the idea using a microservice application with connected services. Finally, we discuss our approach in Section 3.3 and summarize our contributions in Section 3.4.

3.1 Pattern-Based Benchmarking

Our pattern-based benchmarking approach relies on the observation that there are sequences of interactions with resources in REST APIs which recur across APIs. One common example is to list resources of a specific type (e.g., by performing `GET .../customers`), to then retrieve information about one specific resource (e.g., by performing `GET .../customers/1`), and finally deleting that resource (e.g., by performing `DELETE .../customers/1`). Based on this observation, we argue that it is possible to automatically generate benchmarking workloads from

- an abstract description of such patterns and
- a description of how to interact with each of the microservices' APIs (e.g., in OpenAPI format).

In this section, we first describe the challenges in generating such a pattern-based workload. Next, we introduce our pattern-based solution in detail and finally give an overview of our approach's system design.

3.1.1 Challenges

We have identified the following three major challenges facing our approach:

- (A) The first challenge is to define patterns and workloads for arbitrary microservice applications, including the total number of requests and their distribution across patterns.
- (B) Once defined, the individual patterns must be mapped to the individual services and their respective operations. Here, an abstract pattern composed of multiple operations (e.g., `listResources`) must be linked to service-specific resources (e.g., a list of `customers` or `catalog items`) and its operations (e.g., `GET .../customers` and `GET .../catalogue`).
- (C) Finally, the abstract requests must be filled with concrete parameter values depending on the interface definition, which can be challenging for request sequences. This is because parameter values may depend on the outcome of prior requests, potentially to a different service.

3.1.2 From Interaction Patterns to Service-Specific Workloads

The key idea of our approach is to define an abstract workload separately from the services in an application itself and to resolve the actual service-specific workload at runtime. To address the challenges outlined above, which also have interdependencies, we divide this process into six steps (described in detail later). While challenge A is solved in the first two steps, steps 3 and

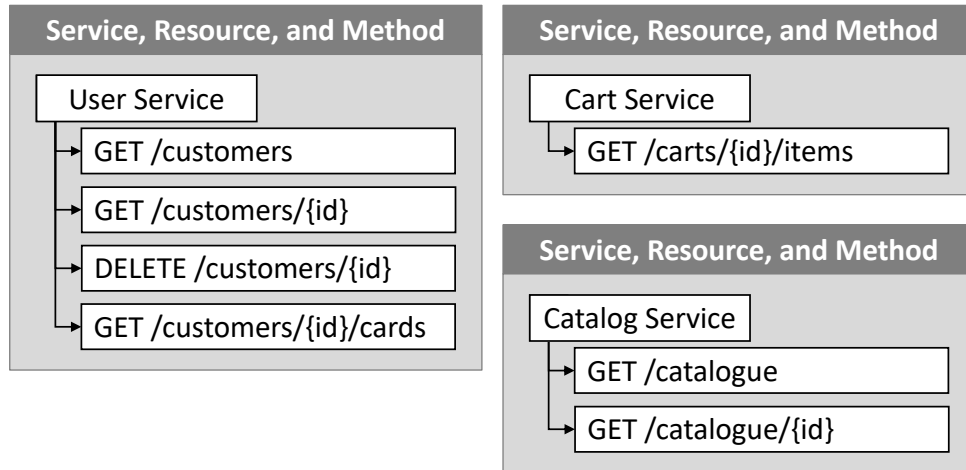


Figure 3.1: The example application with three microservices we use to explain our matching process.

4 aim to cope with the difficulties described in Challenge B. Finally, challenge C, the actual workload generation, is covered in the steps 5 and 6.

1. **Pattern Definition:** Define abstract interaction patterns.
2. **Workload Definition:** Enhance pattern definition and specify frequency and ratio of requests.
3. **Binding Definition:** Optionally, override default binding behavior.
4. **Binding Enactment:** Bind abstract interactions to concrete microservices and their operations.
5. **Workload Generation:** Create application-specific workload.
6. **Benchmark Execution:** Run the workload against the SUT and substitute values at runtime.

In the following, we will explain these steps using the example application listed in Figure 3.1. In the example, the customer and cart service use the same ID field (once as customer ID and once as the ID for the cart of the respective user).

Step 1 – Pattern Definition:

The first step is to define abstract interaction patterns that are independent of the microservices but still applicable to them.

As defined by the second level of Richardson’s maturity model, the typical REST CRUD operations can be mapped to HTTP methods: A new resource can be created at a resource endpoint by calling the POST HTTP method and accessed following a path structure at that endpoint. Individual resources can be read (HTTP GET), updated (HTTP PATCH or PUT), and deleted (HTTP DELETE). Finally, multiple resources can be listed by sending an HTTP GET to a list operation (e.g., GET ../search) which potentially may return multiple items. In the very first step of our

3.1. Pattern-Based Benchmarking

Operation	Description
CREATE	Creates and returns an item.
READ	Reads an item based on some filter (e.g., an ID) and returns the requested item.
SCAN	Reads multiple items based on some (optional) filter (e.g., a keyword) and returns the results.
UPDATE	Modifies an item based on some filter.
DELETE	Deletes an item based on some filter.

Table 3.1: List of currently supported abstract operations which are combined to form abstract interaction patterns.

Step	Operation	Input	Selector	Output
1	SCAN	-	-	list
2	READ	list	RAND	item
3	DELETE	item	-	-

Table 3.2: Abstract interaction pattern which requests multiple resources, reads one random item from the resulting list, and finally deletes the selected item.

approach, we use these basic interactions to define the abstract operations shown in Table 3.1 which we will later use to bind abstract patterns to concrete service resources.

While almost all of these interactions refer to a specific single resource, read operations can request either a single resource or multiple resources; we therefore distinguish the two read operations **READ** (single) and **SCAN** (multiple). Most operations require some filter information about the item to be read, updated, or deleted. These do not only include an ID or key of the requested resource, but also further domain-specific values if multiple items should be read (**SCAN**). Furthermore, we introduce selectors as part of this filter information: If a list of items serves as input for an operation, the selector determines which item to pick from that list (e.g., first, last, or random item).

Our abstract operations already cover the common CRUD interactions with REST services. If necessary, our approach can be extended with additional basic operations. Using the basic operations from Table 3.1, we can now compose an interaction pattern as a sequence of abstract interactions, each defining input and output values.

The complete interaction pattern for the abstract example from the beginning of this section is shown in Table 3.2: First, a **SCAN** operation determines all available resources on a service resource endpoint and stores the resulting values in a variable called `list`. Next, an individual value is selected by a random selector from this list, the corresponding resource is read (possibly from another microservice), and stored into a variable called `item`. Finally, the selected item is deleted.

Step	Operation	Input	Selector	Output
1	SCAN	-	-	list
2	SCAN	list	RAND	sublist

Table 3.3: Abstract interaction pattern which queries a list of resources and then requests all associated resources for one random item.

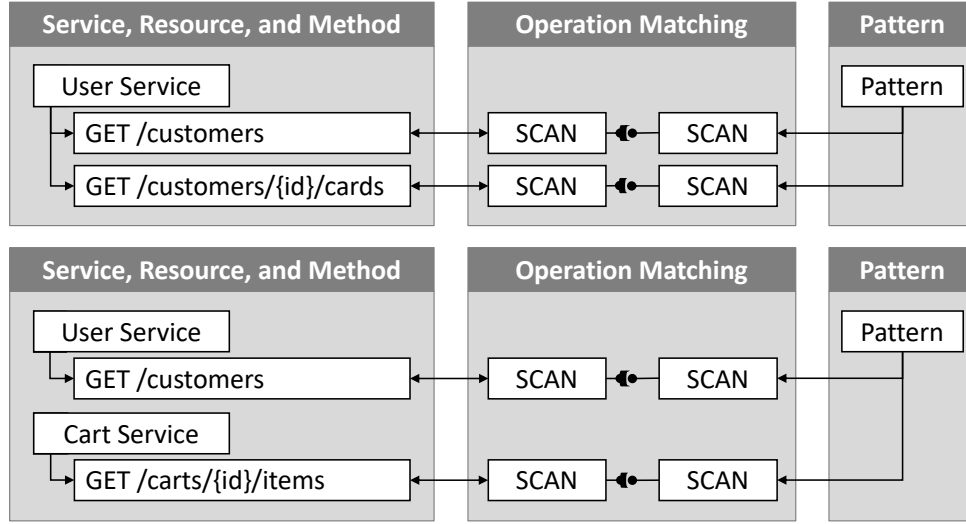


Figure 3.2: Matching the pattern from Table 3.3 to two different interaction sequences.

Table 3.3 motivates a more complex example with sub-resources. The interaction starts again with a **SCAN** of available resources and the outcome is written to a variable called **list**. Applied to a microservice application, this could, e.g., request a list of users which each have a number of sub-resources. Next, a random item from that list serves as input for a subsequent **SCAN** operation which requests all (sub-) resources for the chosen item, e.g., a list of all credit cards for the selected user or a list of all items in the user's shopping cart (see Figure 3.1). Figure 3.2 shows how the binding of the pattern shown in Table 3.3 would later be resolved for the two example sub-resources: In the upper part, the selected customer ID is used to retrieve all credit cards of the selected user from the same microservice as sub-resources. In the lower part, the ID is used to list all items in the selected user's shopping cart via a second microservice.

Step 2 – Workload Definition:

The next step is to specify the actual workload which should be executed against the SUT. Similar to the business transactions in BenchFoundry [19], a pattern definition can include optional conditions for individual patterns (e.g., waiting times between operations to mimic realistic user behavior). Comparable to YCSB [40], which defines a workload based on a total number of operations as well as the respective share of each database operation, we define a workload based on three pieces of information: first, the list of all patterns which should be used; second, the total number of pattern invocations; and third, the share or weight of each pattern. At execution time, multiple such patterns are usually executed in parallel.

In the following, we will refer to the abstract interaction patterns defined in Step 1 and the workload definition as **pattern configuration**.

Step 3 – Binding Definition:

While our matching algorithm automatically identifies links between requests, there are several cases in which these automatic bindings should be suppressed or require additional information. In this optional adjustment step, developers can manually exclude specific service operations from the matching algorithm or define links between microservices. If used, this provides additional information to the default binding process described in Step 4 below.

As one usage scenario, microservice applications sometimes provide multiple resource endpoints (e.g., `/customers` and `/catalogue`) which can be used by the benchmarking client for an interaction pattern. By default, all possible resource endpoints and operations are used by the benchmark. When, however, the automatic binding from pattern to resource and operation should be suppressed (e.g., in case that only the `/customers` resource endpoint should be benchmarked), this can be achieved by excluding the respective service endpoint for a subset of the patterns.

As another usage scenario, a manual definition can also be used to link parameters of two services with the goal of enabling interaction sequences which span multiple microservices of the same application. For example, if the resources of the user service can be accessed through a parameter called `id` and another service uses the same IDs to maintain the shopping carts for the respective user, then our binding algorithm needs this manual link definition to match an abstract pattern to these two services, i.e., to clarify that the second ID is indeed the user ID and not the cart ID. Here, semi-automatic approaches based on text similarity measures can support developers to define these links. For simplicity of presentation, however, we will focus only on manual links between services.

Step 4 – Binding Enactment:

As already described above, our approach for automated binding enactment relies on a number of key ideas: First, REST operations can directly be mapped to the corresponding HTTP methods, e.g., a `CREATE` is mapped to an HTTP `POST`. Second, a microservice which complies with the second level of Richardson’s maturity level exposes its operations in a way that is compliant with the REST operation semantics, e.g., creating a new user will always be exposed as a `CREATE` which can then be mapped to `POST`. Third, the input and output of these operations as well as the corresponding data schema are described in the interface description file, i.e., in our case, the OpenAPI file, so that we can link the output of one operation to the input of another. This allows us to create the cross-operation links in our interaction patterns. Finally, the interface description also provides information on where to find the microservice, hence, we can actually invoke it once we have completed all the mappings as described above. Based on the reasoning above, we can automatically generate a binding between an interaction pattern and the actual

Algorithm 1: Generate Interaction Sequences

Input: *pattern* - abstract interaction pattern
Input: *specs* - list of interface descriptions
Result: *sequences* - supported interaction sequences

```

1
2 /* Initialization */
3 AO = IdentifyAbstractOperations(specs)
4 root = CreateRootNode()
5
6 /* Matching */
7 for i ← 0 to pattern.size do
8   a = pattern[i]
9   candidate = FindByAbstractOperation(a, AO)
10  foreach node ∈ GetNodesByLevel(i, root) do
11    foreach o ∈ candidate do
12      if AreDependenciesResolvable(node, o) then
13        AddChildNode(node, o)
14      end
15    end
16  end
17 end
18
19 /* Sequence Extraction */
20 sequences = ExtractSequences(root)

```

sequence of HTTP calls – subject to the conditions above, e.g., that creating a new user is not exposed as a PUT.

A pattern can be mapped to a microservice application if there is at least one supported interaction sequence for this pattern within the application. As shown in Figure 3.2, this can either affect a single service only or an operation result can be used to interact with another service, thus, enabling a cross-service benchmark run.

Algorithm 1 describes the algorithm we use to match and generate the application-specific interaction sequences; it has the three phases Initialization, Matching, and Sequence Extraction.

Step 4.1 – Initialization: First, we analyze the interface description of all services and determine the abstract operation for all resource paths and operations (line 3). While it is easy to determine the operation for creation, modification, and deletion as they directly map to an HTTP method by convention, this is more difficult for the **SCAN** and **READ** operations. Here, we have to inspect the resource path a bit further and check if it ends with an input parameter. If so, the parameter refers to a key or an ID and supports the **READ** operation; If the resource path does not end with a parameter and returns an array of items, it can be bound to the **SCAN** operation.

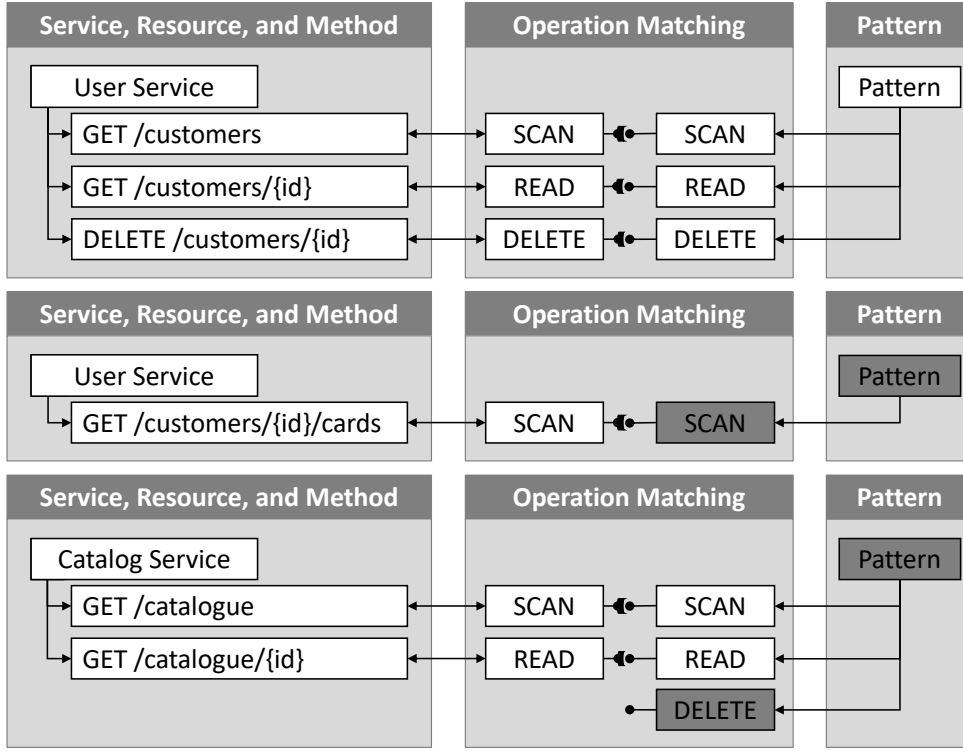


Figure 3.3: Mapping a pattern to interaction sequences, only one sequence (list all users, read one randomly-selected user, and delete the selected user profile) supports the example pattern.

Next, we initialize a virtual root node of a tree data structure which we will use to store intermediate results of pattern matching (line 4). The paths in the tree from root to leaf will later hold our interaction sequences.

Step 4.2 – Matching: In the matching phase, we iterate through the abstract operations in the pattern and select all service operations of the same abstract type as a list of operation candidates (lines 7-9). For the abstract interaction pattern in Table 3.2 and our example application, the first operation can be mapped to three service-specific operation candidates: listing all users, listing all credit cards of a user, and listing all items of the catalog service (see Figure 3.3).

Next, we iterate over all leaf nodes *node* at the specified level in the tree (for the first pattern operation this is the root node) and check for every candidate operation *o* whether its dependencies can be fulfilled on the path from *root* to *node* (lines 10 - 16). Candidate operations that do not require input values (such as listing all users) have no dependencies and can, therefore, always be used. In that case, we simply add this operation as a child node to *node*. If, however, there are dependencies, we verify that the required information can be obtained from previous requests on the path from *root* to *node*. In our example, this affects the sequence starting with listing the credit cards of a user: As it is a candidate for the first operation, there is no information available on the required user ID, see Figure 3.3. Any operation for which all dependencies can be resolved is added as a child node; all other operations are disregarded (lines 12 - 14). In this step, it is also possible to define additional conditions that an operation needs to fulfill before it can be

added as a child node; in our prototype, for instance, we have implemented a filter that only adds operations that target microservices already used on the current path or microservices for which an explicit link has been defined in step 3. We do this to limit the number of interaction sequences, but it is not strictly necessary to do so.

We execute this for all operations in the pattern and, thus, gradually build our tree data structure. In the third example in Figure 3.3 for instance, we can see that the first two operations of the example pattern can be matched against the catalog service. The third operation (**DELETE**), however, cannot be matched since the only available service-specific **DELETE** requires a user ID as input which cannot be obtained from the two previous operations (listing all catalog entries and reading a specific catalog item).

Step 4.3 – Extract Sequences: Finally, we extract the interaction sequences from our tree. For this, every path from the root node to a leaf that – excluding the root node – contains the same number of nodes as the pattern has operations is a valid interaction sequence. Shorter paths are incomplete interaction sequences where one or more pattern operations could not be matched; these can be discarded.

When at least one interaction sequence per pattern has been found, the binding is saved and represents the automatic binding for the combination of patterns, binding definition, and interface descriptions. In some cases, the binding algorithm may find sequences that are not relevant or not desired in the respective use case. These sequences can either be deleted, deactivated manually, or can be used to create additional load on the SUT.

Step 5 – Workload Generation:

With the previously created binding and the interface descriptions, we can finally generate the benchmark workload by building HTTP requests that follow the interaction patterns and the restrictions in the interface description files. First, each pattern operation can be resolved directly to an HTTP method based on the binding. Next, we can fill the required parameters and request body content of each request by inspecting the interface description of the respective microservice and generating random values for all interactions: In OpenAPI, complex parameter values and request bodies are described using JSON Schema.¹ We can use these descriptions to generate the required data items filled with random values. Moreover, we can generate use case specific values such as product names or Bitcoin addresses by augmenting the service description with special keywords.

As stated above, some content of the individual requests may depend on the returned values of previous calls (e.g., identifier values). These values must be injected later in the benchmark execution phase (Step 6) for which we use special markers. Nevertheless, once the required number of pattern executions has been generated, the workload can be persisted and reused across several benchmark runs even if the generated workload is incomplete in that sense.

¹<https://json-schema.org/>

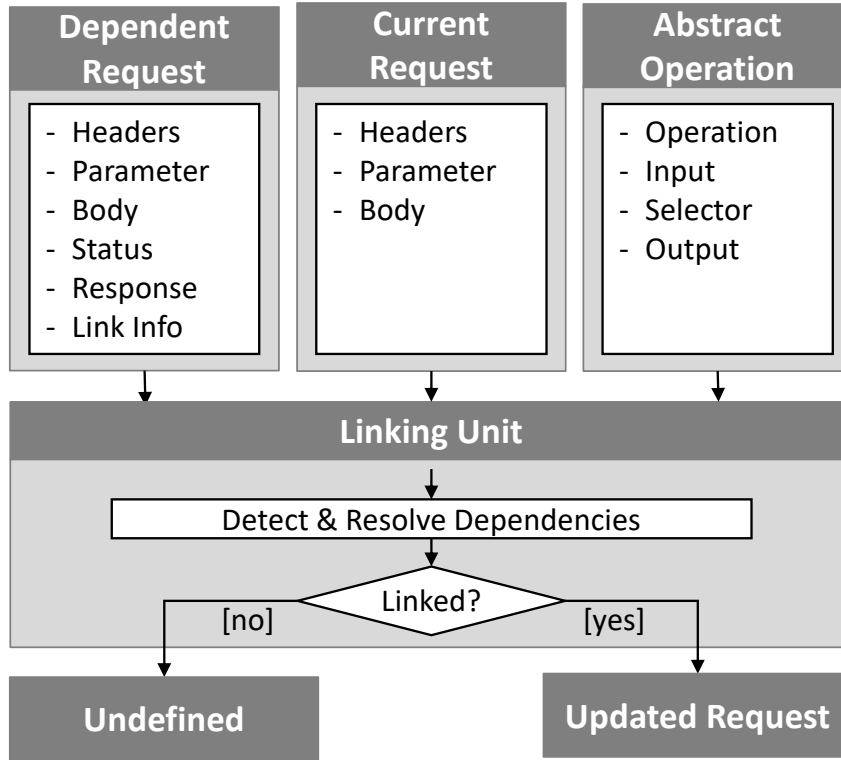


Figure 3.4: Linking two related requests based on the content. If a link is detected, the successive request is updated and returned.

Step 6 – Benchmark Execution:

As already stated above, some values of the workload must be replaced during the execution if there are dependencies between requests. For these values, there are many sources depending on the specific operation: A **CREATE** operation could, for example, return the ID of the created item or respond with an HTTP 200 status code if the ID was part of the request and the item was created successfully. Depending on the implementation, the subsequent **READ** request needs to select the ID from the response or the request body of the preceding request. However, this is only possible if the response had an HTTP 200 status.

For such purposes, we have designed the **linking unit** interface illustrated in Figure 3.4: A linking unit tries to find and resolve dependencies based on the preceding requests (including message body, response, etc.), the current request (including the values to be replaced, e.g., a parameter), and the abstract pattern operation. If a linking unit detects a link, it resolves the dependency, e.g., by replacing the placeholder value in the current request body with some value from the parameters of the previous one, and returns the updated request or **undefined** otherwise. This way, it is possible to apply multiple linking units sequentially until a dependency has been resolved. It also allows us to order the application of several units hierarchically, such as general or very service-specific units first.

Currently, we have identified four different types of linking units but additional, potentially service-specific, custom units can be added:

- **OpenAPI Link Linking Unit:** OpenAPI 3.0 documents can define links that describe further operations and their content after a query. This linking unit inspects the link definitions of the preceding request and replaces the values of the current request accordingly.
- **Binding Linking Unit:** This unit resolves the links manually defined in the binding definition (see step 3).
- **Parameter Name Linking Unit:** Individual resources can often be accessed by following a path structure in REST interfaces, e.g., `/ {username}`. These parameters were initially filled with placeholder values in Step 5 which have to be adjusted now to access actual resources. This linking unit searches for these values in the preceding request based on the parameter name. If exactly one element with this name as key is found in the request (either in parameter values, request body, or response), this value is used in the current request. If there are multiple values to choose from, which one is chosen is determined by the selector (e.g., select a random value).
- **ID Linking Unit:** In some cases, parameter names in the preceding and current requests do not match exactly. For example, a user is created with a field named `id` in the request body and individual users can be accessed via the path `/ {userID}`. This linking unit resolves dependencies by searching field names for the substring "id" and replacing values in the same fashion as the parameter name linking unit.
- **Custom Linking Unit:** Finally, as dependencies cannot always be detected and resolved with our default linking units, developers can also define custom and service-specific linking units that can be added to the application chain of units by implementing the linking unit interface.

3.1.3 System Design

Our system design comprises a number of components; these, along with the corresponding steps, are shown in Figure 3.5. The Pattern Binder creates service-specific interaction sequences based on API description files, a pattern configuration, and optional binding definitions (steps 1 to 3). Once the Pattern Binder has bound every pattern to at least one interaction sequence (Step 4), the binding can be stored and, if necessary, adjusted manually (e.g., if the automatic algorithm identified an unusual but possible interaction sequence). Next, the Workload Generator generates the service-specific yet incomplete workload based on this pattern binding (Step 5). As a pattern execution is by definition independent of other pattern executions (otherwise, they should be merged into one pattern), we can parallelize pattern execution and also distribute this execution across a number of Worker Nodes. Similar to a method proposed by Bermbach et al. [19], the Benchmark Manager does this by partitioning the workload into worker packages to enable concurrent execution (the number of packages corresponds to the number of concurrent Worker Nodes), then distributes the worker packages to the available Worker Nodes, manages the (concurrent) benchmark execution, and collects the results. Finally, as our approach is intended for use in CI/CD pipelines [74, 183], the Benchmark Manager compares the observed metrics to

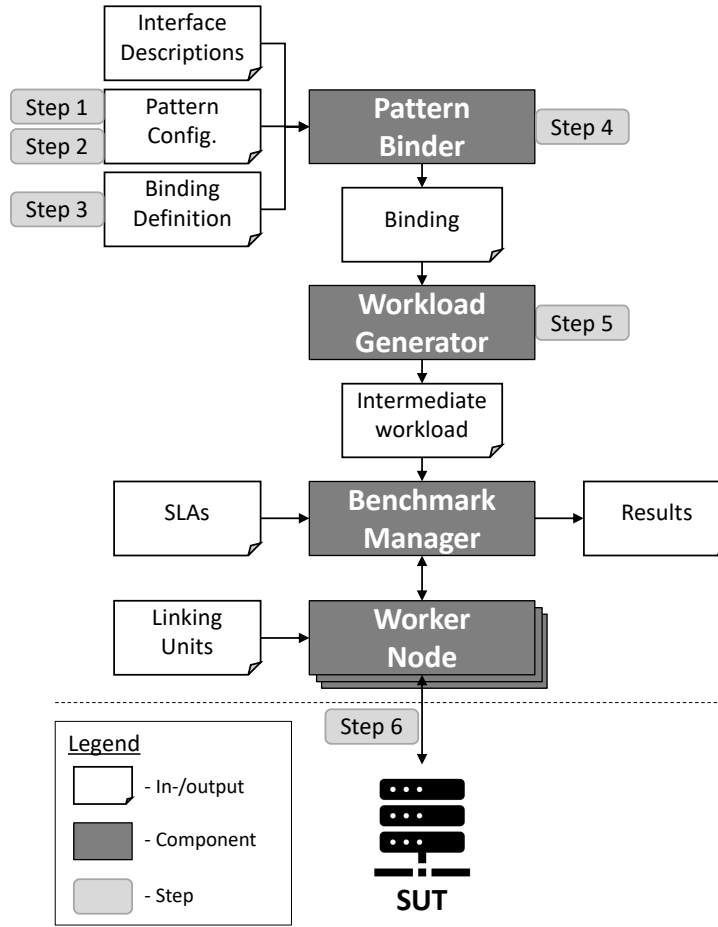


Figure 3.5: Overview of our system design and setup, including input and output documents.

predefined requirements and constraints such as SLAs to ultimately decide on success or failure of the benchmark run.

Within a worker package, requests across patterns can be interleaved as long as requests within a pattern are not reordered. As some requests depend on the outcome of preceding ones (e.g., the `UPDATE` operation requires the resource ID which was part of the result from a previous `CREATE` call), the Worker Nodes cannot simply read the generated workload and run the requests against a service endpoint, but must adapt some values at runtime with the outcome from posted requests based on the linking units (Step 6).

3.2 Evaluation

This section evaluates our benchmarking approach by benchmarking an entire microservice-oriented application with multiple services. As our focus is the general applicability of our approach, the actual measurement results are irrelevant as long as results can be obtained. We first present our proof-of-concept implementation and describe the microservice application that we benchmarked. Next, we describe how we followed the individual steps of our approach to

create and run a pattern-based benchmark workload. Finally, we summarize our evaluation findings.

3.2.1 Proof-of-Concept Implementation

We implemented our approach and system design as an open-source proof-of-concept prototype² written in Kotlin. Our prototype implementation can be integrated in an existing CI/CD pipeline and comprises four components:

1. A Pattern Binder that maps abstract interaction patterns to service-specific operations.
2. A Workload Generator that fills the requests with random values based on the interface description.
3. A Benchmark Manager that orchestrates the benchmark run and aggregates the results.
4. Worker Nodes that execute the workload, resolve dependencies between successive requests using linking units, and measure the execution duration of each operation to report the results.

Our prototype uses the open-source library `json-schema-faker`³ to generate data for the workload. This library allows us to generate the required JSON elements for the individual HTTP requests based on the schema information in the OpenAPI description file. Moreover, our prototype also supports special faker keywords (defined by the library `Faker.js`⁴) which can be added to the OpenAPI file. These additional keywords can be used to generate realistic and scenario-specific workload values (e.g., names, product IDs, or dates).

3.2.2 Sock Shop Microservice Application

We evaluate our approach on a microservice-oriented Webshop⁵ for socks that implements an e-commerce application. In our experiments, we focus on the following three REST services and dependencies (see Figure 3.6):

User Service⁶:

The user service maintains the customer information including usernames, passwords, credit cards, and addresses. Each user has a unique customer ID which is used to access the respective data set as REST resource. Additionally, there are also resource paths for credit cards and addresses. For example, a new address can be created by sending an HTTP POST to `/addresses` and all addresses of a customer can be queried by sending an HTTP GET to `/customers/{id}/addresses`.

²<https://github.com/martingrambow/openISBT>

³<https://github.com/json-schema-faker/json-schema-faker/>

⁴<https://github.com/marak/Faker.js/>

⁵<https://microservices-demo.github.io/>

⁶<https://github.com/microservices-demo/user>

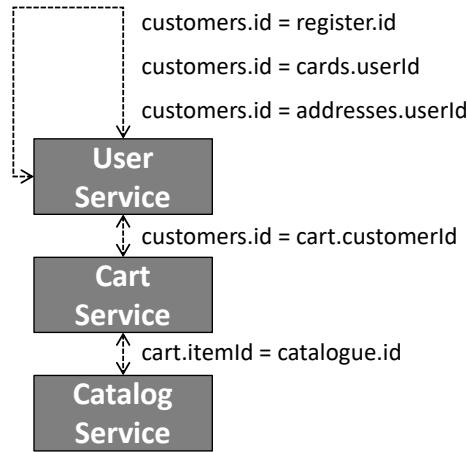


Figure 3.6: Our evaluation focuses on three microservices of the Sock Shop application and their interdependencies.

Cart Service⁷:

Each user has a virtual shopping cart which is a REST resource identified by their customer ID. Items from the catalog service can be added, deleted, or modified.

Catalog Service⁸:

The catalog service provides an interface for retrieving all products available in the shop. A data item consists of an item ID (which is also used for accessing the respective REST resource), a description, tags, and the price of the corresponding product. The service only offers operations for browsing existing products, items can neither be modified, added, or removed.

3.2.3 Experiment

In line with our proposed process, we run the following experiment to evaluate our pattern-based benchmarking approach:

Step 1 – Pattern Definition:

We evaluate the REST microservices discussed above with four self-defined abstract interaction patterns as shown in Table 3.4: First, a simple list pattern which lists available resources and reads an item from that list (LST). Second, our introductory example pattern from Table 3.2 which identifies and deletes a resource (DEL). Third, our more complex example pattern from Table 3.3 which lists sub-resources for a randomly selected root resource (SUBLST). Fourth, another complex pattern which creates a new resource based on two input values (TWOIN).

In all steps where an item needs to be selected from a list, we always use a random selector for reasons of simplicity, but there may be services for which another selector makes more sense,

⁷<https://github.com/microservices-demo/carts>

⁸<https://github.com/microservices-demo/catalogue>

Pattern	Step	Operation	Input	Selector	Output
LST	1	SCAN	-	-	list
	2	READ	list	RAND	-
DEL	1	SCAN	-	-	list
	2	READ	list	RAND	item
	3	DELETE	item	-	-
SUBLST	1	SCAN	-	-	list
	3	SCAN	list	RAND	sublist
TWOIN	1	SCAN	-	-	list1
	2	SCAN	-	-	list2
	3	CREATE	list1, list2	RAND, RAND	item

Table 3.4: An overview of the four interaction patterns which we use in our experiments.

such as picking the latest item. Additionally, we want to emphasize again that these patterns are examples only, as our goal is not to identify a comprehensive pattern catalog.

Step 2 – Workload Definition:

In this evaluation, we aim to showcase the overall functionality and general applicability of our approach, rather than to evaluate the performance of individual microservices. Therefore, we run rather “small” workloads and use only one benchmark run. In practice, however, these parameters would have to be adapted to fulfill usual benchmark best practices, e.g., regarding execution duration [25]. For our experiment, we run a total of 1,000 pattern requests, 250 per pattern. We also run an initial pre-load phase which inserts 1,000 customers including one credit card, one address, and one cart item in advance for each user. The catalog service already offers nine items out of the box.

Step 3 – Binding Definition:

For our experiment, we do not have to manually exclude operations (this may be different in other scenarios). However, we define five manual links for the ID fields of the evaluated microservices as shown in Figure 3.6.

The customer ID, which is the key for accessing the REST resource, is used in four different contexts under different names: Within the user service, the field `id` is used for the `/customers` and `/register` paths. Moreover, the same value is referenced as `userId` in the `/cards` and `/addresses` paths. Besides these links, we define two links which connect the user service to the cart service via the customer ID and the cart service to the catalog service via the item ID.

The diagram illustrates interaction sequences for three different patterns: LST, DEL, and SUBLST. Each pattern is represented by a box containing a list of sequences. Some sequences are shared between patterns, indicated by overlapping boxes.

Interaction Sequences for LST pattern

- 1 GET /customers
- 2 GET /customers/{id}
- 1 GET /cards
- 2 GET /cards/{id}
- 1 GET /addresses
- 2 GET /addresses/{id}
- 1 GET /customers
- 2 GET /carts/{customerId}
- 1 GET /catalogue
- 2 GET /catalogue/{id}

Interaction Sequences for DEL pattern

- 1 GET /customers
- 2 GET /customers/{id}
- 3 DELETE /customers/{id}
- 1 GET /customers
- 2 GET /carts/{customerId}
- 3 DELETE /carts/{customerId}
- 1 GET /customers
- 2 GET /customers/{id}
- 3 DELETE /carts/{customerId}
- 1 GET /customers
- 2 GET /carts/{customerId}
- 3 DELETE /customers/{id}
- 1 GET /cards
- 2 GET /cards/{id}
- 3 DELETE /cards/{id}
- 1 GET /addresses
- 2 GET /addresses/{id}
- 3 DELETE /addresses/{id}

Interaction Sequences for SUBLST pattern

- 1 GET /customers
- 2 GET /customers/{id}/addresses
- 1 GET /customers
- 2 GET /customers/{id}/cards
- 1 GET /customers
- 2 GET /carts/{customerId}/items

Interaction Sequences for TWIN pattern

- 1 GET /customers
- 2 GET /catalogue
- 3 POST /carts/{customerId}/items
- 1 GET /catalogue
- 2 GET /customers
- 3 POST /carts/{customerId}/items

Figure 3.7: All our example patterns can be matched to at least one interaction sequence each.

Step 4 – Binding Enactment:

Here, we bind our example patterns to the target microservices and their resources. Figure 3.7 outlines the resulting binding for each pattern, including our manual definitions from Step 3.

For the **LST** pattern, our binding algorithm identifies five possible interaction sequences. Three of them are limited to the user service (e.g., querying a list of customers and getting the details of a random customer afterwards), one interacts with the catalog service (getting all items and selecting a random one), and one sequence combines the user and cart service (querying a list of customers and getting the virtual cart of one random customer from the resulting list).

The **DEL** pattern can be found in six interactions. The first four interactions start with listing registered customers. Next, one randomly selected customer ID can be used to either get the details for this customer or to get the customer’s shopping cart. Third, the customer ID serves as input to delete either the customer or the cart. The remaining two sequences start with listing all credit cards or addresses. In the first binding iteration, the search is restricted to the respective endpoint and the `/customer` endpoint as no other link has been defined (we discussed this additional condition in Step 4 of Section 3.1). Thus, the `id` fields are not mixed up and the **READ** operation continues to use either the `/cards` or `/addresses` endpoint. The `/customer` endpoint is disregarded because the `customer.id` is not linked to the `cards.id` (but to `cards.userId`). The same holds for the second binding iteration and the final **DELETE** operation.

Our binding for the `SUBLST` pattern identifies three interaction sequences, all starting with listing customers. Next, the randomly selected customer ID can be used to list the user’s addresses, credit cards, or shopping cart items.

The **TWOIN** pattern combines two different inputs to create a new resource. In our evaluated application, this pattern can be used to add items from the catalog to a user’s shopping cart. For the first two operations, it does not matter whether customers or items are listed first as neither operation has a dependency on the respective other one. Thus, our binding algorithm identifies two interaction sequences, one starting with listing customers, the other starting with listing catalog items. Finally, the **CREATE** operation requires one input as a parameter and another input within the request body. Other operations, such as registering a new customer, are not matched because they either expect only one input value or the manually defined service links do not match (e.g., the `card.id` is not linked to the `cart.customerId`).

Step 5 – Workload Generation:

To generate the workload for our experiments with our prototype, we adjust the OpenAPI files slightly for the following reasons: First, we convert⁹ the service descriptions to the current OpenAPI version 3.0 for implementation reasons. Second, we align the description with the actual implementation, as the OpenAPI file is missing a few properties offered by the corresponding implementation. Finally, we add `faker.js` keywords to generate realistic random values, e.g., for names, numbers, and dates.

Step 6 – Benchmark Execution:

We run our benchmark on AWS EC2¹⁰ instances, all in the same availability zone. For our experiment, one instance runs the Sock Shop microservice application including the three evaluated microservices and two instances each run a Worker Node with five threads each to demonstrate the parallelization and scalability features of our prototype. Finally, a fourth instance hosts the Pattern Binder, Workload Generator, and Benchmark Manager.

Our experiment benchmarks three different REST microservices of an example application. As a result, we get pattern execution measurements which include the total pattern duration and the duration of individual requests. These measurements can, e.g., be used to generate box plots for each pattern and interaction sequence. Figure 3.8 shows the latency at the pattern level for our evaluated patterns using box plots¹¹. Again, since the actual results are irrelevant and only show the applicability of our approach, we do not discuss the measured results.

3.2.4 Findings

As described above, our prototype can benchmark typical microservice-oriented applications with minimal adaptation effort. After initializing all application services, our proof-of-concept implementation benchmarks the evaluated and linked microservices (almost) out of the box and only a few minor changes in the description file are needed. Moreover, the abstract workload definition – which, in our evaluation example, is a JSON file with fewer than 100 lines – can

⁹<https://mermade.org.uk/openapi-converter>

¹⁰<https://aws.amazon.com/ec2/>

¹¹Boxes represent the 25%, 50%, and 75% quartiles; whiskers show the minimum and maximum duration for each sequence

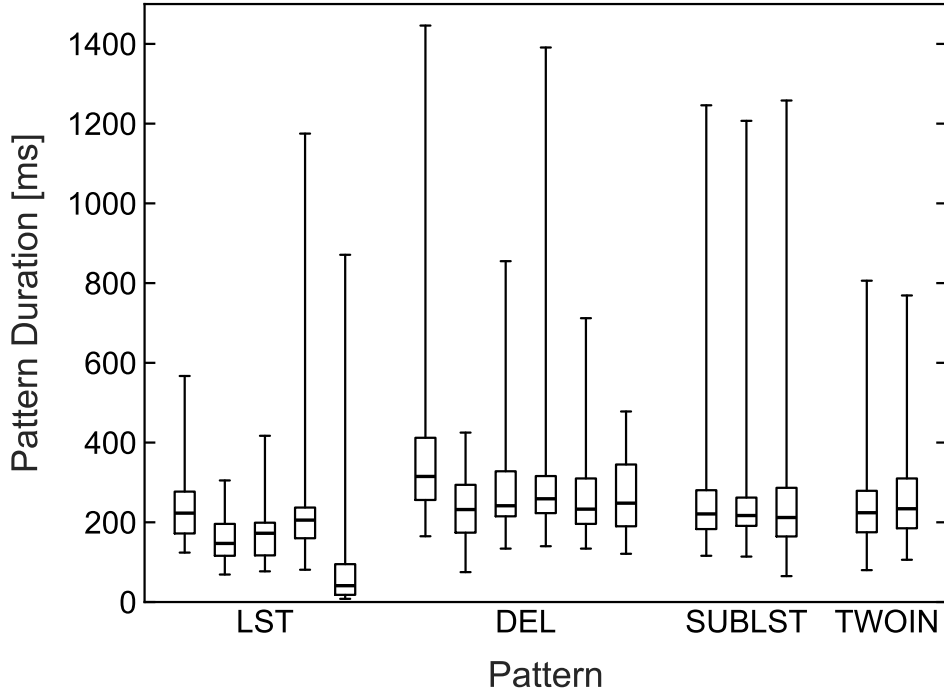


Figure 3.8: Example results for total sequence duration with $n = 250$ measurements per pattern. The box plots use the same order as the interaction sequences in Figure 3.7.

be reused across a variety of services in different versions. The five manual links between the microservices that we had to define are about 30 lines. Unless major breaking interface changes are made, these manual links can be reused across benchmark runs against different versions of the microservice application. Overall, we managed to design and set up the benchmark for the entire microservice applications in less than an hour in total which significantly reduces the effort necessary to benchmark a microservice: there is simply no need anymore to manually implement a benchmark (tool) from scratch which can also be quite challenging [20].

3.3 Discussion

As the evaluation shows, our approach can be used to benchmark REST microservices based on their service description but, nevertheless, there are some points to consider when applying our pattern-based approach and which we want to discuss in more detail here. Moreover, we also present current limitations and propose possible solutions for them.

Usage Scenarios:

While not every pattern and workload definition can be blindly applied to every REST microservice, our approach allows developers to run a benchmark against REST services that share the same characteristics in general, which is helpful in several situations: First, every new service version can be compared to older versions as long as the individual changes do not alter the basic characteristics of the microservice. This is particularly important for use in CI/CD pipelines [45,

74, 183]. Second, if the API changes (e.g., a new parameter is introduced or a schema is adjusted), nothing but the interface description file must be replaced (ideally, this description is generated from the microservices' source code with every build) and the benchmark adapts automatically, there is no need to adjust workload files or source code. Third, our approach can be used to evaluate different services that share the same purpose (e.g., user management). This is particularly useful when replacing a microservice with a new one as both can be benchmarked and compared extensively prior to switching in production.

Realistic Pattern Distribution:

Our design generates a synthetic and trace-based workload based on a pattern and workload definition. Defining a proper workload comes with its own challenges (which, however, are not specific to this approach). If a workload definition creates more resources than deleting existing ones, the list of resources grows with every iteration and may produce unrealistic workloads. E.g., the workload definition from our evaluation may fit the carts service because the number of items is usually constant (about the same number of additions and deletions) but, on the other hand, it may not fit the customer service well, where the number of users usually increases during the service lifetime because there are more new users registering than existing ones leaving. Thus, the actual patterns and a realistic distribution of these patterns have to be considered when defining the workload (e.g., by inspecting the log files to identify common interactions and their frequency [97]). This also implies that an existing workload based on a common pattern catalog yet to be defined cannot be applied blindly to other microservices.

Further Mapping Criteria:

Our approach relies on the semantics of REST-based interfaces and assumes HTTP-based microservices. Microservices using other communication protocols can be used as well but essentially require manual bindings for every operation. Since the basic CRUD semantics exist independently of the protocol used, it will be interesting to see if there are common ways in which these are exposed in non-REST-based microservices and whether they could be leveraged by our approach. E.g., the abstract operations could be mapped via the function name instead of the HTTP verb. Nonetheless, our approach can already be used for a large variety of microservices for which there are no benchmarking alternatives yet.

Unrealistic Bindings:

Our prototype is not limited to single microservices but considers cross-service dependencies and sub-resources. However, there are still some limitations: First, the dependencies between microservices must be defined manually in advance. This requires domain knowledge about the target application and can be difficult for applications with many services. Here, our prototype could be enhanced with an automated analysis based on text similarities that detects the service dependencies automatically to support application developers, e.g., that identifies that `userID` and `user/{id}` refer to the same property. Furthermore, our binding algorithm identifies all possible interaction sequences for a pattern configuration (and an optional binding definition).

This should be handled with caution as the binding might also identify unrealistic sequences that should be disabled before running the benchmark. E.g., two of the DEL interaction sequences which we found in our evaluation might be unrealistic (in another scenario). In this case, the third operation deletes a shopping cart after getting the details for a selected customer or deletes a customer after querying its cart. Thus, we recommend verifying the identified bindings manually before running benchmark experiments, especially for applications that involve multiple microservices. Nevertheless, in the worst case, such unrealistic matches simply add additional load on the SUT – it is only important to disregard their respective results.

Overall, we believe that our approach and its prototypical implementation are useful for a large percentage of microservice deployments as they significantly reduce the implementation effort for microservice developers. Some restrictions such as the REST requirement apply but could also serve as an incentive to switch to REST in some cases where other communication solutions are used for legacy reasons.

3.4 Summary

Benchmarking microservices serves to understand and check their non-functional properties for relevant workloads and over time. Performing benchmarks, however, can be costly: Each microservice requires the design and implementation of a benchmark from scratch, possibly repeatedly as the service evolves. As microservice APIs differ widely, benchmarking tools, which typically assume common interfaces of the SUT, do not exist.

In this chapter, we proposed a pattern-based approach to reduce the effort for defining microservice benchmarks, while still being able to measure the qualities of complex interactions. Our approach assumes that microservices expose a REST API, described in a machine-understandable way, and allows developers to model interaction patterns from abstract operations that can be mapped to the API. Required parameter values are provided at runtime and possible data dependencies between operations are resolved. We implemented our approach in a prototype, which we used to demonstrate the low effort applicability of our pattern-based benchmarking approach to an open-source microservice-oriented application with multiple services and dependencies. With this, we could show that pattern-based benchmarking of microservices is indeed feasible which opens up opportunities for microservice providers and tooling developers.

Deriving Optimized Microbenchmark Suites

For detecting performance changes such as increased request latencies using benchmarking, there are two alternatives with different levels of granularity. First, using application benchmarks, where the SUT is set up including all related components and stressed in an environment that mimics the production conditions (e.g., a database system running on a VM is stressed by a client software which mimics the requests of thousands of users for half an hour) [19, 27, 34, 74]. Second, using microbenchmarks, which analyze individual functions at source code level and execute them repeatedly (e.g., a date conversion method is called a million times) [113, 114]. Currently, neither benchmarking technique is suited to be executed for every code change due to the extensive execution duration of several hours as well as the resulting costs [45, 74, 113, 175, 183]. Optimized microbenchmark suites containing only a small number of microbenchmarks can potentially solve this problem, because their execution is orders of magnitude faster, yet they still have to reliably detect application-relevant performance changes.

In this chapter, we optimize microbenchmark suites of a software project such as a microservice or database system by analyzing function calls of microbenchmarks and a reference run, which can be an application benchmark or (an excerpt from) a production trace. In this analysis, we describe the ratio between the common function calls of a microbenchmark suite on the one hand and the set of function calls in the reference run on the other hand as practical relevance. If every called function of the reference run is also invoked by at least one microbenchmark, we consider the respective microbenchmark suite as 100% practically relevant as the suite covers all functions used in the baseline execution. We optimize and derive practically relevant microbenchmark suites by removing redundancies within the suite and recommending functions that should be covered by additional microbenchmarks. Moreover, we present the reference impact as a metric to quantify the importance of single microbenchmarks on the overall application performance. Microbenchmarks with a high reference impact cover long-running functions in the reference run and thus have a larger impact on performance metrics.

This chapter presents the following contributions:

- Two algorithms for optimizing microbenchmark suites.
- The metrics practical relevance and reference impact for quantifying the coverage and impact of microbenchmark suites.
- Two studies that analyze and apply the optimizations to the microbenchmark suites of two large open-source Time Series Database Systems (TSDBs).

- A performance change detection approach that addresses the variability in cloud environments and differentiates between potential and definitive changes.
- Empirical evidence showing that less expensive and faster optimized microbenchmark suites can be used as a proxy for application benchmarks in certain situations.

Our results show that the microbenchmark suites of our studied projects cover about 40% of the practically relevant source code and removing redundancies can reduce the number of microbenchmarks in a suite by 77% to 90%. Moreover, implementing three recommended microbenchmarks can increase the practical relevance from 40% up to 90%. Using the optimized suites in a realistic CI/CD pipeline can identify nine true positive performance changes, but there are also some false alarms and not all performance changes are detected. In total, our optimizations contribute to the problem of performance testing as part of CI/CD pipelines and enable a more frequent validation of performance metrics to detect regressions sooner. Our approach can provide targeted advice to developers to improve the effectiveness and relevance of their microbenchmark suites. Throughout the rest of this chapter, we will always use an application benchmark as the reference run but our approach can, of course, also use other sources as a baseline.

This chapter contains (partially adapted) material from our published research articles [72, 73] and is structured as follows: First, we present our approach for determining, quantifying, and improving microbenchmark suites in Section 4.1. Next, we evaluate our approach by applying the proposed algorithms to two open-source time series databases: once in an evaluation for a fixed code base in Section 4.2 and once in an applied case study analyzing a series of code changes in Section 4.3. After discussing the strength and limitations of our contributions in Section 4.4, we summarize our findings in Section 4.5.

4.1 Optimizing Microbenchmark Suites

We aim to determine and quantify the practical relevance of microbenchmark suites, i.e., to what extent the functions invoked by application benchmarks are also covered by microbenchmarks. Moreover, we want to improve microbenchmark suites by identifying and removing redundancies as well as recommending important functions that have not been covered yet.

In real setups, developers often do not have access to a (representative) live system, e.g., generally available software such as database systems are used by many companies that install and deploy their own instances and, consequently, the software’s developers usually do not have access to the custom installations and their production traces and logs. Additionally, software is used differently by each customer which results in differing load profiles as well as various configurations. Thus, it is often reasonable to use one or more application benchmarks as the next accurate proxy to simulate and evaluate a representative artificial production system. The execution of these benchmarks for each code change is very expensive and time-consuming, but a

lightweight microbenchmark suite that has proven to be practically relevant could replace them to some degree.

Our ideas are based on the intuition that, regardless of whether software is evaluated by an application benchmark or microbenchmark, both benchmark types evaluate the same source code. Since an application benchmark is designed to simulate realistic operations in a production-near environment, it can reasonably be assumed that it can serve as a baseline or reference execution for quantifying relevance in the absence of a real production trace. On the other hand, microbenchmarks are written to determine the performance of individual functions, and multiple microbenchmarks are bundled as a microbenchmark suite for analyzing the performance of a software system. Both benchmark types run against the same source code and generate a program flow (call graph) with functions as nodes and function calls as edges. We propose to analyze these graphs to (i) determine the coverage of both types to quantify the practical relevance of a microbenchmark suite and the reference impact of each microbenchmark, (ii) remove redundancies by identifying functions (call graph nodes) which are covered by multiple microbenchmarks, and (iii) recommend functions which should be covered by microbenchmarks because of their usage in the application benchmark. In a perfectly benchmarked software project, the ideal situation in terms of our approach would be that all practically relevant functions are covered by exactly one microbenchmark.

This section first describes the combination of application and microbenchmark call graphs for quantifying the practical relevance and reference impact of a microbenchmark in Section 4.1.1. We then present the optimization strategy for removing redundancies in a microbenchmark suite in Section 4.1.2 and finally describe the recommendation algorithm for new microbenchmarks that increase the practical relevance in Section 4.1.3.

4.1.1 Quantifying Practical Relevance and Reference Impact

Our approach assumes that the software project to be analyzed complies with best practices for both benchmarking domains, e.g., [25, 46]. It is necessary that there is both a suite of microbenchmarks and at least one application benchmark for the respective SUT. The application benchmark needs to rely on realistic scenarios to generate a relevant program flow and needs to run against an instrumented SUT which can create a call graph during the benchmark execution. The same applies for the execution of the microbenchmarks, where it needs to also be possible to reliably create the call graphs during the benchmark run. These call graphs can then be analyzed structurally to quantify and improve the microbenchmark suite’s relevance.

An overview of our approach is illustrated in Figure 4.1. After executing an application benchmark and the microbenchmark suite on an instrumented SUT, we retrieve one (potentially large) call graph from the application benchmark run and many (potentially small) graphs from the microbenchmark runs, one for each microbenchmark. In these graphs, each function represents a node and each edge represents a function call. Furthermore, we differentiate in the graphs between so-called project nodes, which refer directly to functions of the SUT, and non-project

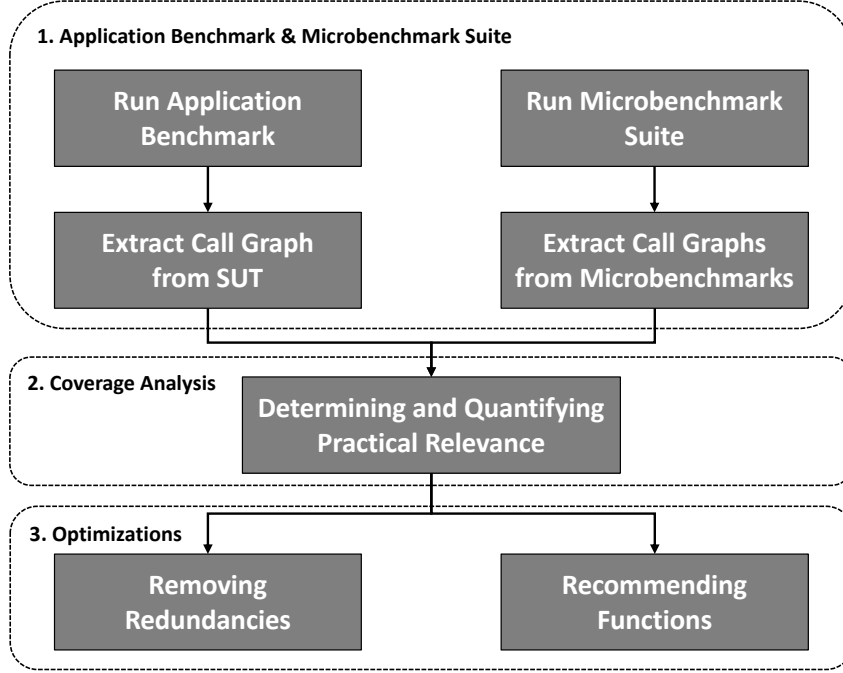


Figure 4.1: First, the SUT is evaluated via application benchmark and microbenchmark suite while generating call graphs. Next, the call graphs are compared to determine and quantify the practical relevance. Finally, two optimization algorithms make the microbenchmark suite more effective.

nodes, which represent functions from libraries or the operating system. After all graphs have been generated, the next step is to determine the function coverage, i.e., which functions are called by both the application benchmark and at least one microbenchmark. To link both benchmark types, we introduce the term **practical relevance** which refers to the extent to which a microbenchmark suite targets code segments that are also invoked by application benchmarks.

Figure 4.2 shows an example: The application benchmark graph covers 17 nodes and has two entry points. These entry points, when invoked, call other functions, which again call other functions (cycles are possible, e.g., in the case of recursion). Nodes in the graph can be both project functions of the SUT or functions of external libraries. There are also two microbenchmarks in this simple example, benchmark 1 and benchmark 2. While benchmark 1 only covers three nodes, benchmark 2 covers five nodes and seems to be more practically relevant.

To determine the function coverage, we iterate through all application benchmark nodes and identify all microbenchmarks that cover this function. As a result, we get a list of coverage sets, one for each microbenchmark, where each entry describes the overlap of nodes (functions) between the application benchmark call graph and the respective microbenchmark graph. Next, we count (i) all project-only functions and (ii) all functions which are called during the application benchmark and in at least one microbenchmark. Finally, we calculate two different coverage metrics: First, the **project-only** coverage of all executed functions in comparison to the total number of project functions in the application benchmark. Second, the **overall** coverage, including external functions.

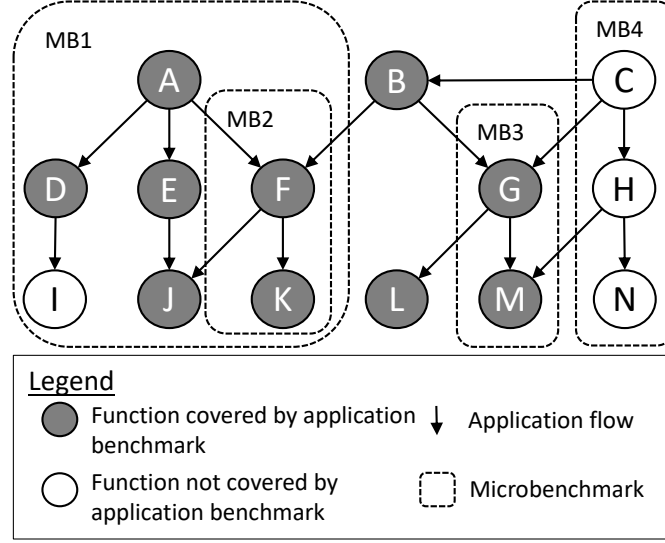


Figure 4.3: Strategy for optimizing microbenchmark suites. A suite containing microbenchmarks MB1 and MB3 would cover 80% of the application call graph. MB2 would not be included as all functions are already evaluated by MB1. MB4 does not evaluate any practically relevant functions.

Figure 4.3 (covering nodes F and K) would be redundant, as all nodes are already covered by other microbenchmarks. To identify a minimal set of microbenchmarks, we adapt the Greedy algorithm proposed by Chen et al. [38] and rank the microbenchmarks based on the number of reachable function nodes that overlap with the application benchmark (instead of *all* reachable nodes as proposed by Chen et al.), as defined in Algorithm 2.

After analyzing the graphs, we get coverage sets of overlaps between the application benchmark and the microbenchmark call graphs (input C). First, we sort them based on the number of covered nodes in descending order, i.e., microbenchmarks which cover many functions of the application benchmark are moved to the top (line 3). In Figure 4.3, the initial order is MB1 (6 nodes), MB2 (2 nodes), MB3 (2 nodes), and MB4 (0 nodes). Next, we pick the first coverage set as it covers the largest set of functions and add the respective microbenchmark to the minimal set (lines 4 to 8). Afterwards, we have to remove the covered set of the selected microbenchmark from all coverage sets (lines 9 to 11) and sort the coverage set again to pick the next microbenchmark. In Figure 4.3, we pick MB1 and the order is MB3 (2 nodes), MB2 (0 nodes), and MB4 (0 nodes). We repeat this until there are no more microbenchmarks to add (i.e., all microbenchmarks are part of the minimal set and there is no redundancy) or until the picked coverage set would not add any covered functions to the minimal set (line 6). For the example in Figure 4.3, this is MB1 and MB3.

In this work, we sort the coverage sets by their number of covered nodes and do not include any additional criteria to break ties. This could, however, result in an undefined outcome if there are multiple coverage sets with the same number of covered additional functions, but this case is a rare event and did not occur in our study. Still, including other secondary sorting criteria such

Algorithm 2: Removing redundancies in the microbenchmark suite.

Input: C - coverage sets
Result: $minimalSet$ - Minimal set of microbenchmarks

```

1  $minimalSet \leftarrow \emptyset$ 
2 while  $|C| > 0$  do
3    $C \leftarrow \text{SortSets}(C)$ 
4    $largestCoverage \leftarrow \text{RemoveFirst}(C)$ 
5   if  $|largestCoverage| == 0$  then
6     return  $minimalSet$ 
7   end
8    $minimalSet \leftarrow minimalSet \cup largestCoverage$ 
9   foreach  $set \in C$  do
10     $set \leftarrow set \setminus largestCoverage$ 
11  end
12 end

```

as the distance to the graph's root node or the total number of nodes in the coverage set might improve this optimization further.

4.1.3 Recommending Additional Microbenchmark Targets

A well-designed application benchmark will trigger the same function calls in an SUT as a production use would. A well-designed microbenchmark for an individual function will also implicitly call the same functions as in production or during the application benchmark. In this second optimization of the microbenchmark suite, we rely on these assumptions to selectively recommend uncovered functions for further microbenchmarking. This allows developers to directly implement new microbenchmarks that will cover a large part of the uncovered application benchmark call graph and thus increase the coverage levels.

Similar to the removal of redundancies, we build on the idea of a well-known, greedy test case prioritization algorithm proposed by Rothermel et al. [159] to recommend functions for benchmarking that are not yet covered. In particular, we adapt Rothermel's *additional algorithm*, which iteratively prioritizes tests whose coverage of new parts of the program (that have not been covered by previously prioritized tests) is maximal. Instead of using the set of all covered functions by a microbenchmark suite, our adaptation uses the function nodes from the application benchmark that are not yet covered as greedy criteria to optimize for.

Algorithm 3 defines the recommendation algorithm. The algorithm requires as input the call graph from the application benchmark, the graphs from the microbenchmark suite, and the coverage sets determined in Section 4.1.1, as well as the upper limit n of recommended functions.

First, we determine the set of project nodes (functions) in the application benchmark call graph that are not covered by any microbenchmark (line 2). In Figure 4.4, these nodes are A, B, D, F, and K. Next, we determine the reachable nodes for each function in this set, only considering project nodes, and store the results in another set N (lines 5 to 7). In Figure 4.4, the resulting

Algorithm 3: Recommending functions which are not covered by microbenchmarks yet.

Input: $\langle A, M, C \rangle$ - application benchmark call graph, microbenchmark call graphs, coverage sets

Input: n - number of microbenchmarks to recommend

Result: R - Set of recommended functions to microbenchmark

```

1  $R \leftarrow \emptyset$ 
2  $notCovered \leftarrow \{a | a \in A \wedge \text{IsProjectNode}(a)\} \setminus C^{total}$ 
3  $N \leftarrow \emptyset$ 
4 while  $n > 0$  do
5   foreach function  $f_a \in notCovered$  do
6      $additionalNodes \leftarrow \text{DetermineReachableNodes}(f_a) \cap notCovered$ 
7      $N \leftarrow N \cup \{additionalNodes\}$ 
8   end
9    $\text{SortByNumberOfNodes}(N)$ 
10   $largestAdditionalSet \leftarrow \text{RemoveFirst}(N)$ 
11  if  $|largestAdditionalSet| == 0$  then
12    return  $R$ 
13  end
14   $R \leftarrow R \cup largestAdditionalSet[0]$ 
15   $n = n - 1$ 
16   $notCovered \leftarrow notCovered \setminus largestAdditionalSet$ 
17 end

```

set for node A would be nodes A and D (node H is not a project node and not part of the reachable nodes). Third, we sort the set N by the number of nodes in each element, starting with the largest set of nodes in it (line 8). If two functions cover the same number of project nodes, we determine the distance to the closest root node and select functions that have a shorter distance. If the functions are still equivalent, we include the number of covered non-project nodes as a third factor and favor the function with higher coverage. Finally, we pick the first element and add the respective function to the recommendation set R (lines 9 to 13), update the set of not covered functions (line 15), and run the algorithm again to find the next function which adds the largest set of additional nodes to the covered set. For example in Figure 4.4, the first recommendation is node B with three reachable nodes. Our algorithm ends if there are n functions in R (i.e., upper limit for recommendations reached) or if the function that would be added to the recommendation set R does not add additional functions to the covered set (line 11).

4.2 Evaluation

We evaluate our approach on two open-source TSDBs written in Go, namely *InfluxDB* and *VictoriaMetrics*, which both have extensive developer-written microbenchmark suites. This section presents an evaluation using a single source code state. Next, Section 4.3 shows a case study exploring a series of code changes.

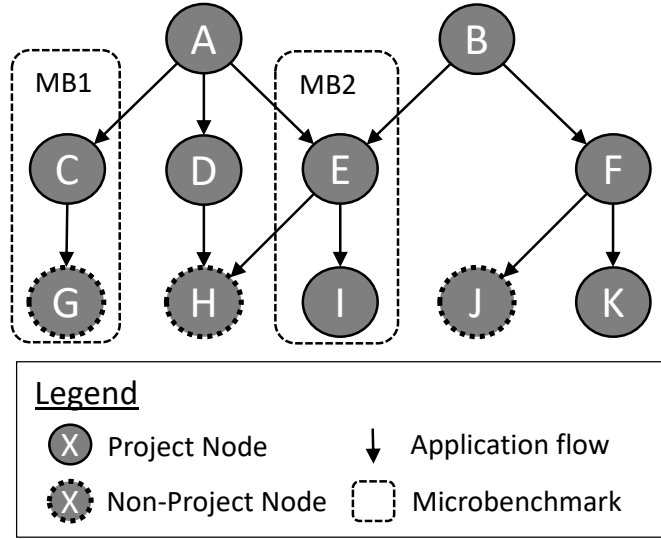


Figure 4.4: Recommending additional microbenchmarks. A benchmark evaluating function B would cover three more project nodes.

As an application benchmark and, therefore, baseline, we encode three application scenarios in YCSB-TS¹. Additionally, we run the microbenchmark suites of the respective systems. We start by giving an overview of both evaluated systems and YCSB-TS in Section 4.2.1 and present three different application benchmark scenarios in Section 4.2.2. Next, we describe how we run the application benchmark in Section 4.2.3 and the microbenchmark suite in Section 4.2.4 to collect the respective set of call graphs. We then use the call graphs to determine the coverage for each application scenario and quantify the practical relevance in Section 4.2.5 before removing redundancies in the benchmark suites in Section 4.2.6 and recommending functions which should be covered by microbenchmarks for every investigated project in Section 4.2.7. Finally, Section 4.2.8 summarizes the findings of this evaluation.

4.2.1 Study Objects

We use Time Series Database Systems (TSDBs) as study objects in this evaluation. TSDBs are designed and optimized for receiving, storing, managing, and analyzing time series data [53]. Time series data usually comprises sequences of timestamped data – often numeric values – such as measurement values. As these values tend to arrive in order, TSDB storage layers are usually optimized for append-only writes because only a few straggler values arrive late, e.g., due to network delays. Moreover, the stored values are rarely updated as the main purpose of TSDBs is to identify trends or anomalies in incoming data, e.g., to identify failure situations. Due to this, TSDBs are optimized for fast aggregation queries over variable-length time frames. Additionally, most TSDBs support tagging which is necessary for grouping values by dimension in queries. Taken together, these features and performance-critical operations make TSDBs a

¹<https://github.com/TSDBBench/YCSB-TS>

4.2. Evaluation

Project	<i>InfluxDB</i>	<i>VictoriaMetrics</i>
GitHub URL	<code>influxdata/influxdb</code>	<code>VictoriaMetrics/VictoriaMetrics</code>
Branch / Release	1.7	v1.29.4
Commit	<code>ff383cd</code>	<code>2ab4cea</code>
Go Files	646	1,284
Lines Of Code (Go)	193,225	462,232
Contributors	407	32
Stars	ca. 19,100	2,500
Forks	ca. 2,700	185
Microbenchmarks in Project	347	65
Extracted Call Graphs	288	62

Table 4.1: Our evaluation uses two open-source TSDBs written in Go as study objects.

suitable study object for the evaluation of our approach. Examples of TSDBs include InfluxDB², VictoriaMetrics³, Prometheus⁴, and OpenTSDB⁵.

To evaluate our approach, we need an SUT that comes with a developer-written microbenchmark suite and that is compatible with an application benchmark. For this, we particularly looked at TSDBs written in Go as they are compatible with the YCSB-TS application benchmark, and since Go contains a microbenchmark framework as part of its standard library. Furthermore, Go provides a tool called pprof⁶ that allows us to extract the call graphs of an application using instrumentation. Based on these considerations, we decided to evaluate our approach with the TSDBs *InfluxDB* and *VictoriaMetrics* (see Table 4.1).

YCSB-TS⁷ is a specialized fork of YCSB [40] (an extensible benchmarking framework for data serving systems) for TSDBs. Typically, every experiment with YCSB is divided into a load phase that preloads the SUT with initial data, and a run phase that executes the actual experiment queries.

InfluxDB is a popular TSDB with over 400 contributors and more than 19,000 stars on GitHub. *VictoriaMetrics* is an emerging TSDB (the first version was released in 2018) which has already collected more than 2,000 stars on GitHub. Both systems are written in Go, offer a microbenchmark suite, and can be benchmarked using the YCSB-TS tool. However, there was no suitable connector for *VictoriaMetrics* in the official YCSB-TS repository, which we implemented based on the existing connectors for *InfluxDB* and Prometheus. Moreover, we also fixed some minor

²<https://www.influxdata.com>

³<https://victoriametrics.com>

⁴<https://prometheus.io>

⁵<http://opentsdb.net>

⁶<https://golang.org/pkg/runtime/pprof>

⁷<https://github.com/TSDBBench/YCSB-TS>

Scenario	Medical Monitoring	Smart Factory	Wind Park
Load			
Records	1, 512, 000	1, 339, 200	2, 190, 000
Run			
Insert	1, 512, 000	1, 339, 200	2, 190, 000
Scan	1, 680	1, 860	35, 040
Avg	100, 800	744	35, 040
Count	0	744	0
Sum	0	2, 976	8, 760
Total	1, 614, 480	1, 345, 524	2, 268, 840
Other			
Duration	7 days	31 days	365 days
Tags	10	10	5

Table 4.2: We configured an application benchmark to use three different workload profiles.

issues in the YCSB-TS implementation. A fork with all necessary modifications, including the new connector and all fixes, is available on GitHub⁸.

4.2.2 Application Benchmark Scenarios

Systems such as our studied TSDBs are used in various domains and contexts, resulting in different load profiles depending on the specific use case. Depending on the workload, the call graphs within an SUT may vary. To consider this effect in our evaluation, we use three different workloads based on the following three scenarios for TSDBs, see Table 4.2. All workload files are available on GitHub⁹.

Medical Monitoring:

An intensive care unit monitors its patients through several sensors that forward the tagged and timestamped measurements to the TSDB. These values are requested and processed by an analyzer, which averages relevant values for each patient once per minute and scans for irregularities once per hour. In our workload configuration, we assume a new data item for every patient every two seconds and deal with 10 patients.

We convert this abstract scenario description into the following YCSB-TS workload: With an evaluation period of seven days, there are approximately three million values in the range of 0 to 300 that are inserted into the database in total. Half of them, about 1.5 million, are initially inserted during the load phase. Next, in the run phase, the remaining records are inserted and the queries are made. In this scenario, there are about 100,000 queries which contain mostly

⁸<https://github.com/martingrambow/YCSB-TS>

⁹<https://github.com/martingrambow/YCSB-TS/tree/master/workloads>

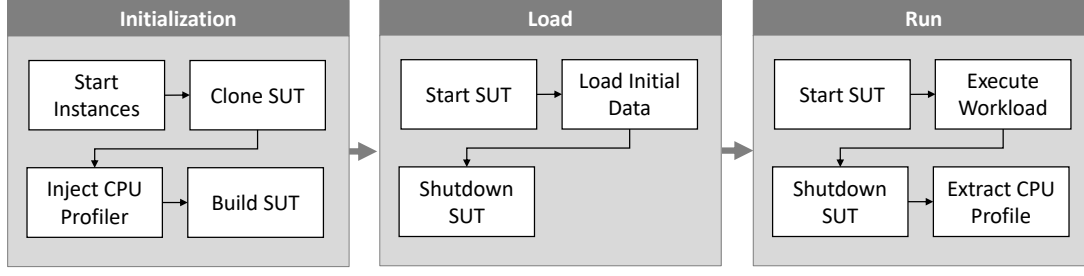


Figure 4.5: After initialization, the SUT is preloaded with initial data and restarted for the actual experiment run to clearly separate the program flow.

AVG as well as 1,680 SCAN operations. Furthermore, the workload uses ten different tags to simulate different patients.

Smart Factory:

In this scenario, a smart factory produces several goods with multiple machines. Whenever an item is finished, the machine controller submits the idle time during the manufacturing process as a timestamped entry to the TSDB tagged with the kind of produced item. Furthermore, a monitoring tool queries the average and the total amount of produced items once per hour and the accumulated idle time at each quarter of an hour. Finally, there are several manual SCAN queries for produced items over a given period. Our evaluation scenario deals with five different products and ten machines which on average each assemble a new item every 10 seconds. Moreover, there are 60 SCAN queries on average per day.

The corresponding YCSB-TS workload covers a 31-day evaluation period during which approximately 2.6 million data records are inserted. Again, we split the records in half and insert the first part in the load phase and insert the second part in parallel with all other queries in the run phase. In this scenario, we execute about 6,000 queries in the run phase which include SUM, SCAN, AVG, and COUNT operations (frequency in descending order). Furthermore, the workload uses five predefined tags to simulate the different products.

Wind Park:

Wind turbines in a wind park send information about their generated energy as timestamped and tagged items to the TSDB once per hour. At each quarter of an hour, a control center scans and counts the incoming data from 500 wind turbines in five different geographic regions. Moreover, it totals the energy produced for each hour.

Translated into a YCSB-TS workload with 365 days evaluation time, this means about 4.4 million records to be inserted and five predefined tags for the respective regions. Again, we have also split the records equally between the load and run phase. In addition, we run about 80,000 queries, split between SCAN, AVG, and SUM (frequency in descending order).

4.2.3 Application Benchmark Setup and Experiment Plan

Each experiment is divided into three phases: initialization, load, and run (see Figure 4.5). During the initialization phase, we create two AWS t2.medium EC2 instances (2 vCPUs, 4 GiB RAM), one for the SUT and one for the benchmarking client in the eu-west-1 region. The setup of the client is identical for all experiments: YCSB-TS is installed and configured on the benchmarking client instance. The initialization of the SUT starts with the installation of required software, e.g., Git, Go, and Docker. Next, we clone the SUT, revert to a fixed Git commit (see Table 4.1) and instrument the source code to start the CPU profiling when running. Finally, we build the SUT and create an executable file.

During the load phase, we start the SUT and execute the load workload of the respective scenario using the benchmarking client and preload the database. Then, we stop the SUT and keep the inserted data. This way we can clearly separate the call graphs of the following run phase from the rest of the experiment.

Afterward, we restart the SUT for the run phase. Since the source code has been instrumented, a CPU profile is now created and function calls are recorded in it by sampling while the SUT runs. Next, we run the actual workload against the SUT using the benchmarking client and then stop the SUT. This run phase of the experiments took between 40 minutes and 18 hours, depending on the workload and TSDB. Note that the actual benchmark duration is in this case irrelevant (as long as it is sufficiently long) since we are only interested in the call graph. Finally, we export the generated CPU profile which we use to build the call graphs.

After running the application benchmark for all scenarios and TSDBs, we have six application benchmark call graphs, one for each combination of scenario and TSDB.

4.2.4 Microbenchmarks

To generate the call graphs for all microbenchmarks, we execute all microbenchmarks in both projects one after the other and extract the CPU profile for each microbenchmark separately. Moreover, we set the benchmark execution time to 10 seconds to reduce the likelihood that the profiler misses nodes (functions), due to statistical sampling of stack frames. This means that each microbenchmark is executed multiple times until the total execution time for this microbenchmark reaches 10 seconds and that the duration is usually slightly higher than 10 seconds (the last execution starts before the 10-second deadline and ends afterwards). Finally, we transform the profile files of each microbenchmark into call graphs, which we use in our further analysis.

4.2.5 Quantifying Practical Relevance

Based on the call graphs for all scenario workloads and microbenchmarks, we analyze the coverage of both to determine and quantify the practical relevance following Section 4.1.1. Figure 4.6 shows the microbenchmark suite’s coverage for each study object and scenario. For *InfluxDB*,

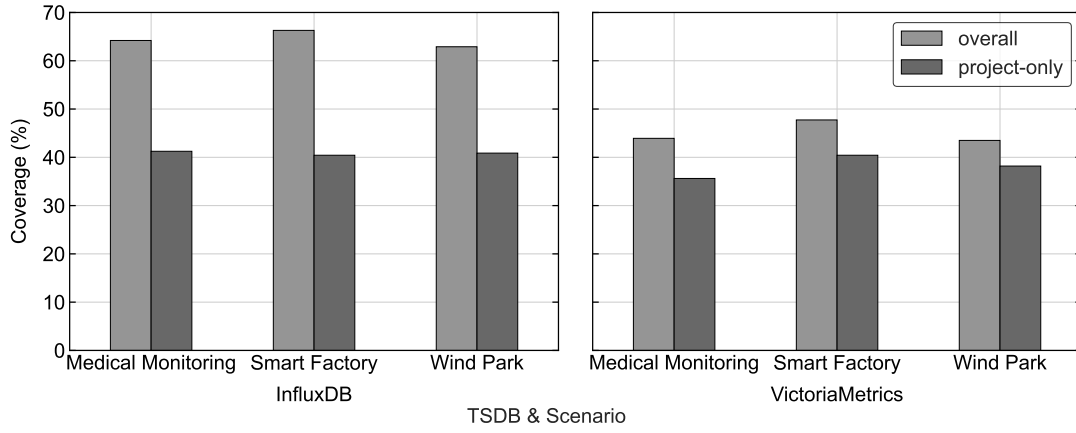


Figure 4.6: The project-only coverage is about 40% for both microbenchmark suites, leaving a lot of potential room for improvement.

Project	Scenario	Node Type	Number of Nodes		Coverage	
			App	Micro	Abs.	Rel.
<i>InfluxDB</i>	<i>Medical Monitoring</i>	<i>overall</i>	1,838	3,069	1,180	64.20%
		<i>project-only</i>	737	1,621	304	41.25%
	<i>Smart Factory</i>	<i>overall</i>	1,504	3,069	997	66.29%
		<i>project-only</i>	517	1,621	209	40.43%
	<i>Wind Park</i>	<i>overall</i>	1,895	3,069	1,192	62.90%
		<i>project-only</i>	778	1,621	318	40.87%
<i>VictoriaMetrics</i>	<i>Medical Monitoring</i>	<i>overall</i>	1,573	1,125	691	43.93%
		<i>project-only</i>	511	454	182	35.62%
	<i>Smart Factory</i>	<i>overall</i>	1,238	1,125	591	47.74%
		<i>project-only</i>	371	454	150	40.43%
	<i>Wind Park</i>	<i>overall</i>	1,600	1,125	696	43.50%
		<i>project-only</i>	542	454	207	38.19%

Table 4.3: All microbenchmarks together form a significantly larger call graph than the application benchmark (number of nodes) but, however, these by far do not cover all functions called during the application benchmarks (coverage).

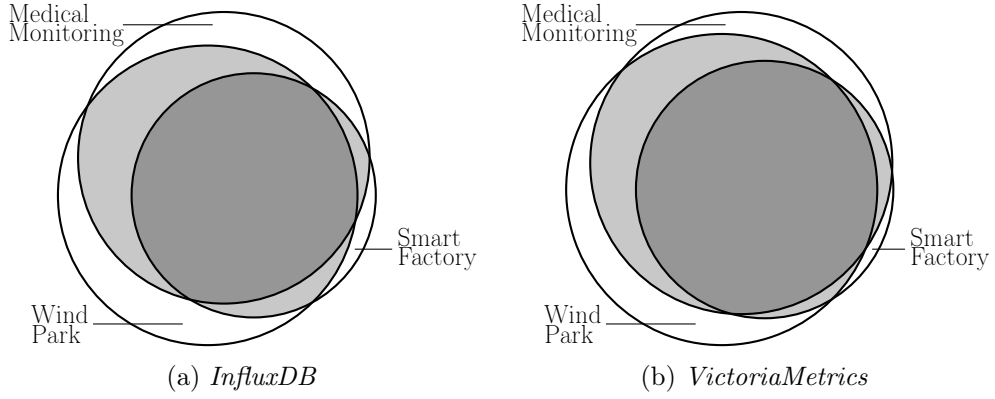


Figure 4.7: All scenarios generate an individual call graph. Some functions are exclusively called in one scenario, while many are called in two or all three scenarios.

Node Type	Scenario	<i>Med. Monitoring</i>	<i>Smart Factory</i>	<i>Wind Park</i>
<i>overall</i>	<i>Med. Monitoring</i>	—	1,411 (76.77%)	1,662 (90.42%)
	<i>Smart Factory</i>	1,411 (93.82%)	—	1,445 (96.08%)
	<i>Wind Park</i>	1,662 (87.70%)	1,445 (76.25%)	—
<i>project-only</i>	<i>Med. Monitoring</i>	—	468 (63.50%)	624 (84.67%)
	<i>Smart Factory</i>	468 (90.52%)	—	484 (93.62%)
	<i>Wind Park</i>	624 (80.21%)	484 (62.21%)	—

Table 4.4: Pairwise overlap between different scenarios in *InfluxDB*.

the overall coverage ranges from 62.90% to 66.29% and the project-only coverage ranges from 40.43% to 41.25%, depending on the application scenario. For *VictoriaMetrics*, the overall coverage ranges from 43.5% to 46.74% and the project-only coverage from 35.62% to 40.43%. Table 4.3 shows the detailed coverage levels.

As a next step, we also analyze the coverage sets of all application benchmark call graphs to evaluate to which degree the scenarios vary and generate different call graphs. Figures 4.7a and 4.7b show the application scenario coverage as Venn diagrams for *InfluxDB* and *VictoriaMetrics* using project nodes only. Both diagrams show the same characteristics in general. All scenarios trigger unique functions which are not covered by other scenarios, see Table 4.2. For both SUTs, the Smart Factory scenario generates the smallest unique set of project-only nodes (29 unique functions for *InfluxDB* and 4 unique ones for *VictoriaMetrics*) and the Wind Park scenario generates the largest one (134 functions for *InfluxDB* and 77 for *VictoriaMetrics*). Furthermore, all scenarios also generate a set of common functions which are invoked in every scenario. For *InfluxDB*, there are 464 functions of 920 in total (50.43%) and for *VictoriaMetrics* there are 341 functions of 603 in total (56.55%) which are called in every application scenario. Tables 4.4 and 4.5 show the overlap details.

Node Type	Scenario	<i>Med. Monitoring</i>	<i>Smart Factory</i>	<i>Wind Park</i>
<i>overall</i>	<i>Med. Monitoring</i>	—	1,171 (74.44%)	1,391 (88.43%)
	<i>Smart Factory</i>	1,171 (94.59%)	—	1,158 (93.54%)
	<i>Wind Park</i>	1,391 (86.94%)	1,158 (72.37%)	—
<i>project-only</i>	<i>Med. Monitoring</i>	—	356 (69.67%)	454 (88.84%)
	<i>Smart Factory</i>	356 (95.96%)	—	352 (94.88%)
	<i>Wind Park</i>	454 (83.76%)	352 (64.94%)	—

Table 4.5: Pairwise overlap between different scenarios in *VictoriaMetrics*.

4.2.6 Removing Redundancies in Microbenchmark Suites

Our first optimization, as defined in Algorithm 2 on Page 43, analyzes the existing coverage sets and removes redundancy from both microbenchmark suites by greedily adding microbenchmarks to a minimal suite that fulfills the same coverage criteria. Figure 4.8 shows the step-by-step construction of this minimal set of microbenchmarks up to the maximum possible coverage.

For *InfluxDB* (Figure 4.8a), the first selected microbenchmark already covers more than 12% of each application benchmark scenario graph. Furthermore, the first four selected microbenchmarks are identical in all scenarios. Depending on the scenario, these already cover a total of 28% to 31% (with a maximum coverage of about 40% when using all microbenchmarks, see Table 4.3). These four microbenchmarks are therefore very important when covering a large practically relevant part of the source code. However, even if all microbenchmarks selected during minimization are chosen and the maximum possible coverage is achieved, the removal of redundancies remains very effective. Depending on the application scenario, the initial suite with 288 microbenchmarks from which we extracted call graphs was reduced to a suite with either 19, 25, or 27 microbenchmarks.

In general, we find similar results for *VictoriaMetrics* (Figure 4.8b). Already the first microbenchmark selected by our algorithm covers at least 17% of the application benchmark call graph in each scenario. For *VictoriaMetrics*, the first four selected microbenchmarks also have similar coverage sets, with only a slight difference in the parametrization of one chosen microbenchmark. In total, these first four microbenchmarks cover 29% to 34% of the application benchmark call graph, depending on the scenario, and there is a maximum possible coverage between 35% and 40% when using the full existing microbenchmark suite. Again, the first four microbenchmarks are therefore particularly effective and already cover a large portion of the application benchmark call graph. Moreover, even with the complete minimization and the maximum possible coverage, our algorithm significantly reduces the number of microbenchmarks: from 62 microbenchmarks down to 13 or 14 microbenchmarks, depending on the specific application scenario.

Since each microbenchmark takes on average about the same amount of time (see Section 4.2.4), our minimal suite results in significant time savings when running the microbenchmark suite. For *InfluxDB*, it would take only about 10% of the original time and for *VictoriaMetrics* about

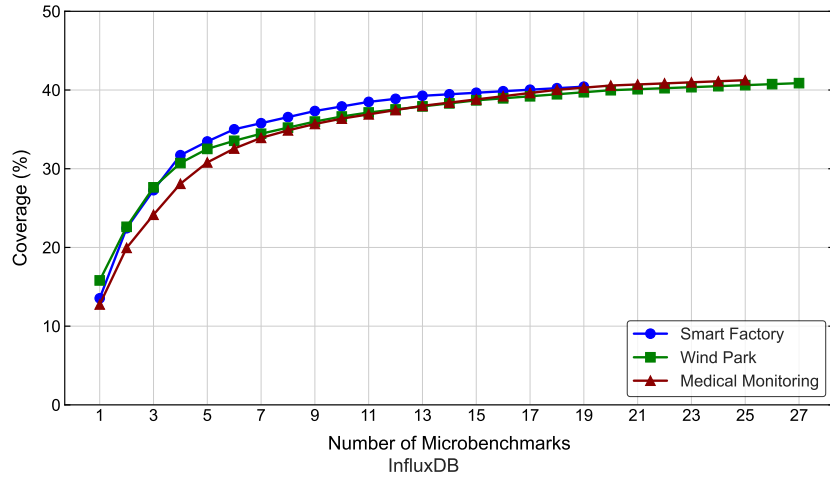
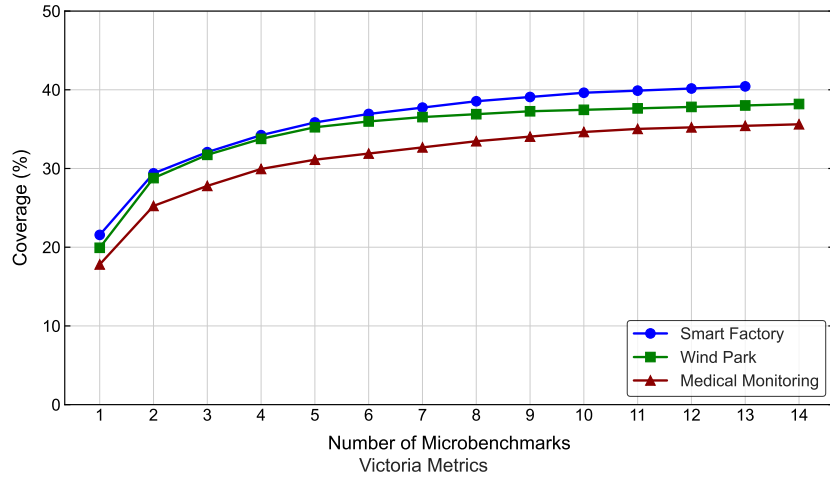
(a) *InfluxDB*(b) *VictoriaMetrics*

Figure 4.8: Already the first four microbenchmarks selecting by Algorithm 2 cover 28% to 31% for *InfluxDB* and 29% to 34% for *VictoriaMetrics* of the respective application benchmark’s call graph.

23% respectively. On the other hand, these drastic reductions also mean that many microbenchmarks in both projects evaluate the same functions. This can be useful under certain circumstances, e.g., if there is a performance degradation detected using the minimal benchmark suite and developers need to find the exact cause. However, given our goal of finding a minimal set of microbenchmarks to use as smoke test in a CI/CD pipeline, these redundant microbenchmarks present an opportunity for drastically reducing the execution time without much loss of information.

4.2.7 Recommending Additional Microbenchmark Targets

Our second optimization, the recommendation, starts with the minimal microbenchmark suite from above and subsequently recommends functions to increase the coverage of the microbenchmark suite and application benchmark following Algorithm 3 on Page 44. Figure 4.9 shows

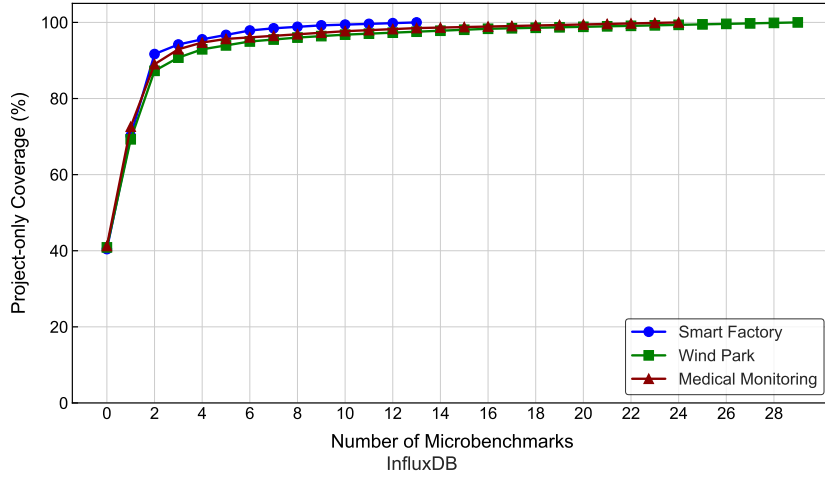
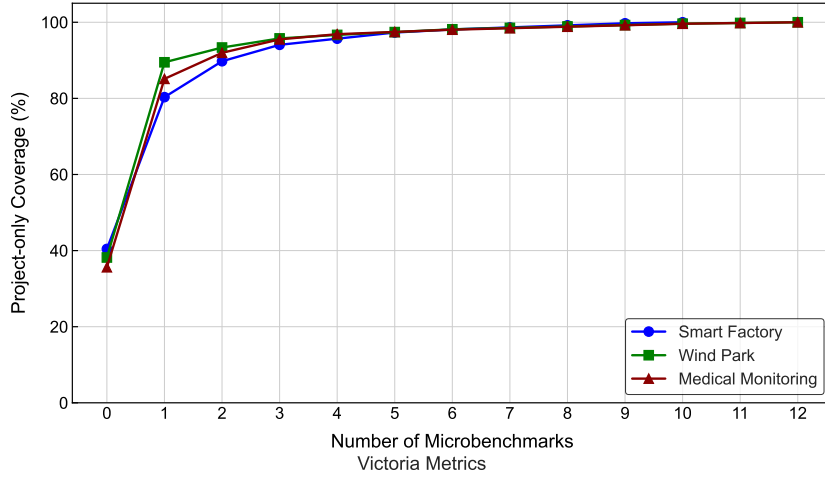
(a) *InfluxDB*(b) *VictoriaMetrics*

Figure 4.9: Already microbenchmarks of the first three recommended functions could increase the project-only coverage up to 90% to 94% for *InfluxDB* and 94% to 95% for *VictoriaMetrics*.

this step-by-step recommendation of functions starting with the current coverage up to a 100% relevant microbenchmark suite.

For *InfluxDB* (Figure 4.9a), a microbenchmark for the first recommended function would increase the coverage by about 30% depending on the application scenario and the first three recommended functions are identical for all scenarios: (i) `executeQuery` runs a query against the database and returns the results, (ii) `ServeHTTP` responds to HTTP requests, and (iii) `storeStatistics` writes statistics into the database. If each of these functions were evaluated by a microbenchmark in the same way as the application benchmark, i.e., resulting in the same calls of downstream functions and the same call graph, there would already be a total coverage of 90% to 94%. To achieve an 100% match, additional 10 to 26 functions needs to be microbenchmarked, depending on the application scenario and always under the assumption that the microbenchmark will call the function in the same way as the application benchmark.

In general, we find similar results for *VictoriaMetrics* (Figure 4.9b). Already a microbenchmark for the first recommended function would increase the coverage by about 40% for the Smart Factory scenario and by about 50% for the other scenarios. Microbenchmarking the first three recommended functions would increase the coverage up to a total of about 95% for all scenarios. Again, these three functions are recommended in all scenarios, with only the ordering differing. The first recommended functions are all anonymous functions, respectively (i) an HTTP handler function, (ii) a merging function, and (iii) a result-related function. To achieve 100% project-only coverage, 10 to 14 additional functions would need to be microbenchmarked depending on the application scenario.

In summary, our results indicate that the microbenchmark suite can be made much more relevant to actual practice and usage by adding a few additional microbenchmarks for key functions. In most cases, however, it is not possible to directly convert the recommendations into suitable microbenchmarks (we discuss this point in Section 4.4). Nonetheless, we see these recommendations as a valid starting point for more thorough analysis.

4.2.8 Findings

Our evaluation with two well-known TSDBs shows that their microbenchmark suites cover about 40% of the functions called during application benchmarks. The majority of the functionality used by an application benchmark, our proxy for a production application, is therefore not covered by the microbenchmark suites of our study objects.

If there are many microbenchmarks in a suite, they are likely to have redundancies and some functions will be benchmarked by multiple microbenchmarks. To achieve the same coverage level with fewer microbenchmarks, it is possible to create a new subset of the microbenchmark suite without these redundancies. This significantly reduces the overall execution duration of the microbenchmark suite. Applying this optimization as part of our evaluation shows that this can reduce the number of microbenchmarks by 77% to 90%, depending on the application and benchmark scenario.

If the microbenchmark suite's coverage is insufficient, the uncovered graph of the application benchmark can be used to locate functions that are highly relevant for practical usage. We present a recommendation algorithm that provides a fast and automated way for identifying these functions that should be covered by microbenchmarks. Our evaluation shows that it is theoretically possible to increase coverage from the original 40% to up to 90% with only three additional microbenchmarks.

4.3 Case Study on the Detection Capabilities of Microbenchmark Suites

This section studies optimized microbenchmark suites for a series of code changes in a realistic case study. Again, we use two open-source TSDBs written in Go, namely *InfluxDB* and

VictoriaMetrics, both of which have extensive developer-written microbenchmark suites and a long commit history. In this study, we apply our redundancy removing optimization on both projects and run application benchmarks, the optimized microbenchmark suite, and the full suite for a series of dozens of code changes. The complete benchmarking dataset is available openly¹⁰.

First, Section 4.3.1 presents our study design and Section 4.3.2 describes the methods we use to analyze the benchmark results. Next, we report the results of the application benchmarks in Section 4.3.3 and use them as a “ground truth” for the optimization algorithm. In Section 4.3.4, we investigate whether the optimized microbenchmark suite can detect the same performance changes with less effort. To quantify the improvements and to verify that the complete microbenchmark suite is not a better proxy for detecting application performance changes, we also execute the complete suite for every 5th commit in our evaluation period and show the results in Section 4.3.5. Finally, we derive implications by combining all information in Section 4.3.6 and Section 4.3.7 summarizes the findings of this evaluation.

4.3.1 Study Design

To study to which degree performance changes can be detected with an optimized microbenchmark suite, we first run application benchmarks for both TSDBs to detect all application performance changes for the production environment. Next, we optimize the respective microbenchmark suites using recorded application benchmark call graphs as reference and run the optimized suite for every code change as well to check whether the optimized suites can detect the same performance changes. Finally, we also run the complete microbenchmark suites to quantify the degradation of detection quality caused by relying only on the optimized microbenchmark suite. Because it is infeasible to examine the entire development cycle over a period of several years, we examine a smaller sample of successive code changes, spanning several months, to simulate a realistic long-term use of all benchmarking techniques.

An optimized suite capable of detecting relevant performance changes can be embedded in a cloud-based CI/CD pipeline. To mimic this realistic setup as closely as possible, we therefore use cloud-based VMs that are created and configured for every experiment run from scratch. To minimize performance variation between different instances and due to random effects such as noisy neighbors in the cloud environment, we adapt and apply recent best practices in each benchmarking discipline to acquire reliable measurement results. We use the Duet Benchmarking technique [32, 33] for application benchmarks and RMIT [1, 2] for execution of microbenchmarks. For all experiments, we use a hardware setup that is similar to the cloud experiment setups in related studies [28, 71, 74, 75, 113]. We run all experiments on e2-standard-2 Google Cloud instances in the europe-west3 region with 2 virtual CPUs and 8 GB RAM, local SSD storage, running Ubuntu 20.04 LTS.

¹⁰<http://dx.doi.org/10.14279/depositonce-15532>

Project	<i>VictoriaMetrics</i>	<i>InfluxDB</i>
Go files	2,088	1,653
Lines of Go Code	742,191	520,716
Branch / Release	master	influx2.0
Start of Evaluation Period	Mar 1, 2021	Jan 1, 2021
End of Evaluation Period	May 31, 2021	May 14, 2021
Number of Commits	70	110
Number of Microbenchmarks	177	426 (109)

Table 4.6: Study objects and meta information. Both study objects are TSDBs that can be evaluated using application and microbenchmarks.

Study Objects:

Both TSDBs we used in our evaluation (see Section 4.2) were suitable study objects because there are both application and microbenchmarks. With this experience and because there is also an extensive history with many code changes, we use the same open-source TSDBs, but later versions. Since the first version of *VictoriaMetrics* was released in 2018, more than 80 contributors have created more than 2,000 files and made more than 2,800 commits as of May 31, 2021. In our experiments, we study the most recent 70 commits at the time of running the experiments, i.e., between March 1 and May 31, 2021, which merge a pull request into the master branch, in more detail. In this period, the complete microbenchmark suite consists of 177 executable microbenchmarks in total, including all parameterized factors.

InfluxDB squashes individual fixes and features in single commits. *InfluxDB* version 2.0, which we investigate further, accumulated over 34,000 commits in more than 1,600 files from 422 contributors up to May 14, 2021. For our detailed study, however, we examine the most recent 110 commits at the experimentation phase in the time frame between Jan 1 and May 14, 2021. In contrast to *VictoriaMetrics*, the complete microbenchmark suite of *InfluxDB* does not remain constant for our evaluation period, but decreases from 426 microbenchmarks in the beginning to 109 at the end (we address and discuss this in Sections 4.3.5 and 4.4). Table 4.6 gives an overview of both study objects and the respective commits that we studied.

Application Benchmarks:

For the application benchmarks, we use two cloud VMs in each experiment, one sending the workload and one running the respective SUT. Moreover, we adapt the Duet Benchmarking technique [32, 33] and set up two versions of the respective TSDB as Docker containers on the same VM: the base version and the variation (see Figure 4.10). We then use a pre-generated workload and start two benchmark clients simultaneously, one targeting the base version’s port and one targeting the variation’s port. Thus, the performance of the two variants can already

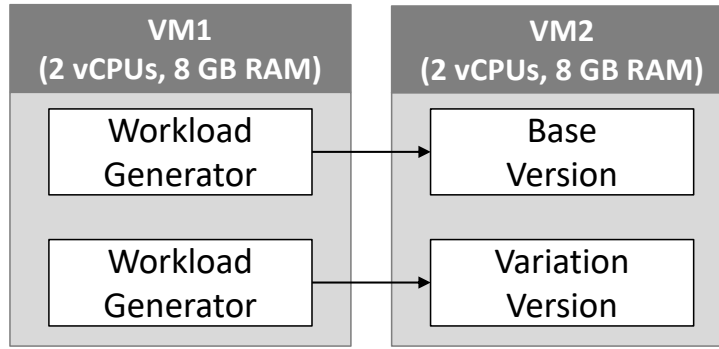


Figure 4.10: Application benchmark setup. To compare the performance of the first version (base) with the current version (variation) of the SUT in the respective evaluation period, we deploy both variants on the same VM and benchmark both simultaneously.

be compared using three experiment repetitions, because both SUT versions are exposed to the same random factors at the same time.

The application benchmark workload is based on the *DevOps* use case in the Time Series Benchmark Suite (*TSBS*¹¹), which in turn is based on the client *influxdb-comparisons*.¹² We use the *TSBS* client to benchmark *VictoriaMetrics* and *influxdb-comparisons* to benchmark *InfluxDB*, but have extended both clients to report latency values of inserts and queries separately.¹³ The *DevOps* use case simulates a server farm in which a specified number of servers sends utilization data (e.g., CPU and RAM) to the TSDB in a specified interval. After a first phase in which the data is inserted into the database, a second phase continues with simple queries, and a final phase with more complex group-by queries completes the experiment. We adjusted the number of simulated servers, the sending interval, the total simulated duration, and the number of respective queries to the specific SUT in a way that the client instance is below 50% utilization and the SUT instance is almost always fully utilized (see Table 4.7 for all workload details). Furthermore, we repeat each experiment at least three times using fresh VM instances to ensure reproducibility.

For our interpretation of results, we have to consider two aspects: First, as in almost all application benchmark experiments, the first few measurements must be considered as part of a warm-up phase and should be discarded [25]; Second, due to the Duet Benchmarking, the last measurements should also be discarded. If one of the two evaluated versions has a better performance, the corresponding benchmark run will also finish earlier than the other one, which will then release resources on the experiment VM. The other container running the slower version will then have access to additional resources and speed up, leading to incorrect measurements. After some initial experiments comparing the first and last commit state of our study periods to determine the expected overall performance change, we chose to disregard the first 5% and the last 20% in the measurement series of each application benchmark run.

¹¹<https://github.com/timescale/tsbs>

¹²<https://github.com/influxdata/influxdb-comparisons>

¹³<https://github.com/martingrambow/benchmarkStrategy>

Project	<i>VictoriaMetrics</i>	<i>InfluxDB</i>
Number of Simulated Servers	800	100
Sending Interval	60s	60s
Simulated Duration	72h	168h
Number of Insert Clients	4	10
Batch Size	400	60
Number of Batches	259,200	113,400
Number of Simple Queries	8,640	1,008
Number of Group-By Queries	1,440	168
Number of Query Clients	10	10

Table 4.7: Workload parameters. The workload for each TSDB differs to ensure full utilization of the respective SUT.

Microbenchmarks:

To remove redundancies in the microbenchmark suite, we initially execute and trace an application benchmark against the first commit of the evaluation period and create an application call graph, which serves as the reference for the optimization algorithm. Next, we execute and trace the full microbenchmark suite for the same commit and generate the call graph for each microbenchmark. With both inputs, the reference application graph and the microbenchmark graphs, we then determine the practical relevance of the microbenchmark suites, remove redundancies, decide which microbenchmarks to include in the optimized suite, and run the optimized microbenchmark suites for every commit in the respective evaluation period. Finally, to rate the improvements and back-test the optimization, we also run the full microbenchmark suites for every 5th commit in the evaluation period. Running all microbenchmarks for every commit is unrealistic in practice due to the high cost of execution. An execution for every 5th commit is a trade-off between a very detailed analysis and a long execution time as well as the corresponding monetary cost. We believe that this is fine-grained enough to detect relevant changes and, in case of anomalies, to further evaluate the relevant benchmark for the intermediate changes.

To mimic the usage of the microbenchmark suites in CI/CD pipelines, which compare a new version with an older commit state, we benchmark both versions on the same VM using RMIT time-shared execution [1, 2]. Here, to counteract infrastructure variation, we randomize the execution order of each suite and run each microbenchmark for both versions successively. To reduce the influence of the microbenchmarks on the performance of the following ones and to make sure that these effects are not systematic, we also randomly vary which microbenchmark version (base or variation) is executed first. Adapted from the configurations used by Laaber et al. [111] and Chen et al. [36], we repeat each of our microbenchmark experiments three times on fresh VMs (instance run), run each suite three times (suite run), and call each benchmark five times (iteration) for one second each (duration). In total, thus, there are 45 measurements per microbenchmark per commit, each comprising many benchmark function calls.

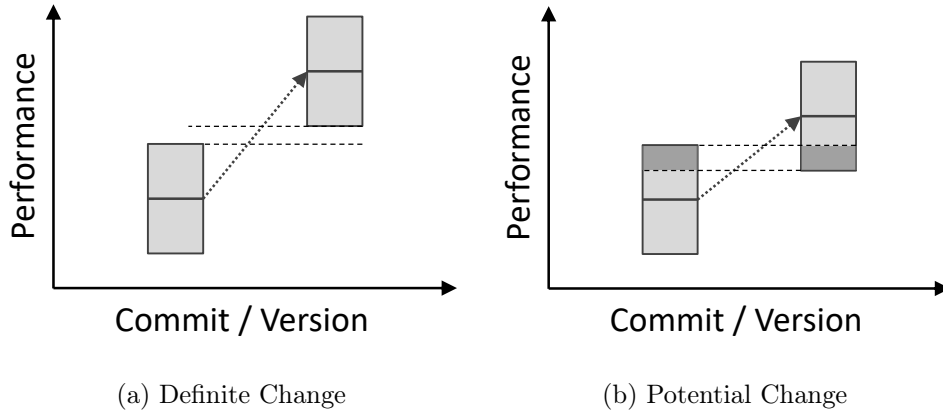


Figure 4.11: Intensity classification. Experiments in cloud environments can show a large variance. We therefore classify detected changes based on the 99% CI as definite or potential.

4.3.2 Analysis

The results of the benchmarks are analyzed as follows:

Compared Versions:

For all our experiments, we keep the base version fixed to the first commit in the evaluation period and iterate over the commits as variation version. Thus, we always compare the current variation with the initial one. Nevertheless, because the results are transitive, the performance changes can be visualized as a pseudo-continuous graph.

Median Performance Change:

To actually compare the versions, i.e., to decide which version performs better, we use the median value of all measurements. For the application benchmark, we use the median latency of all measured latency values for the respective query type. For microbenchmarks, we use the median execution duration of each microbenchmark out of the 45 measurements (3 instance runs * 3 suite runs * 5 iterations). Finally, we calculate the relative change by comparing the median value of the base version with the median of the variation (e.g., if the median latency increases from $100ms$ to $110ms$, a query takes 10% longer).

Confidence Intervals:

To calculate the CI of a performance change, we use a bootstrapping methodology that implements hierarchical random re-sampling with replacement [102]. For the microbenchmarks, we draw 10,000 random samples of 45 values each from the measurements¹⁴, determine the median value in each sample, and use the top and bottom $\alpha = 0.5\%$ of the resulting ordered set of the medians as the 99% CI. For the application benchmarks, we adjust the sample size to match the number of requests for the respective request type, draw 10,000 samples, and determine the CI in the same way.

¹⁴Due to the replacement, values can be drawn multiple times and this precisely maps the actual distribution of the measurements.

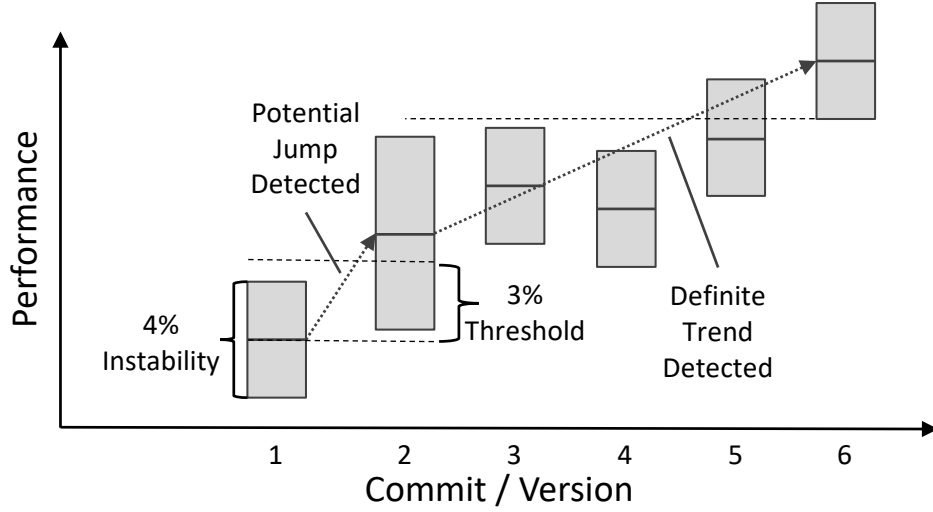


Figure 4.12: Type classification. While the jump detection identifies performance changes in two successive code changes, the trend detection considers a series of commits. The detection threshold adapts dynamically to the previous instability measurements. Thus, the detection threshold for the 3rd commit would increase because of the larger instability in the second one.

Definite and Potential Performance Changes:

A wide CI in an experiment indicates that the performance change of a (micro-) benchmark cannot be clearly quantified and that the individual benchmark is unstable. Thus, we refer to the width of a confidence interval as **instability**. The smaller this instability is, the better and more precise the detection of performance changes becomes. On the other hand, a wide CI implies that it is only possible to detect large performance changes, because overlapping CIs of the respective experiments do not allow us to draw precise conclusions. Thus, we classify the detected performance changes as (99%) **definite** (no overlap) and **potential** (overlapping CIs) performance changes (see Figure 4.11).

Jump and Trend Detection:

We adapt two basic threshold-based algorithms by Grambow et al. [74] to decide if we found a relevant performance change. For this study, we use (i) a **jump** detection algorithm to identify individual commits that introduce performance changes, and (ii) a **trend** detection algorithm to detect performance trends in a series of ten commits. In our study, however, we extend the static threshold and use a dynamic one that constantly adjusts to 75% of the instability of previous measurements (see Figure 4.12). Using 75% of the CI width is a trade-off between many false positives (50%) and potentially many false negatives (100%). Taking half the CI width could create false positive alarms in the change point detection, as the median performance change might just randomly fluctuate into the respective CI. For example, in Figure 4.12, using 50% of the instability in commit 1 corresponds to a 2% threshold, which leads to the median change in commit 2 to be exactly on the CI boundary of commit 1. Using the full CI width (100%) would only detect changes larger than the referenced instability, e.g., 4% for commit 2 in Figure 4.12. For both of our projects studied, using a 75% dynamic threshold provides a good

Project	Instability (99% CI)	
	<i>VictoriaMetrics</i>	<i>InfluxDB</i>
Inserts	6.27% [−2.95; 3.32]	0.88% [−0.71; 0.17]
Simple Queries	1.66% [−1.25; 0.41]	3.37% [−1.36; 2.01]
Group-By Queries	1.71% [−0.79; 0.92]	2.11% [−0.82; 1.29]

Table 4.8: Result instability in A/A benchmarks. All CIs are close to the 0% value but insert requests to *VictoriaMetrics* and queries to *InfluxDB* show a larger instability.

balance between false alarms and (potentially) undetected performance changes. Nevertheless, this threshold parameter is project-specific, especially if the median performance value is not centered in the respective CIs but is shifted to either side.

Code changes that stabilize the measurements will thus narrow the CI automatically (or the other way around) and performance variations during the complete experiment series will automatically be considered in the analysis. Moreover, because we do not consider small performance changes as relevant in our evaluated projects, we also set a minimum threshold of 1%, similar to what best practice suggests [68]. In other projects, however, even smaller changes may also be relevant and this value would have to be adjusted. Finally, our dynamic detection algorithms require an initial threshold that is close to the expected value. If the difference is too high at first, either many false alarms would be triggered (small initial threshold) or relevant changes would not be detected (large initial threshold). Nevertheless, after the threshold has been continuously adjusted across several code changes (in our case ten), the detection mechanisms are adjusted to the respective instability.

4.3.3 Detected Performance Changes Using Application Benchmarks

First, to verify that our results are reliable and correct, we use five repeated A/A benchmarks which compare the first commit as the base version with itself as the variation version. Ideally there should not be any performance change, the CIs should be narrow, and around the 0% value. Table 4.8 reports the respective CIs derived from the bootstrapping method for each query type and SUT. All CIs straddle 0%, which implies no detected performance change. Nevertheless, especially the wide CI for inserts in *VictoriaMetrics* also implies that we can not reliably detect definite performance changes smaller than 6%. Based on the respective CI (reported in Table 4.8), we set the initial detection thresholds for both change point detection algorithms to $\approx 75\%$ of the instability value or 1% (see Section 4.3.2, Jump and Trend Detection): except for the inserts in the case of *VictoriaMetrics* (5%) and the two query types in the case of *InfluxDB* (3% and 2%), we thus set all the initial detection threshold values to 1%.

Figures 4.13a to 4.13c show the relative performance history and the detected performance changes for insertions, simple queries, and group-by queries against *VictoriaMetrics*. A positive percentage value indicates that the respective request latency has increased.

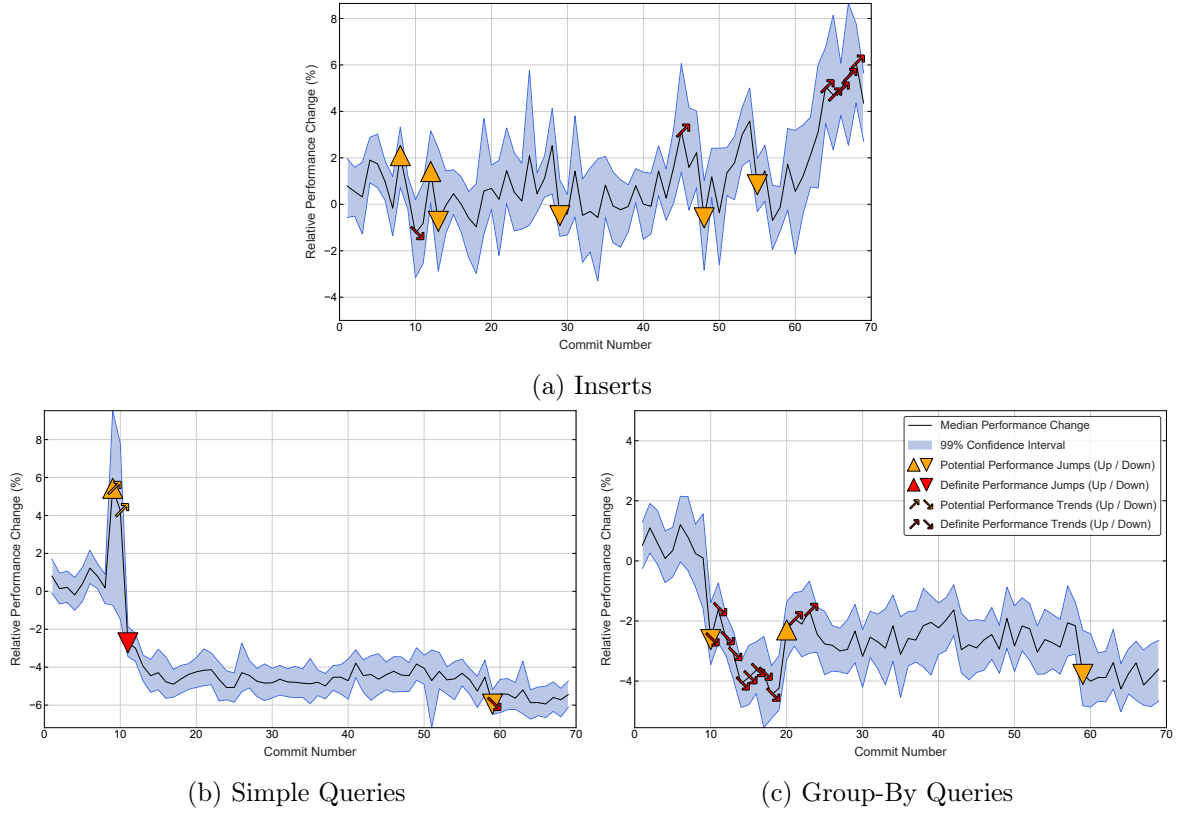


Figure 4.13: Application benchmark results for *VictoriaMetrics*. Negative values show an improvement. There is (i) a definite negative performance trend in the last commits of our evaluation period for inserts, (ii) a definite positive trend for both query types from commit 10 to 20 which is followed by (iii) a negative trend for group-by queries. Finally, there is (iv) a positive trend for both query types around commit 60.

Due to the non-deterministic setup of the internal data structure in *VictoriaMetrics*, there is a large instability for inserts. To overcome this obstacle, we split the initial insertion phase in half, copied the data from the base version container after the first half of insertions, and replaced the data in the variation container with this copy. The second part of the inserts is thus based on the same data structure and the non-determinism has a smaller effect on the result. To ensure that the queries are also based on the same underlying data structure, we repeat this step after the second half of inserts. Despite this instability, Figure 4.13a clearly shows that the inserts become significantly slower in the overall sequence of 70 commits and the change detection algorithm also detects this definite trend in the last ten commits. While simple queries improve during our study period by almost 6%, the performance history of group-by queries shows more change points. Starting with commit 10, the performance of complex group-by queries improves initially, then degrades from commit 20 to 23, and improves again with commit 59.

Figures 4.14a to 4.14c show the detected performance jumps and trends for *InfluxDB* along with the measured relative performance history. Besides several detected (potential) jumps and trends, both query types are significantly improved through commit 37 and there is one major drop for all request types introduced with commit 48.

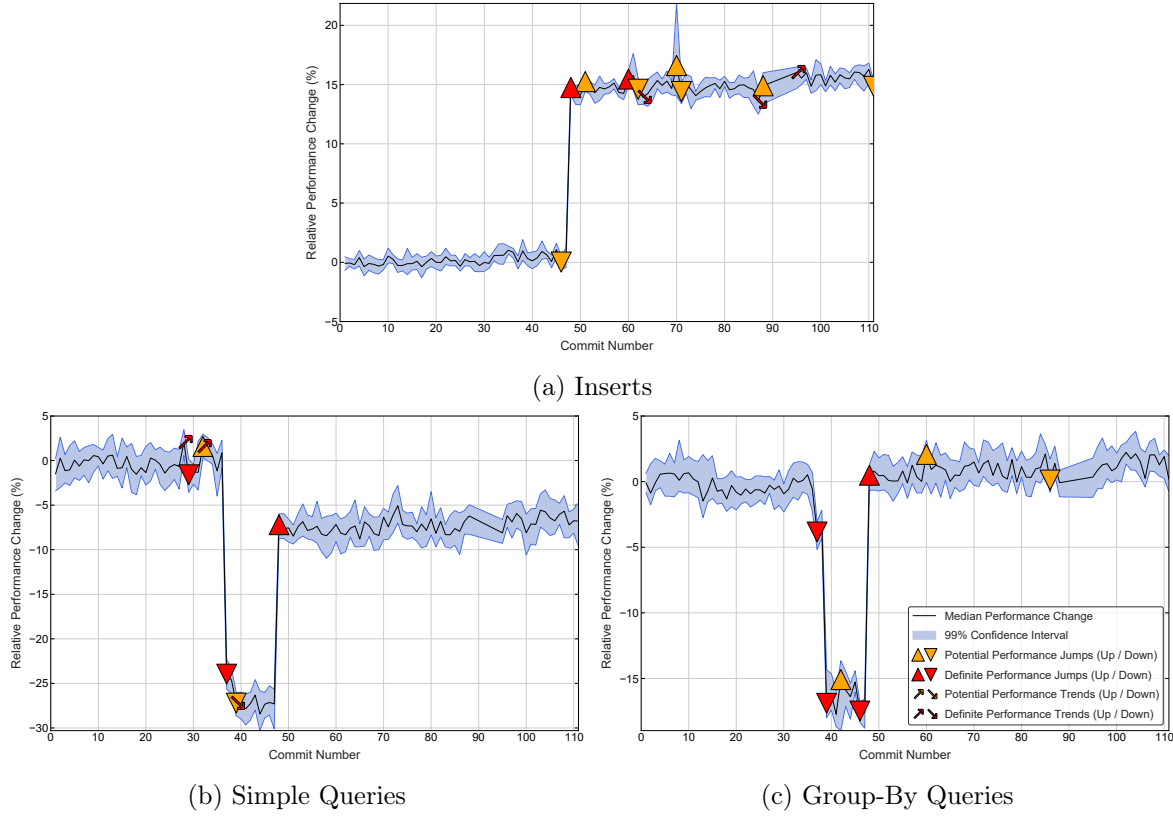


Figure 4.14: Application benchmark results for *InfluxDB*. Negative values show an improvement. There are two large definite performance jumps at (i) commit 48 for all request types and at (ii) commit 37 for both query types. Moreover, there are several (potential) jumps and trends for all request types.

The corresponding commit message for commit 37, `feat(query/stdlib): promote schema and fill optimizations from feature flags [93]`, signals that a new feature successfully speeds up simple queries by around 25% and group-by queries by around 15%. This improvement, however, is reversed in commit 48 through the activation of profiling. The corresponding commit message for commit 48, `feat(http): allow for disabling pprof [93]`, and the code changes indicate that this commit activates the costly profiling of Go by default. Looking at the total study period, simple queries are improved by about 5% by the end of the evaluation and group-by queries show a slight regression. Overall, we detect performance changes for all request types in both study objects.

4.3.4 Detected Performance Changes Using Optimized Microbenchmark Suites

After generating the call graphs for both application benchmark and microbenchmark suite, we determine the practical relevance for both SUTs, and find an optimized microbenchmark suite based on the approach described in Section 4.2.6.

Computing Optimized Suites:

For *VictoriaMetrics*, the call graph analysis shows that 634 project functions are called during

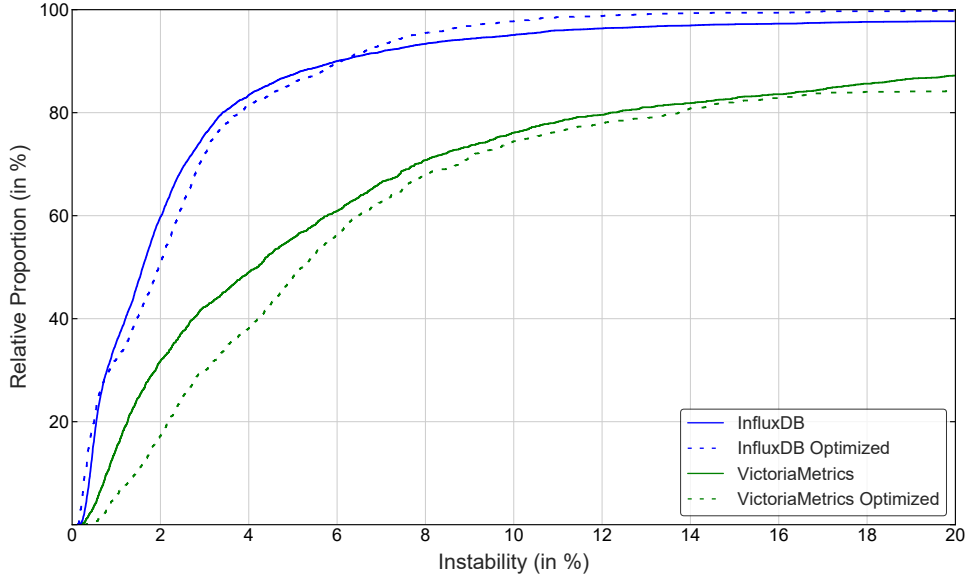


Figure 4.15: Approx. 80% of the microbenchmarks in the respective suites of *InfluxDB* show an instability of less than 4%. For *VictoriaMetrics*, however, only approx. 50% of the microbenchmarks show an instability of less than 4%.

the application benchmark of which 314 are covered by microbenchmarks, thus indicating a practical relevance of $\approx 49\%$. In the next step, by removing redundancies in the suite, the same relevance is already achieved with only 17 microbenchmarks. Many of these microbenchmarks, however, cover only a few additional nodes (three or less) of the application benchmark. Thus, we use only the eight most relevant microbenchmarks for our further analysis, which corresponds to a 47% practically relevant microbenchmark suite.

Initial experiments with the microbenchmark suite of *InfluxDB* showed that there are major changes in the microbenchmark suite in the first 15 commits, which also affects our potentially optimized microbenchmark suite: a performance comparison of two versions of the microbenchmark is only possible if this microbenchmark is also present in both versions and has not been changed. Due to the fact that some of the most relevant microbenchmarks in the suite optimized for commit 1 are missing in commit 15, we set the base version to commit 15 and shorten the evaluation period for the microbenchmarks.

The practical relevance of *InfluxDB*’s complete microbenchmark suite is around 40% at commit 15 (269 of 660 nodes overlap). After computing the optimized suites without redundancies, many of the 26 proposed microbenchmarks add only a few additional nodes to the overlap (three or less). Thus, similar to *VictoriaMetrics*, we continue with only the 10 most relevant microbenchmarks ($\approx 36\%$ practical relevance).

Determining Initial Detection Thresholds:

Figure 4.15 shows cumulative distribution functions for the instabilities of all microbenchmark suites in A/A experiments. While the microbenchmarks of *InfluxDB* are very stable and $\approx 80\%$ of the measurements have an instability below 4%, the performance of *VictoriaMetrics*’s micro-

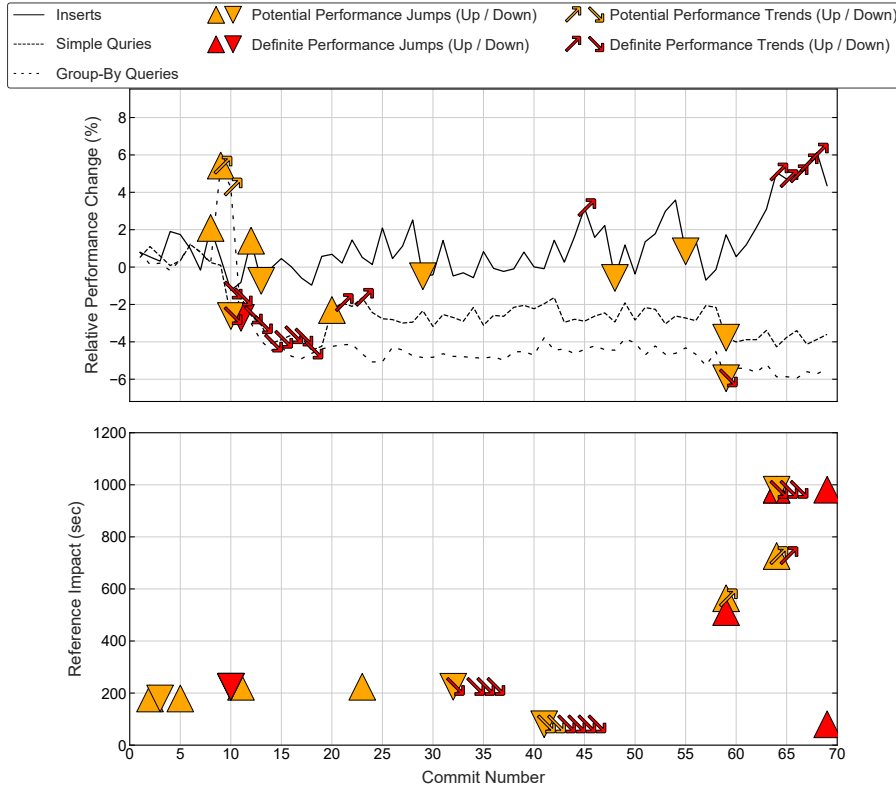


Figure 4.16: Results from the optimized suite for *VictoriaMetrics*. The upper part shows the results of the application benchmark and its detections while the lower part shows the detections from the microbenchmarks (higher means more likely impact on application performance). The optimized suite detects an insert-related change at commit 10, a performance change for queries at commit 59, and slower inserts at commit 64 and 65.

benchmark suite(s) fluctuates more. Here, only $\approx 50\%$ show an instability less than 4%. Thus, to cover $\approx 80\%$ of the microbenchmarks instabilities in the respective suites with the initial detection threshold (see Section 4.3.2, Jump and Trend Detection), we choose a general starting threshold of 12% for *VictoriaMetrics* and 6% for *InfluxDB* for our change detection, i.e., only experiments exceeding these thresholds in the first code changes are classified as performance changes. In the subsequent code changes, this threshold adapts to the respective microbenchmark’s instability and the algorithm will detect changes more reliably.

Detected Changes for *VictoriaMetrics*:

Running both optimized microbenchmark suites detect multiple potential and definite performance changes for several microbenchmarks. Figures 4.16 and 4.17 combine these signals with their corresponding reference impacts and evaluated performance metrics from the application benchmark. Ideally, any relevant performance change in the application benchmark (line chart in the upper part of the figure) should also be detected by a microbenchmark with a large reference impact (lower part of the figure). Nevertheless, because the microbenchmark suites in general only cover 47% and 36% of the application benchmark, we cannot expect to detect all changes.

Most of the signals in *VictoriaMetrics*' optimized microbenchmark suite originate from microbenchmarks with a reference impact of about 200 seconds in the application benchmark. With a total execution duration of the application benchmark of around 30 minutes (without setup), their covered functions are responsible for approximately 10% of the execution duration of the application benchmark.

The first three potential jumps are false positives due to the moving dynamic threshold. At the beginning of the evaluation period, the dynamic threshold is not yet adjusted to the observed instability. Therefore, we do not consider them further.

The first definite change in commit 10 and the next potential jumps and definite trend until commit 36 originate from the microbenchmark `BenchmarkAddMulti`, evaluating a `fast set for uint64` [94] using buckets, which indicates a relevance for inserts. The change detection of the application benchmark, on the other hand, also identifies a definite trend and faster inserts at commit 10. Visual analysis shows that all further signals of the microbenchmark are not clearly reflected in the performance of the inserts at *VictoriaMetrics*.

The detected changes from commit 41 to 46 refer to the microbenchmark `BenchmarkRowsUnmarshal`, which evaluates the unmarshalling of the Influx line protocol (which we use in our benchmarking client). Similarly to the signals before, these are not reflected in the application benchmark's detected changes.

The next definitive change correlates with another potential trend and jump at commit 59. The corresponding microbenchmarks `BenchmarkMergeBlockStreamsFourSourcesBestCase` and `BenchmarkMergeBlockStreamsFourSourcesWorstCase` merge multiple block streams and are related to queries. Although the correlated benchmarks have a longer average execution time, both query types improve in the application benchmark at commit 59 (we discuss this in Section 4.4).

The microbenchmark change detection then raises signals at commit 64 and 65 for microbenchmarks related to insert requests. These significant signals with a reference impact of 981 seconds (`BenchmarkStorageAddRows`) and 726 seconds (`BenchmarkIndexDBAddTSIDs`) are also significantly noticeable in the application benchmark.

Finally, the microbenchmarks `BenchmarkRowsUnmarshal` and `BenchmarkStorageAddRows` detect a definite change at commit 69. This change, however, is not visible in the application benchmark.

In total, the optimized suite detects four true positives (commits 10, 59, 64, and 65), but also raises false alarms for 17 commits. On the other hand, the optimized suite does not detect the negative performance trend of group-by queries starting with commit 20 (we discuss false negatives in more detail later).

Detected Changes for *InfluxDB*:

The optimized suite of *InfluxDB* detects several potential and definite performance changes in five different microbenchmarks (see Figure 4.17).

4.3. Case Study on the Detection Capabilities of Microbenchmark Suites

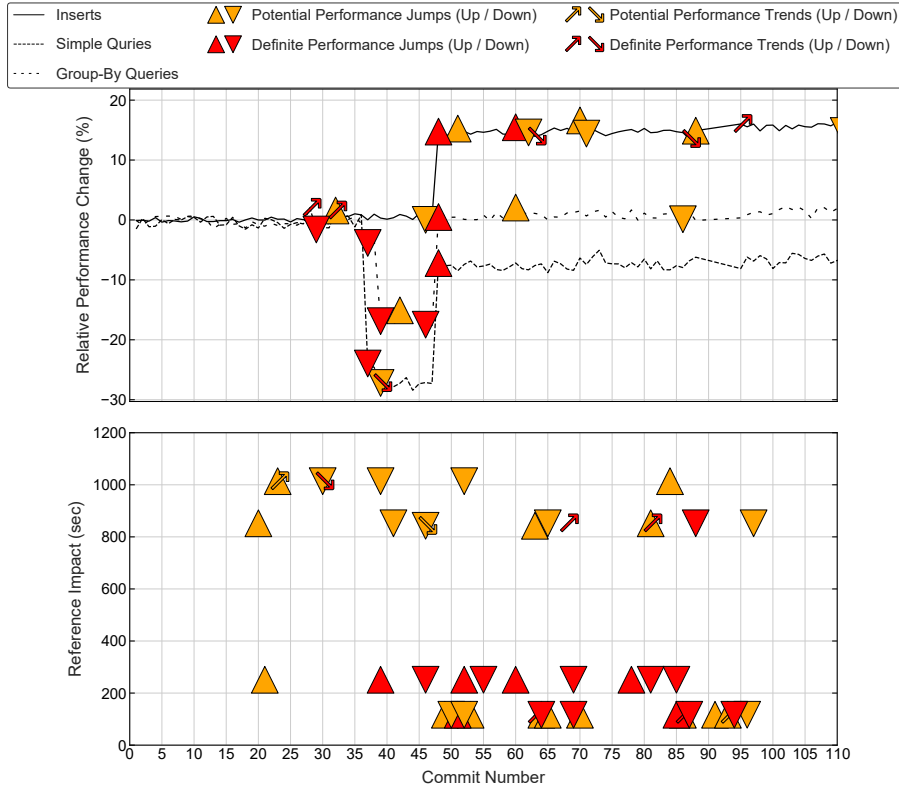


Figure 4.17: Results from the optimized suite for *InfluxDB*. The upper part shows the results of the application benchmark and its detections while the lower part shows the detections from the microbenchmarks (higher means more likely impact on application performance). The optimized suite of *InfluxDB* with $\approx 36\%$ practical relevance detects five true positive alarms but also raises false alarms for 27 commits.

The microbenchmark with the largest reference impact, **BenchmarkCreateIterator** (1,013s impact), accounts for around 40% of the application benchmark’s execution duration and benchmarks the creation of iterators for shard data items. This query-related benchmark identifies five potential performance changes for the commits 23, 30, 39, 52, and 84. While the commits 23, 30, and 52 are configuration-related code changes, which are unlikely to have an impact on application performance, commit 39 and 84 introduce larger changes. Commit 39 updates a flux dependency and this improvement is also visible in the application benchmark for both query types. Commit 84 adds a profiler option and modifies 68 lines in the `query.go` file. The query performance in the application benchmark, however, is not affected by this change.

The second most relevant microbenchmark is named **BenchmarkWritePoints** (942s impact) and it “benchmarks writing new series to a shard” [93], thus affecting insert requests. The first three signals are potential changes at commits 20, 41, and 65, which introduce minor features or fix small bugs. None of the three potential detected changes are visible in the application benchmark. The detected definite trend at commit 68 is caused by a minor configuration-related change. Neither this one, nor the changes from the previous commits (the root cause for the trend detection might also be in earlier commits), however, show any performance change in application performance. Next, commit 81 introduces an optimization which is identified as a

potential jump and a definite trend. This optimization, however, does not have any influence on the application performance. Commit 88 is a minor change but also updates the flux dependency. The application benchmark, on the other side, also detects a performance change for inserts. Finally, there is a minor fix at commit 97 identifying a potential change which is not relevant for application performance.

The third most relevant benchmark `BenchmarkParsePointsTagsUnSorted` (842s impact) benchmarks parsing of values and detects potential changes at commit 46 and 63 which are both also detected by the application benchmark. Commit 46 changes a default parsing option and this is also reflected in a potential improvement signal for inserts and a definite one for group-by queries in the application benchmark. The change introduced with commit 63 prevents a formatting of time strings in certain situations. This improvement is also detected as a potential improvement for inserts in the application benchmark.

The next microbenchmarks, `BenchmarkDecodeFloatArrayBlock` (251 seconds) and `BenchmarkIntegerArrayDecodeAllPackedSimple` (116 seconds), decode array blocks of float64 and integer values and have a significantly lower reference impact. The float benchmark detects one potential and nine definite changes, but only the changes at commit 39 (already identified by the most relevant microbenchmark), 46 (already identified by the third most relevant microbenchmark), and 60 are also detected by the application benchmark. Commit 60 fixes a cache-related race condition and this also impacts the performance of inserts and simple queries in the application benchmark. All other signals, however, are false positives. The least relevant integer benchmark detects changes for 17 commits. Here, five of the 17 signals (for commits 51, 63, 70, 87, and 94) correspond to the signals of the application benchmark for inserts and one matches a detection for grouping queries (commit 86). Nevertheless, because all changes introduce only minor features and smaller bug fixes, which are not related to any core functionality, we assume no direct correlation and consider all of them as false positives.

In total, the optimized suite detects five true positives (commits 39, 46, 60, 63, and 88), but also raises false alarms for 27 commits. Moreover, the optimized suite could not detect the two major performance changes at commit 37 and 48. While the performance change at commit 37 might not be detected because there is no microbenchmark covering the relevant code sections, the change at commit 48 can not be detected because it is related to the runtime environment.

4.3.5 Detected Performance Changes Using Complete Microbenchmark Suites

Running the complete suite with hundreds of microbenchmarks for each commit in practice is unrealistic, as it is too expensive and time-consuming to do so. Nevertheless, to rate the improvement and better compare the optimization technique to this alternative, we execute the complete suite for every fifth commit. In contrast to the optimized suite, we cannot use the reference impact to rank the results because many microbenchmarks do not overlap or only barely overlap with the reference application benchmark call graph. Thus, we aggregate the respective detection when interpreting the results, e.g., if ten microbenchmarks detect a definite

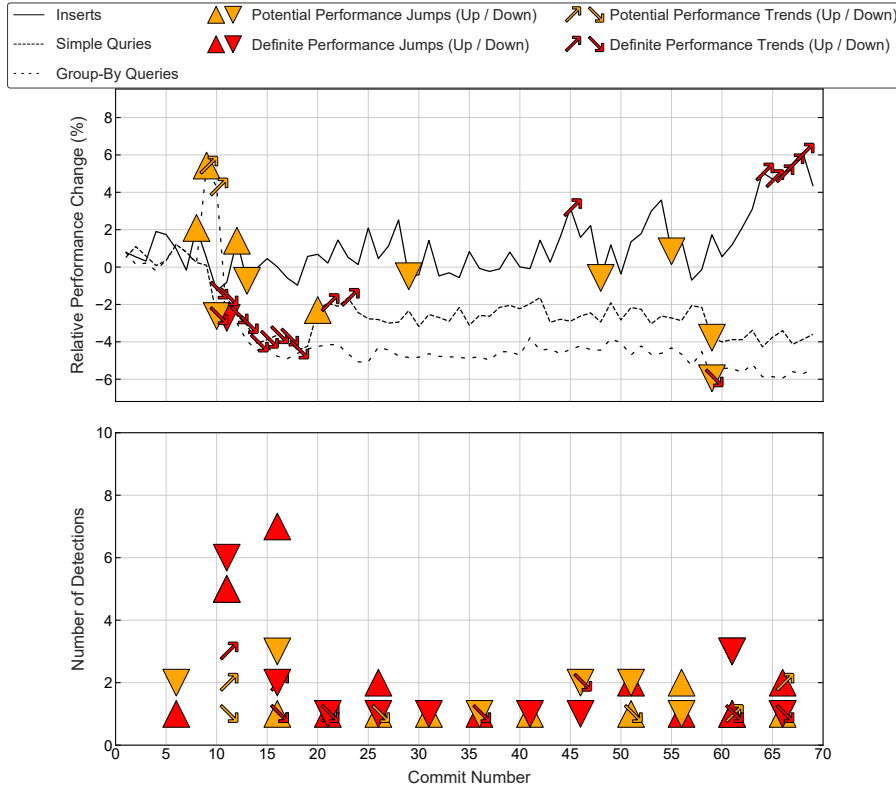


Figure 4.18: Complete suite results for *VictoriaMetrics*. The complete suite with 177 microbenchmarks in total detects 91 changes that can not be directly linked to the application-relevant performance metrics.

change, this change might be practically relevant. Moreover, we also adapt the dynamic detection threshold to the evaluation of every fifth commit only and consider only the last three values (instead of 10) for the trend detection.

Figures 4.18 and 4.19 show the application metrics (above) and the signals from the complete microbenchmark suite (below).

VictoriaMetrics’s complete microbenchmark suite detects 91 changes in total and those cover all evaluated code changes. In particular, we observe that there are not only multiple signals for each commit, but that these signals are also often contradictory (Chen et al. also report this phenomenon [36]). Except for commit 46, where all detected changes are improvements, there are always at least one microbenchmark each measuring a performance degradation and improvement respectively.

The complete suite of *InfluxDB* detects 392 performance changes in total. Similarly, the suite detects contradictory performance changes at each commit and these cannot be matched with the metrics of the application benchmark.

Both complete suites detect hundreds of performance changes at high cost but only some of them are relevant. Without further information and criteria, such as an impact or relevance factor, it is impossible to identify the relevant ones.

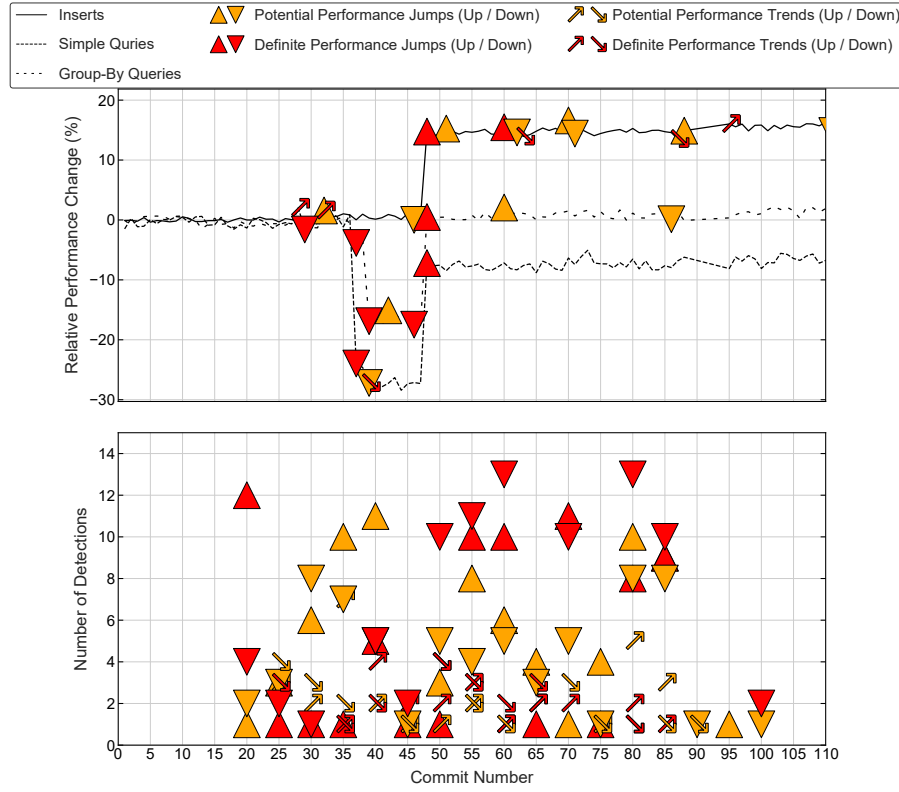


Figure 4.19: Complete suite results for *InfluxDB*. The suite shrinks down from 426 to 109 microbenchmarks during the evaluation period. Nevertheless, the complete suite detects 392 performance changes that can not be mapped to the application-relevant metrics.

4.3.6 Implications

In total, we examined 180 code changes in two open-source TSDBs using 540 application benchmark runs, 495 executions of optimized microbenchmark suites, and 102 runs of complete microbenchmark suites. These correspond to approximately 1,900 hours of benchmark execution duration. Despite this vast number of experiments, we cannot demonstrate a clear benefit of using an optimized microbenchmark suite: while some benefits exist, there are limitations. Overall, our results help to better understand the trade-off between the execution of application benchmarks, optimized, and complete microbenchmark suites (see Tables 4.9 and 4.10).

Application Benchmarks:

The setup of an automated application benchmark is complex and time-consuming. It requires scripts for starting the SUT and client instances, triggering and orchestrating the benchmark, collecting the measurements, and finally for analyzing the measurements. Running this complete pipeline took about 40min for *VictoriaMetrics* and 130min for *InfluxDB* in our experiments, which corresponds to costs of about \$0.13 and \$0.40 per experiment repetition. Once set up, however, an application benchmark is a great tool for reliably detecting performance regressions or improvements in code changes. In our studied systems, this advantage can be illustrated especially with *InfluxDB*: A clear improvement caused by a new feature and a clear drop caused

4.3. Case Study on the Detection Capabilities of Microbenchmark Suites

Benchmark	<i>VictoriaMetrics</i>	<i>InfluxDB</i>
Application Benchmark	$\sim 40min$ ($\sim \$0.13$)	$\sim 130min$ ($\sim \$0.40$)
Optimized Suite	$\sim 20min$ ($\sim \$0.03$)	$\sim 40min$ ($\sim \$0.06$)
Complete Suite	$\sim 4h$ ($\sim \$0.38$)	$\sim 11h$ ($\sim \$1.05$)

Table 4.9: Benchmarking durations and prices. Running complete microbenchmarks suites takes a lot of time while an optimized suite is faster and less expensive than an application benchmark.

by a misconfiguration (which is impossible to detect using a microbenchmark) can be directly linked to specific commits. Furthermore, although small performance shifts between two successive commits may not be detected due to variability, an application benchmark can also be used to reliably detect performance trends. We can observe this characteristic especially for *VictoriaMetrics*: both query types show performance improvements between commit 10 and 20, but due to the large confidence intervals the changes cannot be directly connected to a single commit. Finally, our experiments also show that the Duet Benchmarking technique can not be applied everywhere without further modifications. Due to a non-deterministic characteristic of *VictoriaMetrics*, it is difficult to evaluate insert operations accurately.

Our experiments show that a well-designed application benchmark can reliably detect performance changes even in highly variable cloud environments. In our use cases, an application benchmark is relatively fast and cost-efficient, because we have chosen a rather simple setup with only two instances. In more complex setups using more complex application benchmarks, however, the cost per benchmark run will be higher, and the execution may also take longer. These more complex benchmarks include different load scenarios, involve many more instances and components, or evaluate the impact of changes in the environment, e.g., network fluctuations or (temporal) outages of individual components [81, 175]. Thus, depending on the frequency of code changes, we argue that an application benchmark should usually be scheduled to run daily, weekly, or after major code changes.

Complete Microbenchmark Suite:

Running the complete microbenchmark suites of our evaluated projects takes around 4h for *VictoriaMetrics* and 11h for *InfluxDB*. Thus, evaluating one commit using one single experiment costs about \$0.38 for *VictoriaMetrics* and about \$1.05 for *InfluxDB*. For reliable measurement results, this single experiment should be run at least 3 times (concurrently), thus multiplying the cost. Furthermore, if a microbenchmark detects a performance change, the exact evaluation of the results is still hard due to the large number of experiments, instability of microbenchmarks, and

Project Benchmark	Number of definite changes (+potential)					
	<i>VictoriaMetrics</i>			<i>InfluxDB</i>		
	App	Opti	Full	App	Opti	Full
Jump up	0(+4)	5(+6)	23(+14)	4(+6)	6(+14)	72(+77)
Jump down	1(+5)	4(+4)	17(+15)	5(+6)	10(+10)	83(+67)
Trend up	8(+2)	1(+2)	6(+5)	3(+0)	4(+2)	20(+30)
Trend down	11(+0)	11(+2)	7(+4)	3(+0)	1(+1)	19(+24)

Table 4.10: Result summary. While application benchmark (app) and optimized microbenchmark suite (opti) detect a rather small number of performance changes, the complete suite (full) finds a lot more.

it is often unclear to which degree a change affects the production environment and application-relevant metrics. For example, when running the complete suites for every fifth commit, we observed hundreds of performance changes in the microbenchmarks (see Table 4.10), but these were not reflected in the application-relevant benchmark metrics.

Running and evaluating a complete microbenchmark suite is usually expensive, takes a long time, is difficult to evaluate, and hardly yields any findings or findings that are difficult to interpret. If code changes happen at intervals of minutes or hours, then this type of benchmark is only suitable for daily or weekly performance evaluations. Nevertheless, a complete run can help to analyze a detected performance problem in more detail, help to isolate the issue, and find the root cause. Hence, it could be triggered whenever an application benchmark run has identified a performance change.

Optimized Microbenchmark Suite:

The optimized microbenchmark suite without redundancies runs much faster (around 20min and \$0.04 for *VictoriaMetrics*; around 45min and \$0.07 for *InfluxDB*), is easier to evaluate and, if covering practically relevant parts, can detect the same performance changes that can also be detected by an application benchmark (see Table 4.10). Using optimized microbenchmark suites, we can identify four true positive signals for *VictoriaMetrics* and five true positives for *InfluxDB*. Nevertheless, both optimized suites also raise false alarms, especially through microbenchmarks with a low reference impact (which is part of the reason that the full suite detects so many false positives).

Running only practically relevant microbenchmarks significantly reduces the execution duration and also simplifies the analysis of the results. If the optimized suite covers a large portion of the practically relevant code sections, the suite can quickly detect performance changes and link them to specific commits at low cost. On the other hand, if performance changes relate to the runtime environment, integration, and or interaction of different application components, the microbenchmark suite cannot detect them. The profiling setting, which caused a significant performance drop in *InfluxDB* at commit 48, cannot be found in the microbenchmarks because

it was caused by a general configuration in the production(-like) environment. Another problem when using the microbenchmark suite are the benchmarks within the suite itself. If the suite changes often and especially if this involves the most relevant microbenchmarks with a large reference impact, then a continuous comparison is not possible and the optimized suite has to be re-determined periodically. In our experiments, this problem affects *InfluxDB* twice: once at the beginning of the evaluation period (commit 15); and once at commit 80. Thus, while an optimized benchmark suite can evaluate the performance several times a day, this benchmarking strategy should not be the only benchmarking approach used.

4.3.7 Findings

For the two evaluated systems, the results show that performance changes can be reliably detected by running an application benchmark for less than one hour and that a reduced and optimized microbenchmark suite can detect the same changes with less than ten microbenchmarks. Our experiments identify nine true positive signals for optimized microbenchmark suites. Nevertheless, our study also shows the limitations of the optimization approach and which type of performance issues cannot be identified with an optimized suite. First, the optimized suite does not detect the application performance changes if the microbenchmarks do not cover the practically relevant code sections. Second, performance changes related to the concrete runtime environment may not be detected. Third, if a performance change is detected by a microbenchmark, its impact on application performance is hard to predict. The optimized suites hence often identify false positives.

Derived from this, we envision that a good continuous benchmarking strategy should, e.g., combine a fast and relevant optimized microbenchmark suite and a well-designed application benchmark. While the optimized suite provides an early performance feedback for almost every code change, e.g., as part of a local build process or routine action which is triggered for each submitted code change, the regular runs of a well-designed application benchmark, e.g., once per day, ensures that the desired performance metrics are met. This allows developers to get quick performance feedback for each change, so they can adjust their changed code sections if necessary. Because optimized suites may not or even cannot find all problems, a daily application benchmark run serves as backup to reliably detect the remaining ones and report them the next day.

4.4 Discussion

We proposed an automated approach for analyzing and improving microbenchmark suites. It can be applied to all application systems that allow the profiling of function calls and the subsequent creation of a call graph. This is particularly easy for projects written in the Go programming language as this functionality is part of the Go environment. Furthermore, our approach is beneficial for projects with a large code base where manual analysis would be too complex and costly. Our experiments show that optimized microbenchmark suites can detect application

performance changes in certain situations. While both micro- and application benchmarks may not be suitable for more detailed analysis of every commit in large projects with many code changes, they are still suitable for daily (or nightly) and weekly use as well as for a more detailed analysis after major changes. An optimized microbenchmark suite covering large practically relevant code parts can complement this by providing a fast performance feedback. In total, we proposed three methods for analyzing and optimizing existing microbenchmark suites but can also provide guidance for creating new ones. Nevertheless, there are some limitations and possible extensions which we discuss in the following.

An Application Benchmark is Realistic but not Perfect:

Assuming that the application benchmark reflects a real production system or simulates a realistic situation, the resulting call graphs will reflect this perfectly. Unfortunately, this is not always the case, because the design and implementation of a sound and relevant application benchmark has its own challenges and obstacles which we will not address here [25]. Nevertheless, a well-designed application benchmark is capable of simulating different scenarios in realistic environments in order to identify weak points and to highlight strengths. Ultimately, however, for the discussion that follows, we must always be aware that the application benchmark will never be a perfect representation of real workloads. Trace-based workloads [19] can help to introduce more realism.

Limitations of Application Benchmarks:

Application benchmarking offers the possibility of placing the evaluated system in any requested situation. From examining increased usage during holiday season to studying the effects of component failure, application benchmarks can be implemented for many situations. Nevertheless, they effectively use an artificial load and do not run on the production system, which also has limitations. For example, the actual production load may not match the load assumed by the benchmark, resulting in different results and implications. In addition, not all use cases can provide a second environment that can be used for benchmarks. For example, in IoT scenarios, it is hardly possible to maintain a second identical building with the same smart home devices just for testing and benchmarking purposes. Alternatively and or complementary to application benchmarks, among others, application performance monitoring, gradual roll-outs, or dark launches can be used to detect performance changes in production.¹⁵ While benchmarking is used before deploying to production and does not affect real users, live testing techniques such as gradual roll-outs are applied in the real production environment. Ideally, there should be a holistic combination of approaches from both phases, before and during live deployment.

Considering only Function Calls is Imperfect but Sufficient:

Our approach relies on identifying the coverage of nodes in call graphs and thus on the coverage of function calls. Additional criteria such as path coverage, block coverage, line coverage, or the frequency of function executions are not considered and subject to future research. We delib-

¹⁵If it is possible to record call graphs in the production environment, these graphs can also be used as (a real) reference to compute the optimized suite.

erately chose this simple yet effective method of coverage measurement: (1) Applying detailed coverage metrics such as line coverage would deepen the analysis and check that every code line called by the application benchmark is at least once called by a microbenchmark. However, if the different paths in a function source code are relevant for production and do not only catch corner cases, they should be also considered in the application benchmark and microbenchmark workload (e.g., if the internal function calls in the Medical Monitoring scenario would differ for female and male patients, the respective benchmark workload should represent female and male patients with the same frequency as in production). (2) As our current implementation relies on sampling, the probability that a function that is called only once or twice during the entire application benchmark or microbenchmark is called at the exact time a sample is taken is extremely low. Thus, the respective call graphs will usually only include practically relevant functions. (3) We assume that all benchmarks adhere to benchmarking best practices. This includes both the application benchmark, which covers all relevant aspects, and the individual microbenchmarks, which each focus on individual aspects. This implies that if there is an important function, this function will usually be covered by multiple microbenchmarks which each generate a unique call graph with individual function calls and which therefore will all be included into the optimized microbenchmark suite. Thus, there will still usually be multiple microbenchmarks which evaluate important functions. (4) Both base algorithms [38, 159] are standard algorithms and have recently been shown to work well with modern software systems, e.g., [125]. We therefore assume that a relevant benchmark workload will generate a representative call graph and argue that a more detailed analysis of the call graph would not improve our approach significantly. The same applies to the microbenchmarks and their coverage sets with the application benchmark where our approach will only work if the microbenchmark suite generates representative function invocations. Overall, considering only function calls is imperfect but should be sufficient to determine a practically relevant microbenchmark suite if both benchmark techniques follow the best practices.

The Sampling Rate Affects the Accuracy of the Call Graphs:

The generation of the call graphs in our evaluation is based on statistical sampling of stack frames at specified intervals. Afterwards, the collected data is combined into the call graph. However, this carries the risk that, if the experiment is not run long enough, important calls might not be registered and thus will not appear in the call graph. The required duration depends on the software project and on the sampling rate, i.e., at which frequency samples are taken. To account for this, we chose frequent sampling combined with a long benchmark duration in our experiments which makes it unlikely that we have missed relevant function calls.

The Practical Relevance of a Microbenchmark Suite can be Quantified Quickly and Accurately:

Our approach can be used to determine and quantify the practical relevance of a microbenchmark suite based on a large baseline call graph (e.g., generated from an application benchmark) and

many smaller call graphs from the execution of the microbenchmark suite. On one hand, this allows us to determine and quantify the practical relevance of the current microbenchmark suite with respect to the actual usage. In our evaluation in Section 4.2 using two different TSDBs, we found that this is about 40% for both databases. On the other hand, this means that about 60% of the required code parts for the daily business are not covered by any microbenchmark, which highlights the need for additional microbenchmarks to detect and ultimately prevent performance problems in both study objects. Moreover, it may be necessary to manually remove all microbenchmarks that do not adhere to benchmarking best practices before running our algorithms. In summary, we offer a quick way to approximate coverage and practical relevance of a microbenchmark suite in and for realistic scenarios.

A Minimal Microbenchmark Suite with Reduced Redundancies can be Used as Performance Smoke Test:

Our first optimization to an existing microbenchmark suite, Algorithm 2, aims to find a minimal set of microbenchmarks which already cover a large part of an application benchmark, again based on the nodes in existing call graphs. Our evaluation and case study has shown that a very small number of microbenchmarks is sufficient for achieving a large part of the potential maximum coverage for both study objects. Translated into execution time, this removal of redundancies corresponds to savings from about 11 hours to about 40 minutes for *InfluxDB*. Moreover, our case study has shown that a minimal microbenchmark suite can show developers a rough performance impact of their current changes. It enables developers to run a quick performance validation on each commit, or to quickly evaluate a new version before starting a more complex and cost-intensive application benchmark. In this setup, the application benchmark remains the gold standard for detecting all performance problems while the less accurate optimized microbenchmark suite is a fast and easy-to-use performance check. Finally, it is important to note that the intention of our approach is not to remove "unnecessary" microbenchmarks entirely but rather to define a new microbenchmark suite as a subset of the existing one which serves as a proxy to benchmarking the performance of the SUT. Although our evaluation also revealed that many microbenchmarks benchmark the same code and are therefore redundant, this redundancy is frequently desirable in other contexts (e.g., for detailed error analysis).

The Recommendations cannot Always be Directly Used:

Our second optimization, Algorithm 3, recommends functions which should be microbenchmarked in order to cover a large additional part of realistic application flow in the SUT. Our evaluation with two open-source TSDBs has shown that this is indeed possible and that already with a small number of additional microbenchmarks a large part of the application benchmark call graph could be covered. However, our evaluation also suggests that these microbenchmarks are not always easy to implement, as the recommended functions are often very generic and abstract. Our recommendation should therefore mostly be seen as an initial point for further manual investigation by expert application developers. Using their domain knowledge, they can

estimate which (sub)functions are called and what their distribution/ratio actually is. Furthermore, the application benchmark's call graph can also support this analysis as it offers insights into the frequency of invocation for all covered functions.

The Trade-Off Between Cost and Accuracy:

Within our case study in Section 4.3, we can produce reliable and reproducible results with three experiment repetitions. Nevertheless, several microbenchmarks show wide CIs of more than 20% and are unstable. For each project, it is thus essential to find a good compromise between effort and cost on one side, and accuracy and reliability on the other side.

Besides narrowing the CI width through additional experiment repetitions, which also increases cost accordingly, there are further optimizations by stopping benchmark runs under certain conditions or predicting unstable ones [4, 5, 84, 111, 115]. For example, stopping benchmarks as soon as there is a reliable finding might shorten the benchmark duration, excluding unstable microbenchmarks might avoid unnecessary effort, or multiple smaller microbenchmarks might be more reliable and thus more cost-effective than a large unstable one. In our study, excluding a large unstable microbenchmark would just reduce the practical relevance by removing some microbenchmarks from the optimized suite of both study objects without adding equally relevant ones. Thus, this is subject to further research.

Changes in the Optimized Suite over Time:

In our case study, we use a fixed code state to optimize the microbenchmark suite, i.e., commit No. 0 for *VictoriaMetrics* and commit No. 15 for *InfluxDB*. This base version should not be changed as long as possible to generate a long measurement series for trend detection. Nevertheless, there are situations in both application benchmarks and microbenchmarks where this base version needs to be reset and the optimization needs to be repeated. Thus, the optimized suite cannot be considered static and has to be changed from time to time.

Both types of benchmark require a new base version when the benchmark itself is modified. Regarding the application benchmark this is, e.g., the case if the workload is no longer realistic and needs to be adjusted (e.g., the number of customers has doubled, which means twice as many requests in the production system). An adjusted application benchmark will imply a changed reference call graph and updated reference impact values.

Regarding the microbenchmarks, for example, there is the modification of the invocation parameters and that individual microbenchmarks might be removed (as can be seen in our experiments) or new ones might be implemented. Moreover, as every commit modifies the code that is evaluated by the microbenchmarks, the respective microbenchmark call graphs needs to be updated as well. Both changes may require changes to the optimized suite as well.

Identifying False Alarms:

In our case study, both optimized suites detect nine true positive performance changes but also raise false alarms for 44 commits in total. These false alarms are caused, among other things,

by measurement inaccuracies, but can also be caused by the approach reacting to changes in non-practically relevant functions. For example, if the performance of a non-practically relevant function degrades, but this function is also evaluated by a microbenchmark with large reference impact (e.g., a function of MB1 in Figure 4.2 that is not covered by the application benchmark), then the performance of the microbenchmark will also degrade, even though this change has no impact on application performance. Confirming a detected change or spotting a false alarm would require starting an application benchmark in a realistic setup, which would imply corresponding costs. Thus, identifying false alarms in advance would be major improvement in further research.

Besides using the reference impact as additional classification for the reliability of signals, for example, the detected changes could be flagged and stored if the respective microbenchmark raised a false alarm. Using this history of changes, it might be possible to determine a reliability value for each microbenchmark which can be used to assess whether their detected performance change should be disregarded or not. A microbenchmark that successfully detected application performance changes in the past might also do this for future code changes.

Moreover, tagging microbenchmarks that are affected by a code change (i.e., only a fraction of the optimized suite), might also ease the result analysis. If one of those microbenchmarks raises an alarm, it is worth a detailed evaluation because there is a related code change. If a detected change is not raised by a tagged microbenchmark, it might be a false alarm. Furthermore, it might even be feasible to run only those microbenchmarks that cover modified functions.

Interpretation of Microbenchmark Performance Changes:

To derive concrete implications from statements such as “the performance of a microbenchmark MB1 has dropped by 5%”, there are several options. In the optimal case, application developers know the underlying logic of the respective microbenchmark, can directly relate a detected change to the target functionality (e.g., the request type), and rate the impact on the production environment. This is, however, not realistic, especially for large projects. We suggest interpreting and storing the detected changes as warnings which will trigger an application benchmark to verify the overall system performance and or to use them to support root cause analysis when a future application benchmark shows significant performance changes and the originating code change needs to be identified.

A strict policy that, for example, rejects a commit when a performance issue is detected by a microbenchmark would in many cases be incorrect. In our experiments, for example, a microbenchmark detected that the merging of block streams takes longer for *VictoriaMetrics* in commit 59 while the application benchmark observed faster queries. Because this corresponding code update merges 8 new features and fixes 6 bugs, we cannot identify the exact reason for this phenomenon due to its complexity, but we can find two possible explanations. First, even though merging the block streams takes longer because more data is processed, the query latency decreases because fewer streams need to be merged, thus resulting in fewer calls to the respective function while running the application benchmark. Second, while merging streams takes longer, another feature is introduced that improves the query latency but is not covered by the opti-

mized microbenchmark suite (yet). In such a scenario, the feature leading to the improvement might be the dominant code change while the microbenchmarks can only detect the less relevant degradation covered by the suite.

Implications for Production:

Our extensive experiments using two time series database systems show many interesting aspects when running optimized microbenchmark suites. Nevertheless, there is no general (micro-) benchmarking strategy that can simply be applied to every project. The strategy needs to be determined individually for each project and depends, among other things, on the general development progress, the number of code changes per day, the production environment, the expected load, and the impact of a potential performance issue.

Optimized microbenchmark suites can be a helpful tool for large projects with multiple developers, a large code base, and many code changes per day (e.g., our studied TSDBs). Here, a detailed performance evaluation of every code change is not possible, but optimized suites can help to evaluate these changes well enough, i.e., covering practically used code sections. In smaller projects it can also be useful to save costs. For example, a cost-intensive execution of application benchmarks for each code change possibly can be replaced with the optimized suite while the application benchmark is, e.g., executed only weekly, for every 10th commit, or for each major change.

As the concrete parameter values have to be defined individually for each project, we recommend analyzing past code changes and running some trial benchmarks first, e.g., to estimate variances. Based on these results, it is then possible to derive concrete values such as (micro-) benchmark frequency, detection thresholds, or actions in case of a detected performance change.

Further Improvements and Research Directions:

In our case study, we benchmark successive code changes in the commit history of two large open-source TSDBs and analyzed them in detail. Nevertheless, our findings cannot be generalized to all systems. There might be combinations of microbenchmark suites, SUTs, application benchmarks and their evolution over time in which microbenchmarks can detect performance regressions with neither false positives nor false negatives.

We have evaluated our approach with two TSDBs written in the Go programming language, but we see no major barriers to implementing our approach for applications written in other programming languages. There are several profiling tools for other programming languages, e.g., for Java or Python, so this approach is not limited to the Go programming language and is applicable to almost all software projects. The transfer to other application domains and programming languages is subject to future research.

We believe that our findings are representative for most real world combinations. This is based on the intuition that microbenchmark suites are unlikely to always have full code coverage of the SUT and that the functions studied by individual microbenchmarks may or may not have significant

effects on the execution duration of application benchmarks. Overall, our study motivates further research on the computation, usage, and advantages of optimized microbenchmark suites.

4.5 Summary

Performance problems of an application should ideally be detected as soon as they occur. Unfortunately, it is often not possible to verify the performance of every source code modification by a complete application benchmark for time and cost reasons. Alternatively, much faster and less complex microbenchmarks of individual functions can be used to evaluate the performance of an application.

In this chapter, we determined, quantified, and improved the practical relevance of microbenchmark suites based on the call graphs generated in the application during the two benchmark types and suggested how the microbenchmark suite can be designed and used more effectively and efficiently. The central idea of our approach is that all functions of the source code that are called during an application benchmark are relevant for production use and should therefore be covered by the faster and more lightweight microbenchmarks as well. To this end, we determined and quantified the coverage of common function calls between both benchmark types, suggested two methods of optimization, and illustrated how these can be leveraged to improve build pipelines: (1) by removing redundancies in the microbenchmark suite, which reduces the total execution duration of the suite significantly; and (2) by recommending relevant target functions which are not covered by microbenchmarks yet to increase the practical relevance.

After applying both optimizations, it is possible to cover a maximum portion of an application benchmark with a minimum suite of microbenchmarks which has several advantages. First of all, this helps to identify important functions that are relevant in practice and ensures that their performance is regularly evaluated via microbenchmarks. Instead of a suite that checks rarely used functions, code sections that are relevant for practical use are evaluated frequently. Second, microbenchmarks evaluating functions that are already implicitly covered by other microbenchmarks are selectively removed, achieving the same practical relevance with as few microbenchmarks as possible while reducing the runtime of the total suite. Furthermore, the effort for the creation of microbenchmarks is minimized because the microbenchmarks of the proposed functions will cover a large part of the application benchmark call graph and fewer microbenchmarks are necessary. Developers will still have to design and implement performance tests, but the identification of highly relevant functions for actual operation is facilitated and functions that implicitly benchmark many further relevant functions are pointed out, thus covering a broad call graph.

Our evaluation on two time series database systems showed that the number of microbenchmarks can be reduced significantly (up to 90%) while maintaining the same coverage level and that the practical relevance of a microbenchmark suite can be increased from around 40% to 100% with only a few additional microbenchmarks for both investigated software projects. Moreover, we explored to which degree application-relevant performance changes, such as an increase in query

latency, can also be detected by optimized microbenchmark suites. For this, we used the commit history of *InfluxDB* and *VictoriaMetrics* and studied them by running extensive benchmark experiments. Using application benchmarks, our coverage-based optimization strategy for microbenchmark suites, and running complete suites, we could show that this is indeed possible with some limitations. As we discovered, the approach requires that existing microbenchmarks cover (almost) all application-relevant code sections but still results in both false negative and false positive signals. Thus, optimized suites cannot be a proxy for a regular application benchmark but can provide a fast performance feedback at low cost after code changes in certain situations. For example, an optimized suite could be routinely run for (almost) every code change to detect most performance problems, while a more reliable application benchmark could be used as a daily backup process to detect the missed ones. In this sense, the application benchmark remains as the gold standard revealing all performance problems, while the optimized microbenchmark suite is an easy-to-use and fast heuristic which offers a quick insight into performance yet with obviously lower accuracy.

Overall, our findings open opportunities for practitioners to include new continuous benchmark steps in CI/CD pipelines and to shorten the execution times of established ones. Our results motivate further studies using other systems, developing further microbenchmark selection algorithms, and fine-tuning parameters to cost-efficiently improve the benchmark accuracy.

Benchmarking FaaS Platforms using the BeFaaS Framework

All major cloud providers offer FaaS solutions where users only have to take care of their source code (functions) while the underlying infrastructure and environment are abstracted away by the provider. FaaS applications are composed of individual functions deployed on a FaaS platform that handles, e.g., the execution and automatic scaling. Developers do not have direct control of the infrastructure and can only define high-level parameters, such as the region in which the function should run [17]. Due to this, FaaS platforms are easy to use but comparing cloud platform performance is challenging, as the cloud variability is further compounded by an additional, unknown infrastructure component [15, 118]. Existing work on benchmarking of FaaS platforms usually focuses on the execution of small, isolated microbenchmarks that deploy and call a single function, e.g., a matrix multiplication [11] or a random number generator [128]. While these benchmarks are useful for studying and comparing specific characteristics, they can give only limited insights into the platform behavior that will impact real applications [25].

In this chapter, we propose BeFaaS, an extensible framework for executing application-centric benchmarks against FaaS platforms. BeFaaS is the only FaaS benchmarking framework with out of the box support for federated cloud [110] and edge-to-cloud deployments [13, 16], which allows us to evaluate complex application configurations distributed over platforms running on a mixture of cloud, edge, and fog nodes. Beyond this, BeFaaS supports asynchronous cross-provider event pipelines, focuses on ease-of-use and collects fine-grained measurements which can be used for a detailed post-experiment drill-down analysis, e.g., to identify cold starts or trace request chains in detail. Currently, BeFaaS includes four benchmark applications which we use to evaluate several FaaS providers in realistic and typical FaaS application setups. Specifically, we (i) compare FaaS offerings using a typical microservice-oriented application, (ii) evaluate hybrid edge-cloud FaaS setups, (iii) analyze the event pipeline interaction within and between providers, and (iv) study cold start behavior of FaaS platforms.

This chapter presents the following contributions:

- We derive requirements for an application-centric FaaS benchmarking framework.
- We propose BeFaaS¹, an extensible and open-source framework for the execution of application-centric FaaS benchmarks and describe four example benchmark applications.
- We run a number of experiments and use the results to compare FaaS offerings in several setups.

¹<https://github.com/Be-FaaS>

Our study evaluates four different typical FaaS use cases on Amazon Web Services (AWS), Google Cloud Platform (GCP), Microsoft Azure (Azure), and tinyFaaS [149]. For simple functions which work as glue code between frontend and storage layer, network transmission is a major contributor to overall response latency while the pure computing time of functions is almost negligible. Even with a cloud database system backend, an edge-only function deployment can outperform a mixed edge-cloud deployment in response latency because of transmission latency between functions. While publishing events to event pipelines can be done within $100ms$ for all studied providers, the delay between a published event and the start of the triggered function ranges between about $100ms$ for AWS and $800ms$ for GCP. Despite function execution duration being the highest on Azure Functions, our experiments find that the platform outperforms AWS Lambda and Google Cloud Functions in cold start behavior.

In total, BeFaaS can help FaaS application developers compare cloud offerings and find the best provider for their specific use case by either deriving findings from our benchmark applications, adjusting the workload profiles to match their scenario, or using the BeFaaS library in their specific FaaS application for most accurate findings. Furthermore, FaaS platform developers could use BeFaaS as part of their CI/CD pipelines [74, 183] to detect performance regressions prior to live testing.

This chapter contains (partially adapted) material from our published research articles [76, 77] and is structured as follows: First, we present the BeFaaS framework including requirements, design, and implementation in Section 5.1. We then use BeFaaS to study and compare several FaaS providers on various aspects in Section 5.2. Finally, Section 5.3 discusses the limitations of the framework and Section 5.4 summarizes our contributions and results.

5.1 The BeFaaS Framework

While current FaaS benchmarks are highly useful for studying individual features of a {sut, application-centric benchmarks support end-to-end comparison of different platforms and configurations. Aside from standard benchmarking requirements such as portability or fairness [19, 20, 25, 60, 92], an application-centric FaaS benchmarking framework needs to fulfill a number of specific requirements which we describe in Section 5.1.1. Next, we give an overview of the BeFaaS design in Section 5.1.2, starting with an overview of the BeFaaS architecture and components before describing the key features of BeFaaS. Finally, Section 5.1.3 describes the implementation and Section 5.1.4 the four application-centric benchmarks.

5.1.1 Requirements

Our specific requirements for the FaaS benchmark framework are:

R1 – Realistic Benchmark Application:

The performance of a FaaS platform depends on the application that is deployed on it. For instance, an application that frequently causes cold starts through a growing request rate will be

better off on AWS Lambda while an application that frequently causes cold starts through short temporary load spikes will be better off on Apache OpenWhisk due to their different request queuing mechanisms [18]. This means that the benchmark application should be as close as possible to the real application for which the analysis is made [25], in line with the findings of Shahradd et al. [173]. A key requirement is, hence, that *a FaaS benchmark should mimic real applications as closely as possible.*

R2 – Extensibility for New Workloads:

FaaS platforms are highly flexible and can be used for a wide variety of applications, so the world of FaaS applications is evolving rapidly. As such, any set of “typical” FaaS applications – and thus the workload profile for a FaaS platform – can only be considered a snapshot in time. Likewise, the load profiles of existing FaaS applications, i.e., the amount and type of requests that the application handles, are likely to evolve over time. Therefore, we argue that *a FaaS benchmarking framework should be easily extensible in terms of adding new benchmark applications and updating load profiles for existing benchmarks.*

R3 – Support for Modern Deployments:

FaaS is often used as the “glue” between cloud services, web APIs, and legacy systems [17]. Thus, a benchmarking framework must also consider these links and support external services. Furthermore, applications today are often distributed over cloud, edge, and fog resources, possibly even to the LEO edge [21, 150, 192]. Here, for example, edge servers can keep sensitive functions on premises while non-critical functions are hosted in a public cloud; similar setups exist for edge and fog computing use cases [9, 16, 70, 146]. As such, assuming a single-cloud deployment is unrealistic for benchmarks aiming to be as similar as possible to realistic applications. *A benchmarking framework needs to support external services and federated setups in which application functions are deployed on one or more FaaS platforms distributed across cloud, edge, and fog.*

R4 – Extensibility for New Platforms:

Today, all major cloud service providers offer FaaS platforms and there is a growing range of open-source FaaS systems, e.g., systems that specifically target the edge [67, 149]. As interfaces are constantly evolving and new platforms are being introduced, a cross-platform benchmarking framework *needs to be extensible to support future FaaS platforms.*

R5 – Support for Drill-down Analysis:

An application-centric FaaS benchmark can help to evaluate the suitability of different sets and configurations of FaaS platforms for a specific application. What it can usually not provide are explanations for its finding, e.g., the different cold start management behavior of AWS Lambda and Apache OpenWhisk mentioned above [18]. To facilitate root cause analysis and help evaluators explain the patterns they see in the benchmark results, we argue that *an application-centric FaaS benchmarking framework should support drill-down analysis by logging fine-grained measurement results including all relevant metrics.*

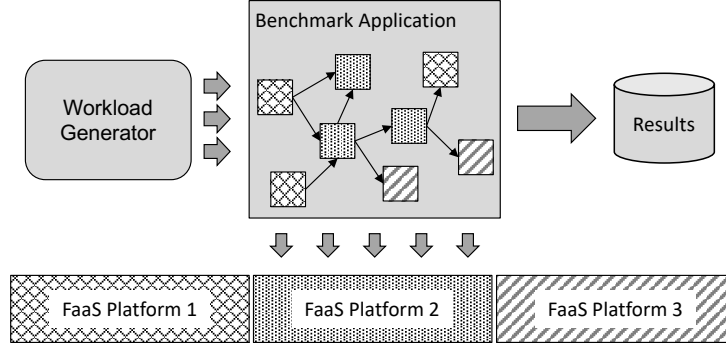


Figure 5.1: High-level overview of the BeFaaS architecture.

R6 – Minimum Required Configuration Overhead:

An application-centric FaaS benchmarking framework should be easy to use and provide reproducible results. This includes configuration, deployment, execution, as well as collection and analysis of results, e.g., using infrastructure automation. Hence, *a FaaS benchmarking framework should be designed to require as little manual effort as possible.*

5.1.2 Design**Architecture and Components:**

In BeFaaS, executing functions of a benchmark application is the workload that actually benchmarks the FaaS platform, i.e., executing a function creates stress on the SUT. Since functions do not “self-start” executing, we need an additional load generator that invokes the FaaS functions of our benchmark application. We show a high-level architecture overview in Figure 5.1.

For a benchmark run, BeFaaS requires three inputs: (i) the source code of the FaaS functions forming the benchmark application, (ii) a load profile for the load generator, and (iii) a deployment configuration that describes the environment configuration for each function and FaaS platform (the SUTs). We show the components of BeFaaS and their interaction in Figure 5.2.

Application code and deployment configuration are initially converted into deployment artifacts by the *Deployment Compiler*. The Deployment Compiler instruments and wraps each function’s code with BeFaaS library calls and injects vendor-specific instructions defined in deployment adapters to enable request tracing and fine-grained metrics. The resulting deployment artifacts are passed to the *Benchmark Manager*.

The Benchmark Manager orchestrates the experiment: First, it configures the *SUT* by deploying each function based on the information in the respective artifact. If there are external services, e.g., a database service, these can either be deployed by the Benchmark Manager as well or linked to the SUT using environment variables. In the second step, the Benchmark Manager initializes the *Load Generator* with the workload information described in a load profile. Then, the benchmark run is triggered and the Load Generator invokes the functions of the benchmark application, which log every request in detail, including timestamps, origin function, and called

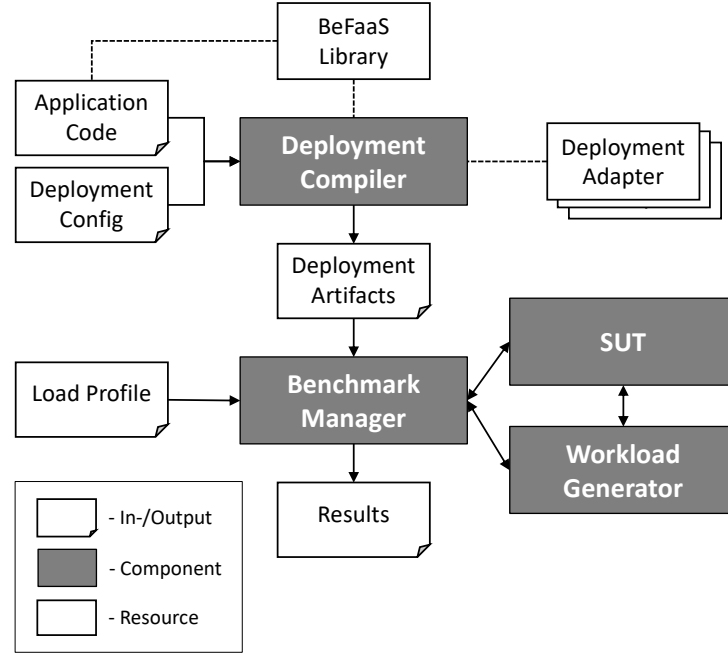


Figure 5.2: The Deployment Compiler transforms application code into individual deployment artifacts based on a deployment configuration. These are then deployed and invoked by the Load Generator to retrieve measurement results. Finally, the Benchmark Manager aggregates and reports fine-grained results.

functions (if applicable). Once the benchmark run is completed, the Benchmark Manager collects function logs, aggregates them, and destroys all provisioned resources.

Realistic Benchmarks:

To provide a relevant and realistic application-centric benchmark (**R1**), BeFaaS already comes with four built-in benchmarks which mimic and represent typical use cases for FaaS applications. These include a microservice-oriented web application to study request-response patterns, an IoT application scenario to evaluate hybrid edge-cloud setups, a smart factory application to measure event trigger performance, and a microservice application to study cold start behavior and elasticity capabilities (details are further explained in Section 5.1.3). Our application benchmarks are in line with the empirical findings of Shahrade et al. [173] regarding typical FaaS applications: All are composed of several functions that interact with each other to form function chains, use synchronous HTTP or asynchronous event triggers, and use external services such as a database system for persistence. The benchmark applications come with a default load profile that covers all relevant aspects as well as several further load profiles to emphasize selected stress situations, e.g., to provoke more cold starts. In combination, the benchmarks each represent complete FaaS applications: load balancing at the provider endpoint(s), interconnected calls of several functions, calls to external services such as database systems, and multiple load profiles which, e.g., provoke cold starts of functions.

The modular design of BeFaaS, however, also allows us to easily add further benchmark applications and load profiles or to adapt existing ones to the concrete needs of the developer (**R2**).

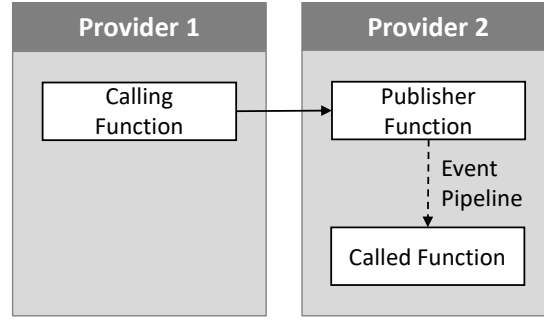


Figure 5.3: A publisher function forwards incoming events to the respective event pipeline to trigger the called function.

For adding a new benchmark, the respective application only needs to use the BeFaaS library (described in Section 5.1.3) for function calls and to have unique function names.

Benchmark Portability and Federated FaaS Deployments:

To support portability of benchmarks and federated deployments, BeFaaS relies on unique function names, individual deployment artifacts for every function, and a single endpoint for every deployed function (**R3**): With globally unique function names, the endpoints of the deployed functions are already known during the compilation phase. The Deployment Compiler maps these endpoints to the canonical function names (defined in the application) and compiles them into the source code. Moreover, the compiler also injects endpoints to external services such as database systems using environment variables which were set in the respective setup script or defined manually. To enable asynchronous function calls, the Deployment Compiler creates and assigns a topic-based event pipeline on the respective provider for each asynchronous function. To trigger this pipeline, requests are sent as events to a publisher function, which is deployed for every provider and forwards the request to the respective event topic (see Figure 5.3). In total, this decouples the ability of a function to call another function or a platform service from its deployment location and enables BeFaaS to support arbitrarily complex deployments: it is indeed possible to run every function on a different FaaS platform.

Each FaaS platform offers a different interface for life-cycle and configuration management of functions. As the smallest common interface, BeFaaS requires that each platform provides API-based access to (i) deploying functions, (ii) retrieving log entries from the standard logging interface, and (iii) removing functions. The Deployment Compiler wraps this functionality using an adapter mechanism and selects the appropriate instructions for the target platform specified in the deployment configuration. Additional FaaS platforms that fulfill this minimal interface can easily be added by implementing a corresponding adapter (**R4**).

Detailed Request Tracing:

To enable a detailed drill-down analysis of experiment results (**R5**), the Deployment Compiler injects and wraps code that collects detailed measurements during the benchmark run: The

compiler adds timestamping to determine start, end, and latency of calls to functions and external services.

Besides these timestamps, the compiler also injects code that generates context IDs and pair IDs to assign individual calls to their respective context later on. Here, a context ID is generated once for each function chain (with the first function call) and propagated to every subsequent call to other functions. To link the individual calls of a function chain, the compiler injects source code to create pair IDs of randomly generated keys that link caller and callee. Thus, it is possible to trace every single request through the benchmark application and to generate call trees for every context and function chain during post-experiment analysis.

Finally, for independently and reliably detecting cold starts, the Deployment Compiler also injects code that evaluates a local environment variable on the executor at the provider side. If this variable is not present, the function runs on a new executor (cold start), the variable is created, filled with a randomly generated key, and the cold start is logged.

All data that enable fine-grained results (timestamps, context IDs, pair IDs, and executor keys) are recorded on the console using the standard logging interface of the respective FaaS vendor. In initial experiments with AWS, GCP, and Azure, we verified that the cost of logging is at most in the microsecond range.

Automated Experiment Orchestration:

The BeFaaS framework requires only the application code, a deployment configuration, and a load profile to automatically perform the benchmark experiment (**R6**). First, all business logic, dependencies, and BeFaaS instrumentation logic are bundled into a single deployment artifact by the Deployment Compiler. Next, the Benchmark Manager orchestrates the experiment and provides a simple interface for starting the benchmark run, monitoring its process, and collecting fine-grained results for further analysis.

5.1.3 Implementation

Our open-source prototype implementation of BeFaaS² includes (i) the BeFaaS library, (ii) four deployment adapters, (iii) the Deployment Compiler, (iv) the Benchmark Manager, (v) four realistic benchmark applications, and (vi) several load profiles for the benchmark applications.

The BeFaaS library is written in JavaScript and handles calls to other functions depending on their canonical name, generates tracing IDs, and takes timestamps. BeFaaS deployment adapters are implemented using Terraform³ commands. Currently, BeFaaS thus supports three major cloud offerings (AWS Lambda, Google Cloud Functions, and Azure Functions) as well as the open-source system tinyFaaS [149], which supports the deployment of functions on private infrastructure, including edge or fog nodes. The Deployment Compiler is a shell script that uses several tools to build the deployment adapters for the respective platforms, parses and injects

²<https://github.com/Be-FaaS>

³<https://www.terraform.io/>

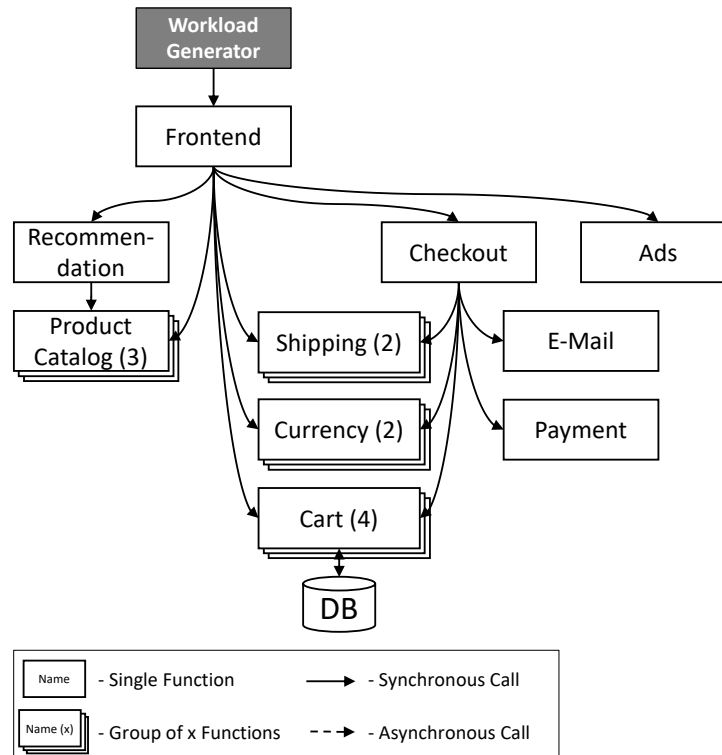


Figure 5.4: The e-commerce application implements a web shop in 17 functions. The *Frontend* serves as a single entry point and an external database is used to store state. We group some functions to increase legibility.

information from the Deployment Configuration, and generates the deployment artifacts from the application code. The Benchmark Manager uses Terraform to create the infrastructure based on these artifacts, collect the logs, and later remove provisioned resources. The implemented benchmark applications are written in JavaScript and include calls to external services such as a Redis⁴ instance. The Load Generator uses Artillery⁵ to call the benchmark application. It either executes a realistic default load profile that stresses all relevant aspects of the application or specific additional load profiles that emphasize stress situations, e.g., to provoke more cold starts. New load profiles can easily be added by specifying new Artillery load descriptions (YAML⁶ configuration files).

5.1.4 Benchmark Applications

The following four benchmark applications are already included in BeFaaS.

Web Shop (Microservices):

Our e-commerce benchmark implements a web shop as a FaaS application derived from Google's microservice demo application.⁷ Our corresponding benchmark implementation follows a typical

⁴<https://redis.io/>

⁵<https://artillery.io/>

⁶<https://yaml.org/>

⁷<https://github.com/GoogleCloudPlatform/microservices-demo>

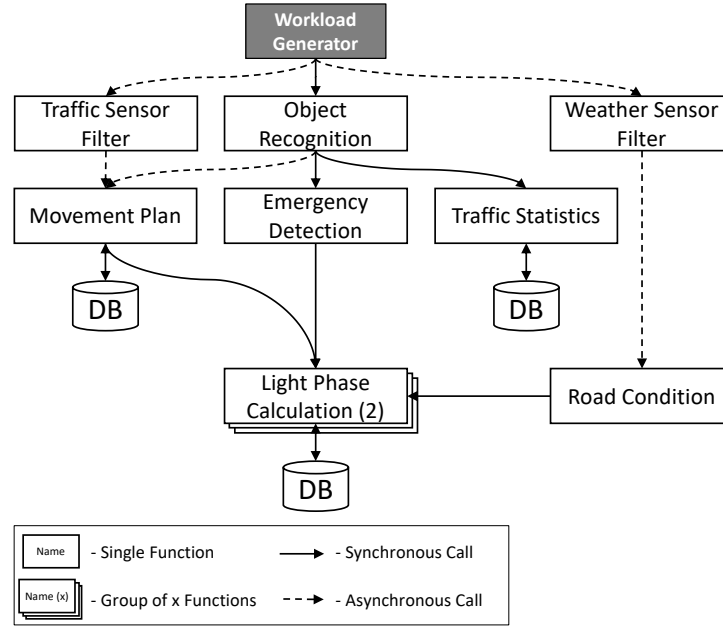


Figure 5.5: The IoT application implements a smart traffic light scenario in nine functions. The *Load Generator* emulates sensor data and sends them to three different entry points.

request-response invocation style, comprises 17 functions, and uses a Redis instance as an external service to persist state (see Figure 5.4). Besides functions that provide recommendations and advertising, customers can log-in, set their preferred currency, view products, fill a virtual shopping cart, check out orders, and finally observe order shipping. Each task is implemented in a separate function and all requests arrive at a single function, the frontend, which takes the customer calls and routes them to the respective backend functions. There are blocking synchronous calls to other functions as well as asynchronous call blocks that idle until all called functions return.

The default load profile simulates four different customer workflows and constant traffic for 15 minutes. Our e-commerce benchmark is particularly well suited for comparing request-response behavior and study request details of different cloud providers but can also be used to explore federated cloud deployments, e.g., for scenarios in which the application is running on multiple cloud platforms.

Smart City (Hybrid Edge-Cloud):

Although several IoT applications and use cases already exist in research (e.g., [8, 29, 70, 133, 154]), none of them could directly be used or adapted as a FaaS application. Thus, we designed our smart city benchmark application around typical IoT patterns and implemented a use case based on a smart traffic control scenario inspired by the *InTraSafEd5G* system [124, 126].

The benchmark application uses a mix of synchronous and asynchronous function calls and implements an IoT use case with a smart traffic light which adapts its light phase based on traffic sensors, a camera, and weather inputs (see Figure 5.5). The functions initially filter incoming data streams and perform object recognition on camera footage to create a movement

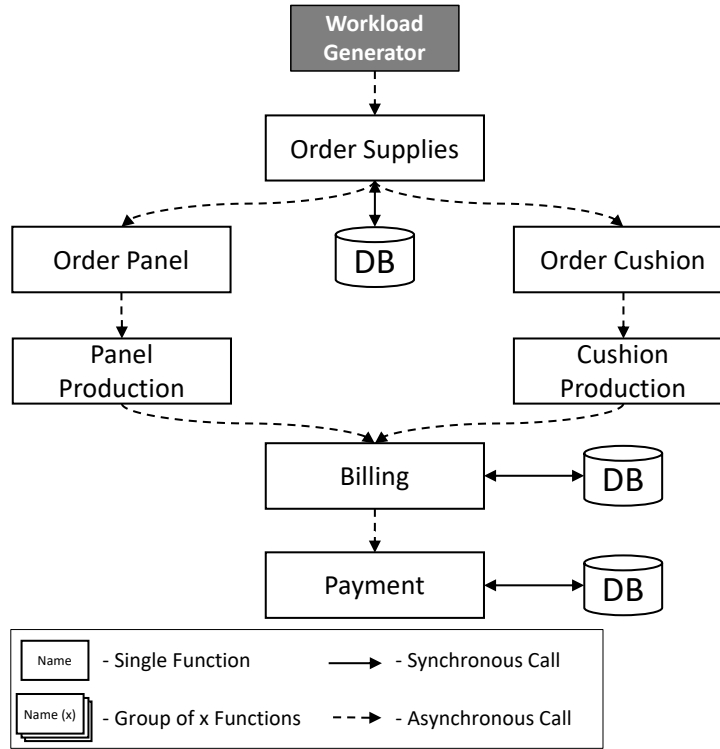


Figure 5.6: The Industry 4.0 application implements a smart factory in seven functions. An **Order Supplies** function serves as single entry point for different asynchronous event pipelines which can be distributed among several providers.

plan, detect ambulance/emergency cars, and maintain a traffic statistic. The regular light phase is then determined based on this movement plan, road conditions, and the current light phase. Emergency services can override the regular phase at any time by raising an emergency event that stops all other traffic.

The load profile for this application emulates sensor data and injects emergency events. The traffic sensor sends an update every two seconds to the Traffic Sensor Filter, the Object Recognition processes one image every two seconds, and the weather is updated every twenty seconds. Furthermore, the Load Generator also injects an emergency event every two minutes which lasts five seconds each. This default load profile runs for 15 minutes. As this use case will in practice typically have a very predictable and stable load profile, we did not implement alternative load profiles – benchmark users can, however, easily add them if needed.

The smart city benchmark is particularly well suited for comparing different deployments across cloud, edge, and fog.

Smart Factory (Event Trigger):

Our smart factory benchmark application implements asynchronous event-based pipelines. In our example use case, users order personalized couches consisting of panels and cushions (see Figure 5.6). First, an order function determines the number and individual sizes of panels and

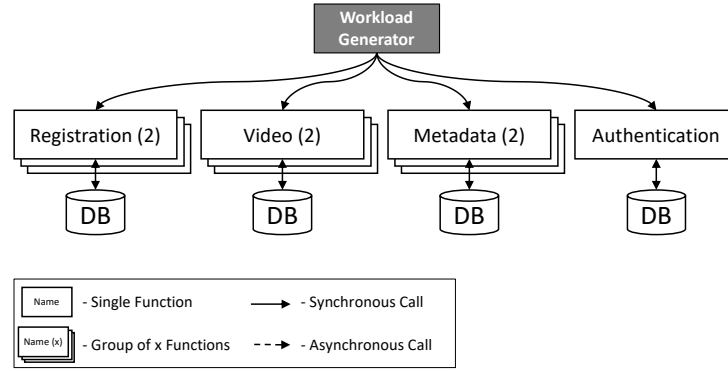


Figure 5.7: The streaming service comprises seven functions and a workload that triggers cold starts. After an Internet outage, there is a high load on functions dealing with authentication and metadata.

cushions which are then each ordered by sending an event to the respective order function. Both order functions, in turn, transform their order into a production event which is sent to the production function which mimics the production of the panel or cushion. After production, the functions emit an accounting event which is consumed by the billing function. Once all panels and cushions are produced, and all accounting events are processed, the payment function finally issues an invoice.

The default load profile orders a new couch every five seconds for 15 minutes. Each order, in turn, implies 8 to 18 order events, depending on the ordered couch model. This smart factory benchmark application is particularly well suited to study event pipelines but can also be used to analyze the interplay between different FaaS providers. For example, when collaborating with suppliers (panels and cushions in our case), they may also be deployed on another provider, thus, requiring cross-cloud interoperability.

Streaming Service (Cold Start Behavior):

An often stated advantage of FaaS applications is their elastic scalability. Thus, we include a streaming service benchmark application which triggers cold starts and can require automatic scaling capabilities. The Load Generator here mimics video streaming devices which register users, request video files, update meta information such as viewing progress, and handle backend authentication (see Figure 5.7). In case of a larger Internet outage, these devices are offline, but it is possible to continue watching already downloaded movies while the corresponding metadata is updated. As soon as network connectivity is restored and the streaming devices are back online, all clients reconnect and concurrently invoke functions, which triggers cold starts.

The default load profile for this benchmark is split into four phases. First, an initial set of users and streaming devices is registered. Once all initial data has been read, the normal load phase starts for 5 minutes in which 500 request flows add new videos, request videos, and update metadata. Third, the failure is simulated by pausing requests for 20 minutes. Finally, the

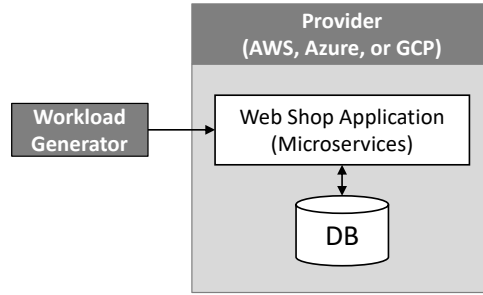


Figure 5.8: As part of the FaaS application, the database instance is deployed in the same region and on the same provider as the rest of the web shop.

benchmark triggers cold starts by suddenly sending 1,500 request flows distributed over another 5 minutes. Our streaming application benchmark is particularly well suited for comparing the cold start behavior and automatic scaling capabilities of different FaaS providers.

5.2 Evaluation

Our evaluation is split into two parts: First, we present the results of four experiments in which we use BeFaaS to stress different FaaS platforms (Sections 5.2.1 to 5.2.4). Second, in Section 5.2.5, we discuss to which degree BeFaaS fulfills our requirements from Section 5.1.1.

In all experiments, we deploy the Load Generator on a (vastly over-provisioned) virtual machine (2 vCPUs and 4 GB RAM) and let it execute the default load profile of the respective benchmark application against the SUT deployed in either `eu-west-1` for AWS, `westeurope` for Azure, or `eu-west-1` for GCP. As runtime for the benchmark applications, we run `node.js 18` on all SUT options and use 256MB of memory per function (SKU Y1 on Azure). Moreover, the Redis database system used by the SUTs also runs on an over-provisioned virtual machine (2 vCPUs and 4 GB RAM; `ta3.medium` at AWS, `Standard_B2S` in Azure, and `e2-medium` at GCP) at the respective provider site. This ensures that the database instance and Load Generator will not be a bottleneck during the experiments [25]. All experiment results reported here are from the period June to July 2023.

5.2.1 Comparing Major Cloud FaaS Providers in Single Provider Setups

In our first experiment, we deploy BeFaaS in single cloud provider setups in which all functions of the web shop application are deployed on a single provider (namely AWS, Azure, and GCP) and use the default load profile to compare them (see Figure 5.8). During each experiment, the Load Generator executes 18,000 workflows, which each consist of 1 to 9 requests, over a time span of 15 minutes.

Figure 5.9 shows the execution duration of four selected functions with varying degree of complexity which are called from the frontend function (visualized as box plots; boxes represent quartiles, whiskers show the minimum and maximum values without outliers beyond 1.5 times the interquartile range). For the four functions examined in more detail, the overall picture is

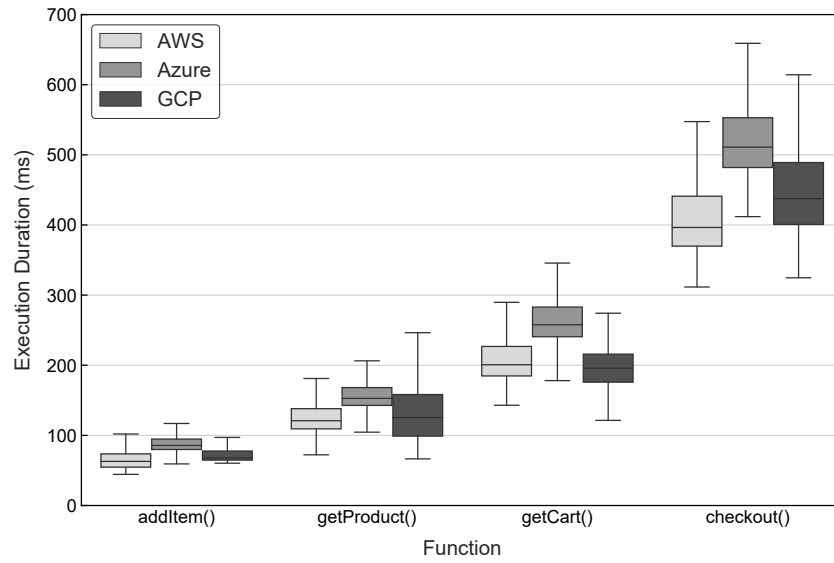


Figure 5.9: A detailed analysis of four functions called from the frontend shows that AWS provides the best performance and that the execution duration has the highest variance on GCP.

similar for all three providers: As expected, simpler functions that only read or write a single value have a lower execution duration than more complex ones such as the `getCart()` or `checkout()` function. In our experiment, Azure provided the slowest environment for this single run while GCP showed a higher variance for the `getProduct()` and `checkout()` function.

In a further fine-grained analysis, we investigate the distribution of computing, network transmission, and database query latency for a function sequence putting an item into the shopping cart. This includes synchronous and blocking calls to two functions and several database operations.

For this evaluation, we consider the (i) computation part as function execution duration without the duration of outgoing network calls, (ii) network latency as the duration of outgoing calls to other function without the execution duration of the called function itself, and (iii) query latency as the duration of calls to the external database. The detailed timestamp mechanisms of BeFaaS allow us to easily separate these times.

The results of this analysis are shown in Figure 5.10. In this specific but typical interaction in which FaaS functions are the glue code to interact with external services, it is noticeable that for all providers time is mostly spent on network transmission followed by the database round-trip time while the actual computing time is below 1ms for all providers. Furthermore, database access takes longer for GCP than for the other providers.

5.2.2 Evaluating Hybrid Edge-Cloud Setups Using the Smart City Application

In this experiment, we compare an edge-focused and a hybrid edge-cloud setup. For the mixed setup, we split the smart city application into a cloud part, which is deployed on either AWS, Azure, or GCP, and an edge part, which is deployed on a local Raspberry Pi in Berlin, Germany running the tinyFaaS platform (see Figure 5.11). For the edge-focused setup, we deploy all

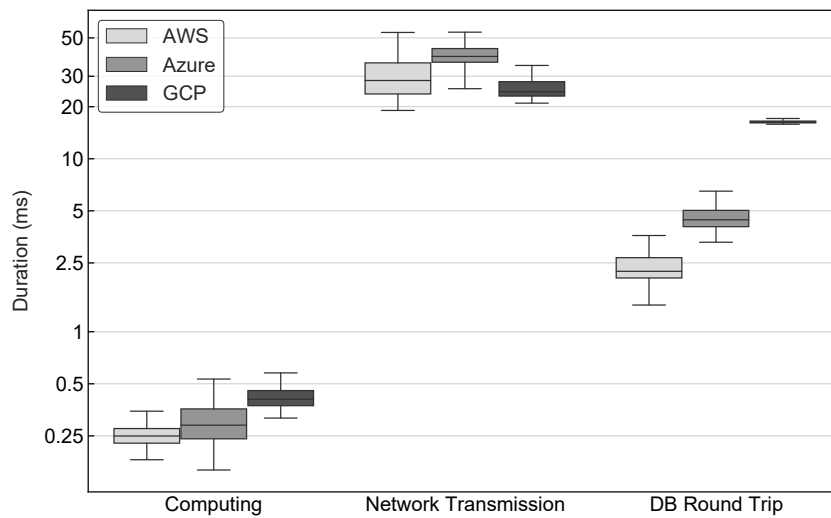


Figure 5.10: A drill-down analysis of a function sequence reveals that the network transmission time is the most relevant driver of execution time on all providers.

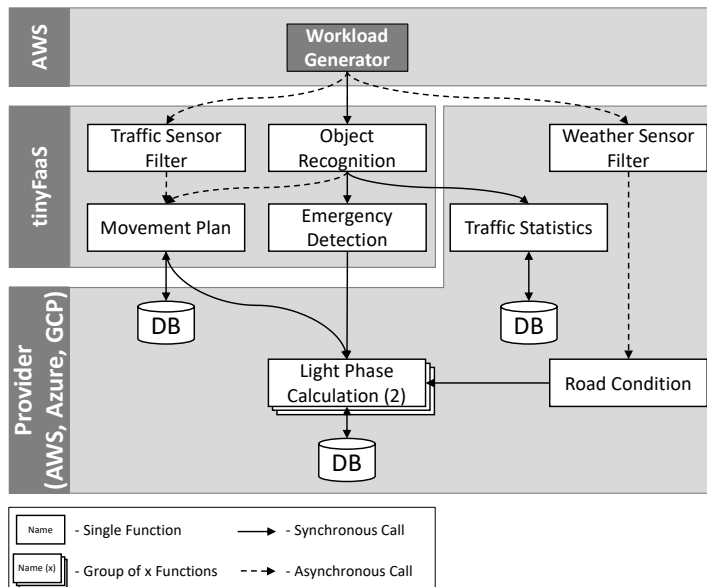


Figure 5.11: Functions related to the traffic light are deployed on a Raspberry Pi at the edge location while others run on public cloud providers.

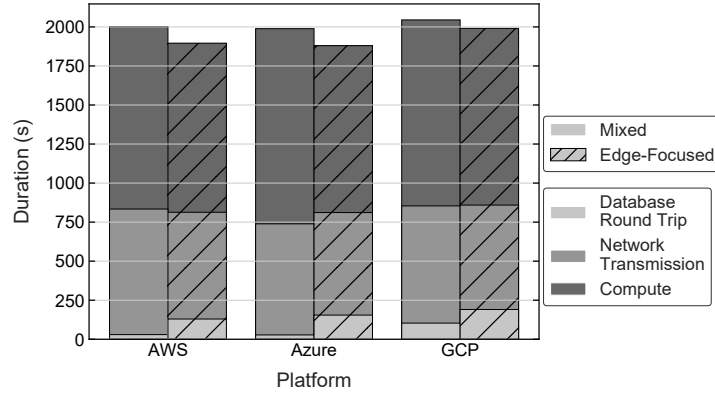


Figure 5.12: The cloud database backend increases the database request latency for the edge setup for all providers. In total, however, both compute and network duration are reduced, and the total execution duration is lower for all edge setups.

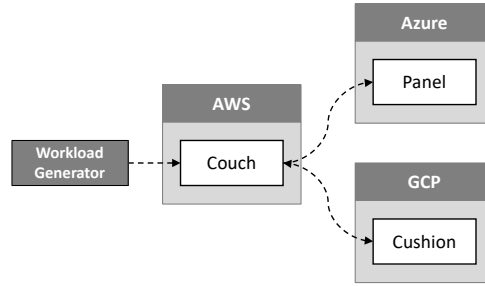


Figure 5.13: Each provider hosts a part of the smart factory application.

functions of the smart factory application on tinyFaaS and only use a cloud-located database at the respective provider. During the experiment, the Load Generator runs the smart city scenario for 15 minutes by triggering the traffic sensor and object recognition every two seconds and the weather sensor every 10 seconds for both evaluated setups.

Similar to our first experiment, we analyze the computing, network transmission, and database round trip times for both setups and all providers (see Figure 5.12). For all providers, the default workload finishes slightly faster in the edge-focused setup. Here, the network transmission times are reduced as all functions run on the same edge device, the compute duration is shorter compared to the mixed cloud-edge setup, but the database round trip time increases as every database access triggers a cloud request. This last addition, however, does not outweigh the other two improvements for the given default load.

5.2.3 Analyzing the Event Pipeline Interplay Within and Across FaaS Providers.

This experiment intends to investigate both how event pipelines perform within a provider, but also how well the interplay of cross-provider pipelines works. Thus, we split the event-based smart factory application into three parts: 1. **Couch** (running supplies, billing, and payment), 2. **Panel** (running panel order and production), and 3. **Cushion** (running cushion order and production). Each part is deployed on AWS, Azure, or GCP (see Figure 5.13). The load for this

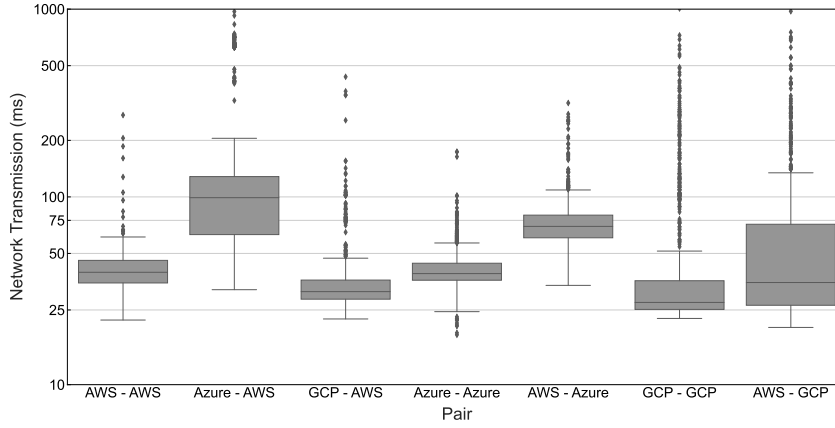


Figure 5.14: The network latency to publish an event usually ranges between $25ms$ and $100ms$.

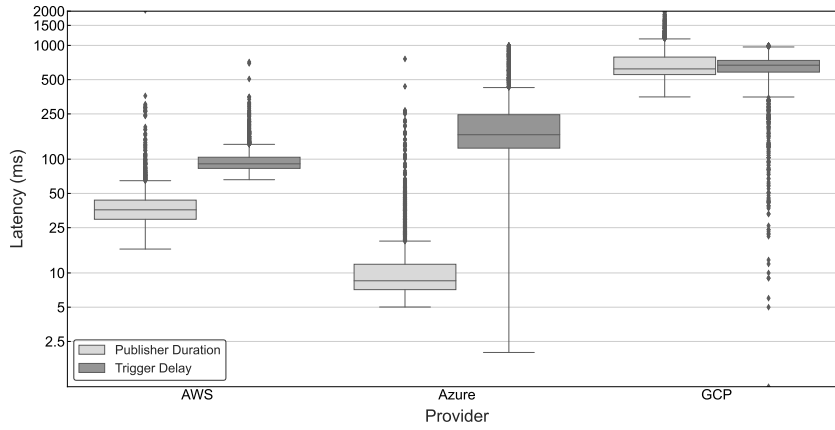


Figure 5.15: The execution time of the publisher function ranges from about $10ms$ on Azure to about $800ms$ on GCP. The trigger delay between publisher start and function start varies from $100ms$ on AWS to about $800ms$ on GCP.

experiment consists of 180 couch orders over a time span of $15min$, which triggers thousands of function invocations.

First, we analyze the time it takes to publish an event at the respective provider endpoint (see Figure 5.14). Again, we measure the outgoing call from the calling function and subtract the execution duration from the execution time of the called publisher function running on the destination provider. Depending on the origin and destination provider this usually takes between $25ms$ and $100ms$, except for the Azure-AWS pair, which may take up to $200ms$. Moreover, there are outlier values of more than one second in five pairs.

Second, we also investigate the total execution time of the publisher function running on the destination provider and the trigger delay between the start of the publisher function and the start of the triggered function (see Figure 5.15). Here, Azure and AWS show fast publishing functions which usually finish within $100ms$ while for GCP it usually takes between $500ms$ and $1,000ms$ to run the publisher function. For the trigger delay, AWS triggers the respective function fastest with about $100ms$, followed by Azure with 75% of values below $250ms$, and GCP with 75% of values above $500ms$. Furthermore, it is noticeable that outliers in the other direction

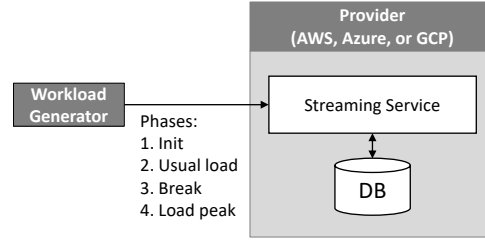


Figure 5.16: Similar to experiment 1, we deploy BeFaaS in single cloud provider setups in which all functions of the steaming service application are deployed on a single provider (namely AWS, Azure, and GCP).

are possible, i.e., an event sometimes triggers the function execution immediately within $10ms$ for Azure and GCP.

5.2.4 Studying the Cold Start Behavior of Different Providers.

In our last experiment, we study the cold start behavior of all cloud providers and deploy the streaming service application once on each provider (see Figure 5.16). Running the default workload for this use case, we execute 500 requests within the first $5min$ load phase, which is then followed by a $20min$ break, and finally execute 1,500 requests for another 5 minutes.

Figure 5.17 shows the function execution time of all calls of the streaming application in the first 30 seconds of the *Load Peak* phase. In this phase, AWS and GCP both log cold starts and show larger latency values in the first seconds of the experiment. While for AWS the values do not stabilize for the first 30 seconds, the GCP execution times are stable after 10 seconds. Azure does not report any cold starts and shows a stable execution duration for all functions. However, we possibly also missed the respective log entry due to the log framework limitations: The Azure logging framework only writes 250 log lines per second at maximum, further lines are discarded. Thus, only 9,143 out of about 15,000 values across the whole experiment time are available for this analysis.

5.2.5 Discussion of Requirements

In Section 5.1.1, we had identified six requirements for application-centric FaaS benchmarking frameworks. We now discuss to which degree BeFaaS fulfills these requirements.

BeFaaS already comes with four standard benchmark applications that cover many representative FaaS application scenarios, namely a standard web application, a hybrid edge-cloud scenario, an event-based smart factory application, and a microservice-oriented streaming service. Moreover, BeFaaS can be easily extended by implementing more FaaS application scenarios using the BeFaaS library and further workload profiles. We, hence, believe that BeFaaS fulfills the requirements **R1** (*Realistic Benchmark Application*) and **R2** (*Extensibility for New Workloads*).

In BeFaaS, benchmark users can define arbitrarily complex deployment mappings of functions to target FaaS platforms including federated multi-cloud setups or mixed cloud/edge/fog de-

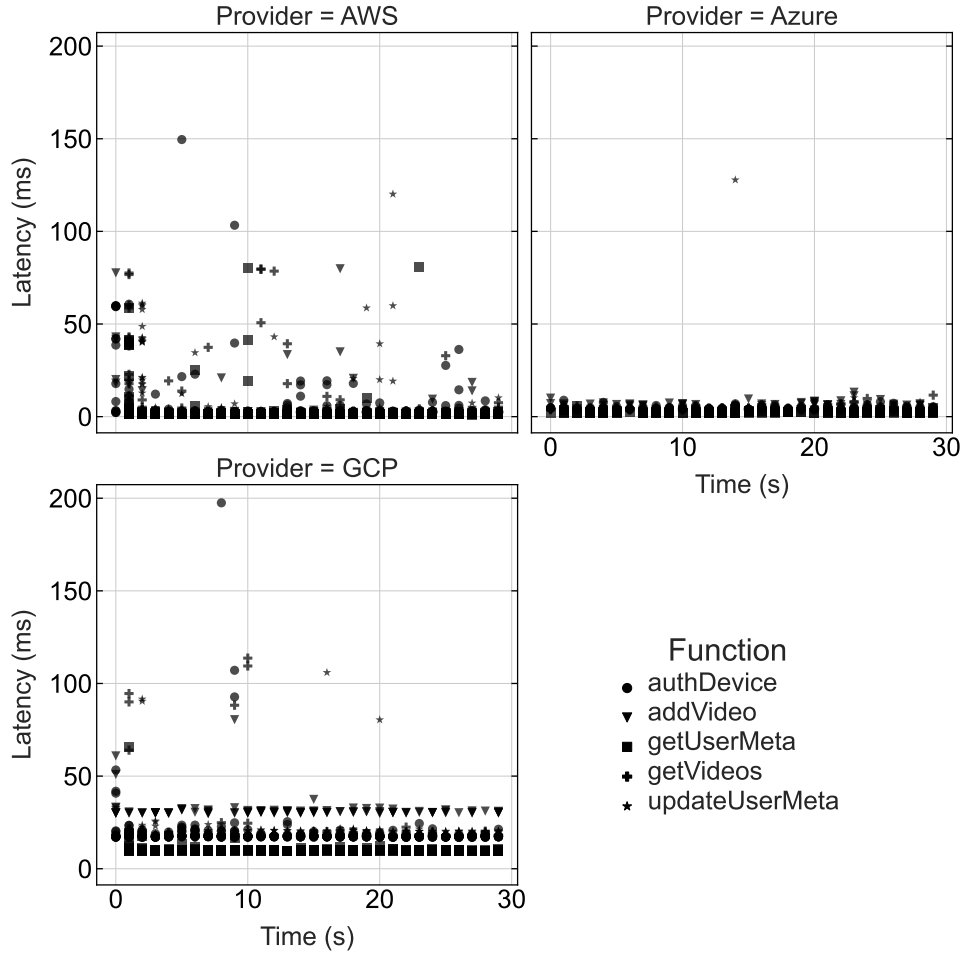


Figure 5.17: In the first 30 seconds of the load peak phase, AWS and GCP show larger latency values due to cold starts. Azure is presumably not affected by cold starts for this workload.

ployments. In fact, each function could run on a different platform. To achieve this, BeFaaS transforms the benchmark application into deployment artifacts fitted to the target platform. Adding another target platform is also straightforward and only requires the benchmark user to implement an adapter component for the respective FaaS platform or to copy and adapt an existing adapter component. Based on this, we argue that BeFaaS fulfills the requirements **R3** (*Support for Modern Deployments*) and **R4** (*Extensibility for New Platforms*).

At runtime, BeFaaS collects fine-grained measurements and traces individual requests similar to what Dapper [174] does for microservice applications. This offers the necessary information basis for drill-down analysis. Beyond this, BeFaaS also offers visualization capabilities for select standard measurements to further support analysis needs. Overall, we hence conclude that BeFaaS addresses requirement **R5** (*Support for Drill-down Analysis*).

Finally, we believe that BeFaaS is easy to use due to its experiment automation features and requires only very few configuration files (requirement **R6** – *Minimum Required Configuration Overhead*). Nevertheless, this is a highly subjective matter that depends on the respective individual.

5.2.6 Findings

Our evaluation studies four different typical FaaS use cases on Amazon Web Services (AWS), Google Cloud Platform (GCP), Microsoft Azure (Azure), and tinyFaaS [149]. First, while there are no major differences between the major cloud providers in terms of execution time, it is noticeable that the variance in execution time differs depending on the provider. A closer look at the different elements of the execution time shows that network transmission is a major contributor to overall response latency while the pure computing time of functions is almost negligible. Second, an edge-focused deployment can outperform a hybrid edge-cloud setup, although the database round trip latency is larger. Third, sending events to event pipelines can be done within $100ms$ for all FaaS providers. The delay between the published event and the start of the respective function is about $100ms$ for AWS, $250ms$ for Azure, and $800ms$ for GCP. Fourth, Azure is not affected by cold starts for our workload while AWS and GCP show larger latency values during a load peak phase. In total, our experiments show that BeFaaS is able to analyze FaaS providers in detail and meets all the requirements of a modern application-centric FaaS benchmarking framework.

5.3 Discussion

BeFaaS is a powerful application-centric FaaS benchmarking framework. There are, however, also some points to consider and limits when using BeFaaS.

Tracing Overhead:

BeFaaS supports a detailed tracing of requests by injecting a small token in each call. On the one hand, this supports the clear mapping of different calls to function chains, yet on the other hand, it also causes an additional network overhead. This token, however, is of constant size (depending on the length of the respective function name), so the overhead can be easily determined and considered in results analysis. Furthermore, this will only matter if the goal of the benchmark is to find the optimal deployment for an existing application which is then instrumented to be used as a BeFaaS benchmark. For any of our standard benchmarks, it simply increases the benchmark workload stress slightly.

Measuring External Services:

Currently, BeFaaS handles external services and components as a black box and only measures end-to-end latency of such service calls. In future work, however, we plan to implement a small BeFaaS sidecar proxy that can be deployed on external service instances to forward calls to the respective service and to inject the BeFaaS tracing token there as well.

Fairness with External Dependencies:

The included benchmark uses an external database system to persist state but further benchmarks and use cases may also require external services such as pub/sub message brokers or

web APIs. Although the modular design of BeFaaS supports this, there are also some pitfalls in terms of fairness and comparability: In our experiments, we deployed the database instance with the same provider and in the same region as our functions to minimize latency between functions and the database. In this setup, a function calling the external service and awaiting the response will not idle for a long time and the execution environment at the provider side will soon be available again for the next request. On the other hand, a function calling an external service in another region with larger latency will block the environment and (may) cause a cold start for the next incoming request. Thus, when using external services, these should be located and deployed with similar latency for all alternatives. Moreover, as cloud environments at least appear to be infinitely scalable, it has to be assured that the external service does not become a performance bottleneck during the experiment. Otherwise, the benchmark would benchmark the compute resources of the external service instead of the performance of the FaaS platforms.

Provider-specific Features:

Competing FaaS vendors are constantly developing new and exclusive features that simplify development and deployment for customers. These features, however, can also affect the portability of the BeFaaS framework if a (future) benchmark uses exclusive features that are not available at all vendors. Thus, we strongly recommend not to use exclusive features of individual providers when developing new BeFaaS benchmarks. BeFaaS can, however, help to determine the impact of new features within a provider or across multiple providers by adjusting and configuring the respective deployment adapter.

Time Synchronization:

The drill-down analysis features of BeFaaS require approximately synchronized clocks. Although this will usually be provided by the provider with sufficient accuracy, a user should assert this before running experiments as this will affect the reliability of tracing insights. Nevertheless, such detailed insights may often not be needed and the tracing of BeFaaS also offers a mechanism to partially mitigate this: If the call follows a request-response pattern, BeFaaS measures the total round trip time at the calling function and knows the computing duration at the called function. Thus, it is possible to approximate the network transmission latency under the assumptions that both directions took comparably long. This is even possible for event-based calls that do not return a message to the sender, as calling functions submit the trigger events to the respective publisher function which runs on the same provider as the called function. In our experience, though, this is neither a problem in the cloud nor for self-hosted FaaS platforms, where the user has direct control over clock synchronization.

5.4 Summary

FaaS platforms are a popular cloud compute paradigm and have also been proposed for edge environments. For comparing and choosing different FaaS platforms in terms of performance,

developers usually rely on benchmarking. Existing FaaS benchmarks, however, focus on specific aspects – an application-centric FaaS benchmarking framework is still missing.

In this chapter, we presented BeFaaS, an extensible framework for executing application-centric benchmarks against FaaS platforms which comes with four realistic FaaS benchmark applications. BeFaaS is the first benchmarking framework with out of the box support for federated cloud setups which allows us to also evaluate complex configurations in which an application is distributed over multiple FaaS platforms running on a mixture of cloud, edge, and fog nodes. Beyond this, BeFaaS is focused on ease-of-use through automation and collects fine-grained measurements which can be used for a detailed post-experiment drill-down analysis, e.g., to identify cold starts or other request-level effects; it can easily be extended with additional benchmarks or adapters for further FaaS platforms.

With BeFaaS, we provide developers with the necessary tool to explore, compare, and analyze FaaS platforms for their suitability for application scenarios. We also offer researchers the ability to study the performance effects of different FaaS deployment options across cloud, edge, and fog through experiments. Finally, FaaS platform developers can use BeFaaS in their CI/CD pipeline to compare their own platform to previous versions of it as well as to their competitors.

Related Work

In the following, we discuss related work to this thesis. First, Section 6.1 focuses on studies related to benchmarking in cloud environments and Section 6.2 on benchmarking in CI/CD pipelines. Next, Section 6.3 discusses approaches for detecting and quantifying performance changes. Sections 6.4 to 6.6 discuss related studies on benchmarking microservices, optimizing microbenchmark suites, and benchmarking FaaS platforms. Finally, Section 6.7 summarizes this chapter. This chapter contains (partially adapted) material from our published research articles [72–78].

6.1 Benchmarking in Cloud Environments

Benchmarking is a well-established method to quantify and verify QoS of hardware or software systems [25, 83]. There are many benchmarks for different kinds of SUT, especially for database and storage systems, e.g., [19, 50, 107, 137, 148], but also for virtual machines, e.g., [28, 170], web APIs [23, 24], or cloud-based queuing systems, e.g., [108]. Regardless of the type of SUT and kind of benchmark, all benchmarks should aim for design goals such as relevance, portability, or repeatability to provide reliable results [20, 25, 60, 92, 101].

A key requirement of benchmarking, the repeatability, is difficult to realize in variable cloud environments due to the many random factors that affect the benchmark [1, 15, 27, 32, 33, 50, 55, 60, 96, 109, 113, 114, 117, 118, 130, 153, 163, 175, 181]. These studies and approaches are relevant for the application of our presented contributions. Our studies on optimized microbenchmark suites and our experiments using our application-centric FaaS benchmarking framework continue this research and show the performance variability in cloud environments. If the performance variance in the benchmark environment is known and or can be reduced to a minimum, application engineers are able to decide better and on a sound basis when and to what extent each benchmark type should be executed.

Despite the variability, to ensure that measurements are as accurate as possible and to derive correct conclusions, benchmarking experiments must be repeated several times to ensure their reproducibility. One way to minimize the effects of this variability and the number of experiment repetitions is to benchmark multiple SUTs concurrently on the same VMs [32, 33]. This is, however, not always easy to implement or even possible. In more complex systems that include several components distributed on different instances, for example, it would be necessary to ensure that the individual components are also exposed to the same load concurrently to provide a valid application benchmark.

6.2 Continuous Benchmarking

The idea of using performance testing or benchmarking in CI/CD pipelines, also in cloud environments, has already been addressed in several related papers. Researchers, argue that application performance after new commits should be tracked and propose an automatic approach based on call trees [134], manually inject performance issues in three study objects to verify their automatic performance regression detection approach [61, 62], include microbenchmarks in a CI/CD pipeline [183], and propose tools supporting benchmarking in CI/CD pipelines [81, 82, 100, 175]. Moreover, several studies also conduct performance case studies using a dedicated benchmarking step and real software projects, e.g., [44, 45, 74, 95]. Our contributions to the optimization of microbenchmark suites continues this research by studying, through extensive experimentation, to what degree different benchmarking approaches can detect performance changes of open-source systems as part of a CI/CD pipeline.

One major challenge in using benchmarking in the CI/CD process is the long duration of the benchmarks, which does not give developers a fast performance feedback. Thus, there are various approaches to reduce the benchmark duration or make them more effective, e.g., by stopping the benchmark run when the system reaches a stable performance state [4, 5], using a statistical approach based on kernel density estimation to stop once a benchmark is unlikely to produce a different result with more repetitions [84], or using the functional tests of software projects and extracting classifiers for predicting tests that will reveal performance changes [37]. Such approaches can be combined with our contributions and optimizations for workload generation or microbenchmark suites under certain conditions. The main aspect here is that relevant functions might never be called if the benchmark run is terminated early. Another large body of research is performance regression testing, which utilizes microbenchmarks between two commits to decide whether and what to test for performance. Huang et al. [90] and Sandoval Alcocer et al. [161, 162] use models to assess whether a code commit introduces a regression and select versions that should be tested for performance. Our approach for optimizing microbenchmark suites shortens benchmark duration as well and can be further optimized by combining related approaches.

Continuous benchmarking is a powerful mechanism for evaluating QoS of a new system version in a production-like environment. As such, it relies on existing benchmarking approaches [19, 20, 23, 27, 28, 40, 50]. An alternative but also complementary approach to continuous benchmarking are live testing techniques such as canary releases [91] or dark launches [58]. In contrast to continuous benchmarking, live testing is characterized by the fact that a new version (of a software artifact) is directly deployed into the production environment in parallel with the older version.

For canary releases [91], this new version is initially rolled out for a very small subset of users or machines and developers monitor its behavior in production. If there are errors or QoS issues in the new version, the impact only affects a few users, and the version is reverted or shut down. Otherwise, more and more users are added to the set of test users until the new version has been completely rolled out. While canary releases aim to only affect a small subset of users in case of failures, dark (or shadow) launches [58, 178] eliminate potentially unsatisfied users

completely by deploying a new version in the production environment without serving real user traffic – so-called shadow instances. This way, no user is confronted with the new version and its potential issues.

Live testing techniques can be used to detect performance and other QoS issues in production. However, testing new versions in a production environment might be problematic for several reasons: First, a production system is usually in a normal state with usual load and regular traffic. Thus, a new version is never evaluated in production under extreme conditions or for rare corner cases. Second, a roll-out of several new versions of multiple software artifacts is administratively complex and error-prone, though tools like BiFrost [166] try to overcome these problems. Third, theoretical setups and architectures, including new versions, are hard to evaluate with live testing techniques. Finally, live testing does not necessarily create the right data to identify QoS degradation in the system release as varying workloads depending on user traffic will lead to varying observable QoS behavior. All this can be done with continuous benchmarking, e.g., by creating benchmark setups for extreme load peak situations or rare corner cases. As benchmarking, however, can never be identical to a production load, we propose to combine the strengths of both approaches and, that is, to use both live testing and continuous benchmarking in parallel.

Similar to live testing, monitoring allows for continuously observing an application in production and acting immediately in case of errors or detecting the root cause of performance changes, e.g., [54, 122, 164, 182, 186–188]. These techniques can be combined very well with (continuous) benchmarking, for example, by monitoring the application during the benchmark run and pinpointing the root cause if an issue occurs.

6.3 Detecting and Quantifying Performance Changes

There are several studies that aim to identify (the root cause of) performance regressions [45, 62, 74, 130, 141, 183]. In our case study using optimized microbenchmark suites, we apply a dynamically adapted performance detection threshold that adjusts along with the analyzed code changes to the respective micro or application benchmark instability. Besides basic threshold metrics [74, 136], there are more complex techniques for detecting and quantifying performance changes. For example, by using the performance signatures of past experiment runs and determining confidence measures [61] or considering noise in the performance evaluation and clustering the time series experiment data to identify performance change points [45, 132]. Moreover, even though the Iter8 framework is designed for live testing, the proposed decision Bayesian learning-based algorithms can also be adapted to decide which version performs better [179]. Each of these approaches could be used as alternatives for detecting performance changes and might, e.g., reduce the number of false alarms. On the other hand, however, each of these approaches also increases the complexity and implementation effort of the analysis. Finally, there are approaches that focus on automatically identifying the respective root causes of performance changes [85, 122, 141, 188].

6.4 Benchmarking Microservices

There is, to the best of our knowledge, currently no generic approach for benchmarking microservices. We believe that this is largely due to the fact that microservices do not come with the common interface typical of other system domains such as Create Read Update Delete (CRUD) interfaces for data management. Without such a common interface, it becomes quite hard to implement a benchmark that complies with standard benchmark requirements – especially portability [20, 25, 60, 92, 106]. Recent work in the microservice benchmarking domain presents general benchmark requirements and evaluations for microservices [3, 65, 83], studies the instability of microservices in cloud environments [55], focuses on the automation of performance tests for microservices [48, 86–88], or implements a benchmark suite with six different microservice applications [66].

Nevertheless, there are some approaches and tools which can ease microservice benchmarking beyond building a complete benchmark from scratch: Load generators such as Artillery IO¹ or LoadUI² can run a defined and service-specific workload against a microservice. By manually defining scenarios that represent typical interactions, a service-specific workload can be created with parameter settings that include the number of requests or the distribution of scenarios. While it is possible to import service description files and external data items as “workload”, this is always specific to a particular microservice and its respective version, i.e., there is no portability. Besides this, there is one approach that also generates requests based on (regularly updated) service descriptions but focuses on web services and manually defines each test specification [103]. With our workload generation approach, on the other hand, arbitrary REST microservices can be benchmarked as long as the service supports the respective interaction patterns and the pattern definitions can be reused for benchmarking other microservices. It is also possible to analyze OpenAPI specifications and automatically generate functional tests to detect bugs and security vulnerabilities, thus focusing on functional requirements [10]. The proposed algorithm resolves dependencies by analyzing actual service responses and can be used to identify dependencies between services automatically, which can ease our binding definition step.

Zheng et al. [195] also use interaction patterns composed of basic operations (create, get, delete) to benchmark object storage services. Their approach relies on the standard Cloud Data Management Interface (CDMI) and thus does not need to deal with interface heterogeneity. Apart from these, there are several systems that could serve as load generators. Benchmarking systems such as YCSB [40] or NDBench [147] can be used to create synthetic workloads against a CRUD endpoint. While these tools are very powerful load generators – particularly when considering the broad range of configuration options – they completely disregard the mapping from the generic CRUD to a specific microservice. Although creating such a mapping will be possible for a large percentage of microservices, actually programming the mapping still remains a manual effort that needs to be repeated for every microservice and version that shall be benchmarked.

¹<https://artillery.io/>

²<https://www.soapui.org/professional/loadui-pro.html>

Furthermore, we believe that benchmarking interactions with microservices should preferably be based on sequences of operations instead of isolated operations to obtain more realistic results (systems such as YCSB+T [49] or BenchFoundry [19] are probably a better match).

Besides workload generation and invocation of REST endpoints, our approach generates synthetic data for the workload. For data generation, we rely on JSON schema and the `faker.js`³ library. Alternative approaches are more powerful options for data generation and also support parallel generation, e.g., [153]. Such parallelization could improve our prototype in which generating the workload trace prior to distributing it onto the worker nodes can be rather slow. Nevertheless, we do not see parallelization as a critical feature since the generated workload can be persisted and reused instead of being generated from scratch for every benchmark run.

Finally, an alternative for clients of a microservice can be to rely on SLAs while monitoring violations, e.g., [104, 123]. However, this approach only shifts the responsibility for ensuring microservice performance to another organizational entity and does not actually solve the challenge of detecting performance changes of microservices early on and applying continuous benchmarking.

6.5 Optimizing Microbenchmarks

In this thesis, we propose and follow an optimization strategy that combines application and microbenchmarks to detect redundancies and optimize microbenchmark suites. Similar to us, researchers also apply a mutation-testing-inspired technique to dynamically assess redundant benchmarks and detect redundancies between microbenchmarks of the same suite [113]. Others decide based on source code indicators and information from prior benchmark runs which microbenchmarks to execute on every commit [6, 144]. These approaches, however, treat every code section equally and do not favor practically relevant code, i.e., functions that are actually used in production. Furthermore, there is an approach that studies the usability of functional unit tests for performance testing and builds a machine learning model to classify whether a unit test lends itself to performance testing [51]. Our redundancy removal approach could augment this approach by filtering out unit tests (for performance) that lie on the hot path of an application benchmark.

Our optimization strategy can be combined or replaced with other microbenchmark prioritization strategies to reduce the number of false alarms (e.g., [112, 135]) or to further shorten the execution duration by stopping microbenchmarks once the results are stable and or do not show significant performance changes (e.g., [4, 5, 84, 115]). One might also execute the microbenchmarks in parallel on cloud infrastructure to further shorten the benchmark duration. Recent work studied how and to what degree such an unreliable environment can be used [33, 114, 156].

Synthesizing microbenchmarks could be a way to increase coverage of important parts of an application. These could, for instance, be identified by an application benchmark. SpeedGun

³<https://fakerjs.dev/>

generates microbenchmarks for concurrent classes to expose concurrency-related performance bugs [152]; and AutoJMH randomly generates microbenchmark workloads based on forward slicing and control flow graphs [158]. Both approaches are highly related to our microbenchmark suite optimization strategy as they propose solutions for not yet existing benchmarks. However, both require as input a class or a segment that should be performance tested. Our recommendation algorithm could provide this input.

Finally, several studies empirically analyze how microbenchmarks – sometimes also referred to as performance unit tests – are used in open-source Java projects and found that adoption is still limited [117, 177]. Others focused on creating performance awareness through documentation [89] and removing the need for statistical knowledge through simple hypothesis-style, logical annotations [30, 31]. Moreover, other studies characterize code changes that introduce performance regressions and show that microbenchmarks are sensitive to performance changes [36], or study bad practices and anti-patterns in microbenchmark implementations [46]. All these studies are complementary to our studies on optimized microbenchmark suites as they focus on different aspects of microbenchmarking that are neither related to time reduction nor recommending functions as benchmark targets. Nevertheless, they contribute valuable insights for the further optimization of microbenchmark suites.

6.6 Benchmarking Function as a Service Platforms

Existing research on benchmarking of FaaS environments has so far mostly focused on benchmarks of single functions. Application-centric benchmarks that consider the overall performance of multiple functions, the interaction with external services, and the effects of different application load profiles are mostly still missing. Beyond FaaS, there are a number of application-centric benchmarking frameworks in other domains, e.g., for database and storage systems [19, 50] or for virtual machines [28]. However, these cannot be easily adapted to FaaS platforms.

Existing work on benchmarking of FaaS platforms usually focuses on the execution of small, isolated benchmarks that deploy and call a single function, e.g., a matrix multiplication or a random number generator. These functions are often designed for a specific purpose, e.g., stressing the CPU or evaluating disk-intensive workloads [11, 59, 116, 128, 129, 131, 172, 184, 189]. Besides function scaling, cold start latency, containerization overhead, and instance lifetimes, the studies also evaluate metrics such as CPU utilization, network throughput, and costs. Since the publication of the initial version of BeFaaS [77], recent research work has focused on benchmarking function triggers [167], studying tail latency [180], implications of used programming languages [43], hardware influence factors [42], concurrent function executions [12], take a closer look at the cost perspective [151], performance variations over time [171], fine-grained tracing of requests [168, 169], and general FaaS characteristics [56]. Almost all experiments, however, focus on a single isolated aspect and do not create a holistic comparability of platform performance for FaaS application developers.

Several studies also consider more complex applications and focus on specific FaaS related features, e.g., by deploying image processing pipelines [105], analyzing chained functions, or deploying real-world applications on serverless platforms [189]. While the authors of these studies also use application-centric workloads for experiments, their goal was not to propose a comprehensive framework for the execution of application-centric FaaS benchmarks. Furthermore, there are several studies and frameworks that share some of the features and goals of BeFaaS: PanOpticon [176] uses a deployment, workload, and metrics module to evaluate chained functions and a simple chat server on two different FaaS vendors. Although PanOpticon has similar goals as BeFaaS, it neither supports detailed drill-down analysis nor federated multi-provider setups. Van Eyk et al. develop a high-level architecture and state requirements for serverless benchmarking [57]. FaaSdom shares our motivation for a full application deployment [127]. It supports multiple platforms, several languages (e.g., Node.js, Python, Go), and an automatic deployment of performance tests via a web frontend. SeBS is a FaaS benchmarking framework that highlights the cost efficiency of executions and, similar to FaaSdom, also only considers single-function applications [41].

Besides a library that supports multi-cloud setups [194], to the best of our knowledge, BeFaaS is still the only FaaS benchmarking framework for evaluating federated cross-provider setups which can also be used to trace requests in the edge to cloud continuum.

6.7 Summary

Benchmarking microservice platforms and applications in cloud environments is related to many other research areas. Besides this thesis, there are many other studies and approaches with the goal of improving the efficiency and significance of benchmark experiments in cloud environments. The continuous benchmarking step to routinely verify non-functional properties is motivated and applied in various studies, as early detection and quantification of performance changes enables the monitoring of software performance metrics and their compliance with agreed SLAs. Besides our contribution towards load generation for microservices, there are other publications that address the benchmarking of microservice applications and refer to various other aspects. The more efficient execution of microbenchmarks is also discussed in many studies and the presented approaches can be combined with our optimizations under certain conditions. Finally, there are several other studies on benchmarking FaaS platforms, which often focus on single aspects and do not have a holistic application focus.

Conclusion

Benchmarking enables the determination of performance metrics in an artificial environment and thus ensures, for example, the compliance with SLAs. Benchmarking microservice platforms and applications in cloud environments, however, is challenging. First, there are performance variations that make it difficult to accurately determine performance metrics and reproduce results. Thus, there is a trade-off between accuracy on the one hand and effort on the other: The longer a benchmarking experiment runs and the more often it is repeated, the more accurately it is possible to quantify performance metrics and detect performance changes. Second, as microservices applications are constantly evolving, extensive benchmarking experiments after every code change are unrealistic. Long lasting benchmarking experiments are very time-consuming, costly, and slow down the fast development cycles with several source code changes per day. Third, because cloud platforms are constantly updated as well. New hardware or software updates on the provider side also affect the performance of a microservice application.

This chapter contains (partially adapted) material from our published research articles [72–78]. First, Section 7.1 summarizes the contributions of this thesis. Section 7.2 then discusses the contributions in the overall problem context and Section 7.3 presents an outlook on further research.

7.1 Summary

In this thesis, we addressed three challenges in benchmarking microservice platforms and applications in cloud environments and proposed possible solutions:

Automatic Workload Generation for Benchmarking Microservices:

Microservices do not share common interfaces, which would simplify the creation of benchmarks. Until now, benchmarks had to be created manually for each microservice application. Moreover, existing benchmarks had to be adjusted after every service code change which, e.g., introduced a new required request parameter. In Chapter 3, we proposed a benchmarking approach for REST-based microservice applications which automatically generates a workload based on interface description files and abstract interaction patterns. Our approach identifies links between interactions and resolves respective workload values at runtime. In total, automatically deriving the benchmark workload for a microservice application from service descriptions reduces the manual implementations effort drastically and eases the embedding of a continuous benchmarking step in CI/CD pipelines of REST-based microservice applications. Even though it is probably not possible to run a benchmark after every code change due to the long execution time

of application benchmarks, defined QoS requirements can be ensured every night or for every release with little effort.

Deriving Practically Relevant Microbenchmark Suites:

While application benchmarks are the gold standard and very powerful as they benchmark complete systems, they are hardly suitable for regular use in continuous integration pipelines due to their long execution time and high costs [22, 25]. Alternatively, less complex and therefore less costly microbenchmarks could be used, which are also easier to integrate into build pipelines due to their simpler setup [113]. However, especially in large software projects with hundreds or thousands of microbenchmarks, a complete suite execution can take several hours, making the evaluation for every code change impractical. Thus, applying one of these two approaches to a large project with many application developers, hundreds of source code files, and multiple code changes per day would soon create a stack of benchmark tasks that would prevent fast-paced software development and integration of individual changes. In Chapter 4, we presented an approach that removes redundancies in a microbenchmark suite and recommends functions which should be covered by a microbenchmark based on an application benchmark call graph. Using our approach reduces the number of microbenchmarks and significantly shortens the overall suite execution duration as it removes redundancies and the final optimized suite solely consists of microbenchmarks that actually evaluate functions that are relevant in a production environment (which is represented by the application benchmark call graph). Our extensive evaluation using two open-source applications shows that our approach can help to reduce the duration of a continuous benchmarking step in a CI/CD pipeline if false alarms can be tolerated and microbenchmarks cover all relevant code sections.

A Benchmarking Framework for FaaS Environments:

Comparing FaaS platforms is hard as developers do not directly control the infrastructure and only define high-level parameters. Existing work on benchmarking FaaS platforms so far focused on single aspects while ignoring the overall application performance. In Chapter 5, we presented BeFaaS as an extensible application-centric benchmarking framework for studying and comparing FaaS platform providers in detail. Our evaluations using BeFaaS show that network transmission is a major contributor to the overall response latency, edge-focused deployments can be more efficient, the delay between triggering a function by publishing an event and the actual start of the function can be up to 800ms, and Azure appears unaffected by cold starts for our workload. Moreover, developers can use the extensible features of the framework to study their microservice-oriented FaaS application in several setups to find the most effective one or identify performance bottle necks. BeFaaS meets all requirements on a modern FaaS benchmarking framework and is, at this time, the only framework supporting cross-platform deployments of FaaS applications.

7.2 Discussion

Performance variations in cloud environments cause long running and repetitive benchmarking experiments to accurately quantify performance metrics. Our case study benchmarking two large open-source projects has shown that techniques such as duet benchmarking and RMIT execution enable reasonably accurate results and bootstrapping helps to estimate the respective CIs. Taken together, this enables reliable conclusions as performance variations are taken into account in the analysis and results. Furthermore, optimized microbenchmark suites shorten the execution time significantly while still detecting performance changes when covering the practically relevant source code. The early performance change detection capabilities in relation to application benchmarks have also been proven in a realistic testbed [99].

Frequent source code changes within the services of a microservice application require frequent adaptations and time-consuming maintenance of application benchmarks. Our approach to automatically generate the benchmark and workload drastically simplifies this process and automates many parts of it, requiring only a few manual inputs. While it is unrealistic to run a complex and costly application benchmark for every code change, it can ensure performance parameters for every release. In addition, if (ideally) all services of the application can record call graphs, this can provide valuable input for optimized microbenchmark suites of the individual services. Instead of evaluating each service after a source code change in a complex application benchmark, an optimized microbenchmark suite can give a fast performance feedback for the individual single service. In this setup, both application benchmark and microbenchmark suite would have to be constantly reviewed and maintained, but this can be automated for the most parts.

Changes and updates by the cloud provider also affect the performance metrics of microservice applications, further complicating benchmarking. This is particularly challenging for services that are hosted as functions on FaaS platforms, as there is only a limited influence on the exact deployment. Nevertheless, BeFaaS enables performance evaluation of platforms and comparisons between different providers. The extensibility and library of the framework can also be used to evaluate any FaaS application; it is not limited to the four application scenarios of the framework. Although the framework does not allow a direct comparison between two versions of the FaaS application without further modifications, it can be used to precisely evaluate the current state of the application and compare the performance of the application on platforms from different providers. Detecting performance changes caused by updates to the source code can be done, for example, using the RMIT execution sequence [69].

In total, the frequency and type of benchmarks depends on the individual microservice application and various factors, such as the number of services or their update frequency. The benchmarking of microservice applications is complex and time-consuming, but there are methods and techniques supporting it. Application benchmarks represent realistic workflows and can reliably determine performance metrics, e.g., to ensure compliance with SLAs. Automated load generation simplifies these benchmarks, and the BeFaaS framework helps to evaluate FaaS microservice applications and compare providers. Furthermore, optimized microbenchmark suites

can provide faster performance feedback, while a complete run of a microbenchmark suite can help identify root causes of performance changes.

7.3 Outlook

Embedding a periodic benchmarking step into a CI/CD pipeline of modern cloud-deployed microservice applications is still challenging. Our contributions help to tackle some problems by automatically generating a benchmarking workload for microservices, shortening the execution time of microbenchmark suites, and studying the cloud platforms. In practice, however, there is still a long way to go before benchmarks can simply be executed after every code change. Here, the major challenges are the execution duration of the benchmarks and the variability in the cloud environment. Developers ideally want to have performance feedback within ten minutes, but setup, benchmark execution, and analysis take far longer. Nevertheless, there are also further ideas on how the process can be taken forward.

First, generic benchmarking frameworks can help to introduce a continuous benchmarking step. Although benchmarks are highly individual, as are the systems they evaluate, many steps for benchmarks in the cloud environment can be generalized. There is a setup action which initializes virtual machines or sets up other cloud services, the installation and configuration of the SUT and a benchmarking client, a trigger mechanism to start the benchmark, an observation tool, a result collection, and an analysis to finally come up with insights. While there will certainly be no benchmark framework that meets all the requirements of the various systems out of the box, modular and extensible benchmarking frameworks are definitely thinkable. The users of such frameworks could either use already defined steps, customize them if necessary, or overwrite them with their own logic.

Another option could be to trigger different benchmarks types depending on the situation. Besides the execution of application benchmarks, complete microbenchmark suites, and optimized microbenchmark suites, it is also possible to trigger specific microbenchmarks which evaluate code sections that are modified by a code change. A benchmark management system could, e.g., evaluate the respective code change and decide which benchmark type to execute. While a minor change in a function will only trigger a single microbenchmark immediately, a larger code change with modifications in several class files will trigger the application benchmark and complete microbenchmark suite during the night.

Bibliography

- [1] Ali Abedi and Tim Brecht. “Conducting Repeatable Experiments in Highly Variable Cloud Computing Environments”. In: *Proc. of the International Conference on Performance Engineering (ICPE '17)*. ACM, 2017, pp. 287–292. DOI: 10.1145/3030207.3030229.
- [2] Ali Abedi, Andrew Heard, and Tim Brecht. “Conducting Repeatable Experiments and Fair Comparisons using 802.11n MIMO Networks”. In: *ACM SIGOPS Operating Systems Review*. ACM, 2015, pp. 41–50. DOI: 10.1145/2723872.2723879.
- [3] Carlos Mendes Aderaldo, Nabor C. Mendonça, Claus Pahl, and Pooyan Jamshidi. “Benchmark Requirements for Microservices Architecture Research”. In: *Proc. of the International Workshop on Establishing the Community-Wide Infrastructure for Architecture-Based Software Engineering*. IEEE, 2017, pp. 8–13. DOI: 10.1109/ECASE.2017.4.
- [4] Hammam M. AlGhamdi, Cor-Paul Bezemer, Weiyi Shang, Ahmed E. Hassan, and Parminder Flora. “Towards reducing the time needed for load testing”. In: *Journal of Software: Evolution and Process* (2020). Wiley. DOI: 10.1002/smr.2276.
- [5] Hammam M. AlGhamdi, Mark D. Syer, Weiyi Shang, and Ahmed E. Hassan. “An Automated Approach for Recommending When to Stop Performance Tests”. In: *Proc. of the International Conference on Software Maintenance and Evolution (ICSME '16)*. IEEE, 2016, pp. 279–289. DOI: 10.1109/ICSME.2016.46.
- [6] Deema Alshoaibi, Kevin Hannigan, Hiten Gupta, and Mohamed Wiem Mkaouer. “PRICE : Detection of Performance Regression Introducing Code Changes Using Static and Dynamic Metrics”. In: *Proc. of the International Symposium on Search Based Software Engineering (SSBSE '19)*. Springer, 2019, pp. 75–88. DOI: 10.1007/978-3-030-27455-9_6.
- [7] David Ameller, Claudia Ayala, Jordi Cabot, and Xavier Franch. “How do Software Architects Consider Non-functional Requirements: An Exploratory Study”. In: *Proc. of the International Requirements Engineering Conference (RE '12)*. IEEE, 2012, pp. 41–50. DOI: 10.1109/RE.2012.6345838.
- [8] Atakan Aral and Ivona Brandić. “Learning Spatiotemporal Failure Dependencies for Resilient Edge Computing Services”. In: *Transactions on Parallel and Distributed Systems* 32.7 (2021). IEEE, pp. 1578–1590. DOI: 10.1109/TPDS.2020.3046188.
- [9] Mohammad S. Aslanpour, Adel N. Toosi, Claudio Cicconetti, Bahman Javadi, Peter Sbarski, Davide Taibi, Marcos Assuncao, Sukhpal Singh Gill, Raj Gaire, and Shahram Dustdar. “Serverless Edge Computing: Vision and Challenges”. In: *Proc. of the Australasian Computer Science Week Multiconference (ACSW '21)*. ACM, 2021, pp. 1–10. DOI: 10.1145/3437378.3444367.

- [10] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. “RESTler: Stateful REST API Fuzzing”. In: *Proc. of the International Conference on Software Engineering (ICSE '19)*. IEEE, 2019, pp. 748–758. DOI: 10.1109/ICSE.2019.00083.
- [11] Timon Back and Vasilios Andrikopoulos. “Using a Microbenchmark to Compare Function as a Service Solutions”. In: *Proc. of the European Conference on Service-Oriented and Cloud Computing (ESOCC '18)*. Springer, 2018, pp. 146–160. DOI: 10.1007/978-3-319-99819-0_11.
- [12] Daniel Barcelona-Pons and Pedro García-López. “Benchmarking parallelism in FaaS platforms”. In: *Future Generation Computer Systems* 124 (2021). Elsevier, pp. 268–284. DOI: 10.1016/j.future.2021.06.005.
- [13] Luciano Baresi and Danilo Filgueira Mendonça. “Towards a Serverless Platform for Edge Computing”. In: *Proc. of the International Conference on Fog Computing (ICFC '19)*. IEEE, 2019, pp. 1–10. DOI: 10.1109/ICFC.2019.00008.
- [14] Ilja Behnke, Lauritz Thamsen, and Odej Kao. “Héctor: A Framework for Testing IoT Applications Across Heterogeneous Edge and Cloud Testbeds”. In: *Proc. of the International Conference on Utility and Cloud Computing (UCC '19)*. ACM, 2019, 15–20. DOI: 10.1145/3368235.3368832.
- [15] David Bermbach. “Quality of Cloud Services: Expect the Unexpected”. In: *Internet Computing (Invited Paper)* 21.1 (2017). IEEE, pp. 68–72. DOI: 10.1109/MIC.2017.1.
- [16] David Bermbach, Jonathan Bader, Jonathan Hasenburg, Tobias Pfandzelter, and Lauritz Thamsen. “AuctionWhisk: Using an auction-inspired approach for function placement in serverless fog platforms”. In: *Software: Practice and Experience* 52.5 (2022). Wiley, pp. 1143–1169. DOI: 10.1002/spe.3058.
- [17] David Bermbach, Abhishek Chandra, Chandra Krintz, Aniruddha Gokhale, Aleksander Slominski, Lauritz Thamsen, Everton Cavalcante, Tian Guo, Ivona Brandic, and Rich Wolski. “On the Future of Cloud Engineering”. In: *Proc. of the International Conference on Cloud Engineering (IC2E '21)*. IEEE, 2021, pp. 264–275. DOI: 10.1109/IC2E52221.2021.00044.
- [18] David Bermbach, Ahmet-Serdar Karakaya, and Simon Buchholz. “Using Application Knowledge to Reduce Cold Starts in FaaS Services”. In: *Proc. of the Symposium on Applied Computing (SAC '20)*. ACM, 2020, pp. 134–143. DOI: 10.1145/3341105.3373909.
- [19] David Bermbach, Jörn Kuhlenkamp, Akon Dey, Arunmozhi Ramachandran, Alan Fekete, and Stefan Tai. “BenchFoundry: A Benchmarking Framework for Cloud Storage Services”. In: *Proc. of the International Conference on Service-Oriented Computing (ICSOC '17)*. Springer, 2017, pp. 314–330. DOI: 10.1007/978-3-319-69035-3_22.
- [20] David Bermbach, Jörn Kuhlenkamp, Akon Dey, Sherif Sakr, and Raghunath Nambiar. “Towards an Extensible Middleware for Database Benchmarking”. In: *Proc. of the Technology Conference on Performance Evaluation and Benchmarking (TPCTC '14)*. Springer, 2015, pp. 82–96. DOI: 10.1007/978-3-319-15350-6_6.

-
- [21] David Bermbach, Frank Pallas, David García Pérez, Pierluigi Plebani, Maya Anderson, Ronen Kat, and Stefan Tai. “A Research Perspective on Fog Computing”. In: *Proc. of the Workshop on IoT Systems Provisioning and Management for Context-Aware Smart Cities (ISYCC '17)*. Springer, 2017, pp. 198–210. DOI: 10.1007/978-3-319-91764-1_16.
 - [22] David Bermbach and Stefan Tai. “Benchmarking Eventual Consistency: Lessons Learned from Long-Term Experimental Studies”. In: *Proc. of the International Conference on Cloud Engineering (IC2E '14)*. IEEE, 2014, pp. 47–56. DOI: 10.1109/IC2E.2014.37.
 - [23] David Bermbach and Erik Wittern. “Benchmarking Web API Quality”. In: *Proc. of the International Conference on Web Engineering (ICWE '16)*. Springer, 2016, pp. 188–206. DOI: 10.1007/978-3-319-38791-8_11.
 - [24] David Bermbach and Erik Wittern. “Benchmarking Web API Quality – Revisited”. In: *Journal of Web Engineering* 19.5-6 (2020). IEEE, pp. 603–646. DOI: 10.13052/jwe1540-9589.19563.
 - [25] David Bermbach, Erik Wittern, and Stefan Tai. *Cloud Service Benchmarking: Measuring Quality of Cloud Services from a Client Perspective*. Springer, 2017. DOI: 10.1007/978-3-319-55483-9.
 - [26] Cor-Paul Bezemer et al. “How is Performance Addressed in DevOps?” In: *Proc. of the International Conference on Performance Engineering (ICPE '19)*. ACM, 2019, pp. 45–50. DOI: 10.1145/3297663.3309672.
 - [27] Carsten Binnig, Donald Kossmann, Tim Kraska, and Simon Loesing. “How is the Weather tomorrow? Towards a Benchmark for the Cloud”. In: *Proc. of the International Workshop on Testing Database Systems (DBTest '09)*. ACM, 2009, pp. 1–6. DOI: 10.1145/1594156.1594168.
 - [28] Amir Hossein Borhani, Philipp Leitner, Bu-Sung Lee, Xiaorong Li, and Terence Hung. “WPress: An Application-Driven Performance Benchmark For Cloud-Based Virtual Machines”. In: *Proc. of the International Enterprise Distributed Object Computing Conference (EDOC '14)*. IEEE, 2014, pp. 101–109. DOI: 10.1109/EDOC.2014.23.
 - [29] Giacomo Brambilla, Marco Picone, Simone Cirani, Michele Amoretti, and Francesco Zanichelli. “A Simulation Platform for Large-Scale Internet of Things Scenarios in Urban Environments”. In: *Proc. of the International Conference on IoT in Urban Space (URB-IOT '14)*. ICST, 2014, pp. 50–55. DOI: 10.4108/icst.urb-iot.2014.257268.
 - [30] Lubomír Bulej, Tomáš Bureš, Vojtěch Horký, Jaroslav Kotrč, Lukáš Marek, Tomáš Trojánek, and Petr Tůma. “Unit testing performance with Stochastic Performance Logic”. In: *Automated Software Engineering* 24 (2017), pp. 139–187. DOI: 10.1007/s10515-015-0188-0.
 - [31] Lubomír Bulej, Tomáš Bureš, Jaroslav Kezníkl, Alena Koubková, Andrej Podzimek, and Petr Tůma. “Capturing Performance Assumptions Using Stochastic Performance Logic”. In: *Proc. of the International Conference on Performance Engineering (ICPE '12)*. ACM, 2012, pp. 311–322. DOI: 10.1145/2188286.2188345.

- [32] Lubomír Bulej, Vojtěch Horký, and Petr Tůma. “Initial Experiments with Duet Benchmarking: Performance Testing Interference in the Cloud”. In: *Proc. of the International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS '19)*. IEEE, 2019, pp. 249–255. DOI: 10.1109/MASCOTS.2019.00035.
- [33] Lubomír Bulej, Vojtěch Horký, Petr Tůma, François Farquet, and Aleksandar Prokopec. “Duet Benchmarking: Improving Measurement Accuracy in the Cloud”. In: *Proc. of the International Conference on Performance Engineering (ICPE '20)*. ACM, 2020, pp. 100–107. DOI: 10.1145/3358960.3379132.
- [34] Lubomír Bulej, Tomáš Kalibera, and Petr Tůma. “Repeated results analysis for middle-ware regression benchmarking”. In: *Performance Evaluation* 60.1 (2005). Elsevier, pp. 345–358. DOI: 10.1016/j.peva.2004.10.013.
- [35] Andrea Caracciolo, Mircea Filip Lungu, and Oscar Nierstrasz. “How Do Software Architects Specify and Validate Quality Requirements?” In: *Proc. of the European Conference on Software Architecture (ECSA '14)*. Springer, 2014, pp. 374–389. DOI: 10.1007/978-3-319-09970-5_32.
- [36] Jinfu Chen and Weiyi Shang. “An Exploratory Study of Performance Regression Introducing Code Changes”. In: *Proc. of the International Conference on Software Maintenance and Evolution (ICSME '17)*. IEEE, 2017, pp. 341–352. DOI: 10.1109/ICSME.2017.13.
- [37] Jinfu Chen, Weiyi Shang, and Emad Shihab. “PerfJIT: Test-level Just-in-time Prediction for Performance Regression Introducing Commits”. In: 48.5 (2022). IEEE, pp. 1529–1544. DOI: 10.1109/TSE.2020.3023955.
- [38] Tsong Yueh Chen and Man Fai Fai Lau. “A simulation study on some heuristics for test suite reduction”. In: *Information and software technology* 40.13 (1998). Elsevier, pp. 777–787. DOI: 10.1016/S0950-5849(98)00094-9.
- [39] Lawrence Chung and Julio Cesar Sampaio do Prado Leite. “On Non-Functional Requirements in Software Engineering”. In: *Conceptual Modeling: Foundations and Applications: Essays in Honor of John Mylopoulos* (2009), pp. 363–379. DOI: 10.1007/978-3-642-02463-4_19.
- [40] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. “Benchmarking Cloud Serving Systems with YCSB”. In: *Proc. of the Symposium on Cloud Computing (SOCC '10)*. ACM, 2010, pp. 143–154. DOI: 10.1145/1807128.1807152.
- [41] Marcin Copik, Grzegorz Kwasniewski, Maciej Besta, Michal Podstawski, and Torsten Hoefler. “SeBS: A Serverless Benchmark Suite for Function-as-a-Service Computing”. In: *Proc. of the International Middleware Conference (Middleware '21)*. ACM, 2021, pp. 64–78. DOI: 10.1145/3464298.3476133.
- [42] Robert Cordingly, Wen Shu, and Wes J. Lloyd. “Predicting Performance and Cost of Serverless Computing Functions with SAAF”. In: *Proc. of the International Conference on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and*

-
- Technology Congress (DASC/PiCom/CBDCCom/CyberSciTech '20)*. IEEE, 2020, pp. 640–649. DOI: 10.1109/DASC-PiCom-CBDCCom-CyberSciTech49142.2020.00111.
- [43] Robert Cordingley, Hanfei Yu, Varik Hoang, David Perez, David Foster, Zohreh Sadeghi, Rashad Hatchett, and Wes J. Lloyd. “Implications of Programming Language Selection for Serverless Data Processing Pipelines”. In: *Proc. of the International Conference on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCCom/CyberSciTech '20)*. IEEE, 2020, pp. 704–711. DOI: 10.1109/DASC-PiCom-CBDCCom-CyberSciTech49142.2020.00120.
- [44] David Daly. “Creating a Virtuous Cycle in Performance Testing at MongoDB”. In: *Proc. of the International Conference on Performance Engineering (ICPE '21)*. ACM, 2021, pp. 33–41. DOI: 10.1145/3427921.3450234.
- [45] David Daly, William Brown, Henrik Ingo, Jim O’Leary, and David Bradford. “The Use of Change Point Detection to Identify Software Performance Regressions in a Continuous Integration System”. In: *Proc. of the International Conference on Performance Engineering (ICPE '20)*. ACM, 2020, pp. 67–75. DOI: 10.1145/3358960.3375791.
- [46] Diego Elias Damasceno Costa, Cor-Paul Bezemer, Philipp Leitner, and Artur Andrzejak. “What’s Wrong With My Benchmark Results? Studying Bad Practices in JMH Benchmarks”. In: *Transactions on Software Engineering* 47.7 (2021). IEEE, pp. 1452–1467. DOI: 10.1109/TSE.2019.2925345.
- [47] Maya Daneva, Luigi Buglione, and Andrea Herrmann. “Software Architects’ Experiences of Quality Requirements: What We Know and What We Do Not Know?” In: *Proc. of the International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ '13)*. Springer, 2013, pp. 1–17. DOI: 10.1007/978-3-642-37422-7_1.
- [48] André De Camargo, Ivan Salvadori, Ronaldo dos Santos Mello, and Frank Siqueira. “An Architecture to Automate Performance Tests on Microservices”. In: *Proc. of the International Conference on Information Integration and Web-based Applications and Services (iiWAS '16)*. ACM, 2016, pp. 422–429. DOI: 10.1145/3011141.3011179.
- [49] Akon Dey, Alan Fekete, Raghunath Nambiar, and Uwe Röhm. “YCSB+T: Benchmarking Web-scale Transactional Databases”. In: *Proc. of the International Conference on Data Engineering Workshops (ICDEW '14)*. IEEE, 2014, pp. 223–230. DOI: 10.1109/ICDEW.2014.6818330.
- [50] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudre-Mauroux. “OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases”. In: *Proceedings of the VLDB Endowment* 7.4 (2013). VLDB Endowment, pp. 277–288. DOI: 10.14778/2732240.2732246.
- [51] Zishuo Ding, Jinfu Chen, and Weiyi Shang. “Towards the Use of the Readily Available Tests from the Release Pipeline as Performance Tests. Are We There Yet?” In: *Proc. of the International Conference on Software Engineering (ICSE '20)*. ACM, 2020, 1435–1446. DOI: 10.1145/3377811.3380351.

- [52] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. “Microservices: Yesterday, Today, and Tomorrow”. In: *Present and Ulterior Software Engineering*. Springer, 2017, pp. 195–216. DOI: 10.1007/978-3-319-67425-4_12.
- [53] Ted Dunning and Ellen Friedman. *Time Series Databases: New Ways to Store and Access Data*. O’Reilly, 2014. ISBN: 978-1-491-91702-2.
- [54] Jens Ehlers, André van Hoorn, Jan Waller, and Wilhelm Hasselbring. “Self-Adaptive Software System Monitoring for Performance Anomaly Localization”. In: *Proc. of the International Conference on Autonomic Computing (ICAC ’11)*. ACM, 2011, pp. 197–200. DOI: 10.1145/1998582.1998628.
- [55] Simon Eismann, Cor-Paul Bezemer, Weiyi Shang, Dušan Okanović, and André van Hoorn. “Microservices: A Performance Tester’s Dream or Nightmare?”. In: *Proc. of the International Conference on Performance Engineering (ICPE ’20)*. ACM, 2020, pp. 138–149. DOI: 10.1145/3358960.3379124.
- [56] Simon Eismann, Joel Scheuner, Erwin Van Eyk, Maximilian Schwinger, Johannes Grohmann, Nikolas Herbst, Cristina L. Abad, and Alexandru Iosup. “The State of Serverless Applications: Collection, Characterization, and Community Consensus”. In: *Transactions on Software Engineering* 48.10 (2022). IEEE, pp. 4152–4166. DOI: 10.1109/TSE.2021.3113940.
- [57] Erwin van Eyk, Joel Scheuner, Simon Eismann, Cristina L. Abad, and Alexandru Iosup. “Beyond Microbenchmarks: The SPEC-RG Vision for A Comprehensive Serverless Benchmark”. In: *Proc. of the International Conference on Performance Engineering (ICPE ’20)*. ACM, 2020, pp. 26–31. DOI: 10.1145/3375555.3384381.
- [58] Dror G. Feitelson, Eitan Frachtenberg, and Kent L. Beck. “Development and Deployment at Facebook”. In: *Internet Computing* 17.4 (2013). IEEE, pp. 8–17. DOI: 10.1109/MIC.2013.25.
- [59] Kamil Figiela, Adam Gajek, Adam Zima, Beata Obrok, and Maciej Malawski. “Performance evaluation of heterogeneous cloud functions”. In: *Concurrency and Computation: Practice and Experience* 30.23 (2018). Wiley, pp. 1–16. DOI: 10.1002/cpe.4792.
- [60] Enno Folkerts, Alexander Alexandrov, Kai Sachs, Alexandru Iosup, Volker Markl, and Cafer Tosun. “Benchmarking in the Cloud: What it Should, Can, and Cannot Be”. In: *Proc. of the Technology Conference on Performance Evaluation and Benchmarking (TPCTC ’12)*. Springer, 2013, pp. 173–188. DOI: 10.1007/978-3-642-36727-4_12.
- [61] King Chun Foo, Zhen Ming Jiang, Bram Adams, Ahmed E. Hassan, Ying Zou, and Parminder Flora. “Mining Performance Regression Testing Repositories for Automated Performance Analysis”. In: *Proc. of the International Conference on Quality Software (QSIC ’10)*. IEEE, 2010, pp. 32–41. DOI: 10.1109/QSIC.2010.35.
- [62] King Chun Foo, Zhen Ming (Jack) Jiang, Bram Adams, Ahmed E. Hassan, Ying Zou, and Parminder Flora. “An Industrial Case Study on the Automated Detection of Performance Regressions in Heterogeneous Environments”. In: *Proc. of the International Conference on*

-
- Software Engineering (ICSE '15)*. IEEE, 2015, pp. 159–168. DOI: 10.1109/ICSE.2015.144.
- [63] Martin Fowler. *Richardson Maturity Model*. 2010. URL: <https://martinfowler.com/articles/richardsonMaturityModel.html>.
- [64] Martin Fowler. *Continuous Integration*. 2024. URL: <https://martinfowler.com/articles/continuousIntegration.html>.
- [65] Paolo Di Francesco, Ivano Malavolta, and Patricia Lago. “Research on Architecting Microservices: Trends, Focus, and Potential for Industrial Adoption”. In: *Proc. of the International Conference on Software Architecture (ICSA '17)*. IEEE, 2017, pp. 21–30. DOI: 10.1109/ICSA.2017.24.
- [66] Yu Gan et al. “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems”. In: *Proc. of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. ACM, 2019, pp. 3–18. DOI: 10.1145/3297858.3304013.
- [67] Gareth George, Fatih Bakir, Rich Wolski, and Chandra Krintz. “NanoLambda: Implementing Functions as a Service at All Resource Scales for the Internet of Things”. In: *Proc. of the Symposium on Edge Computing (SEC '20)*. IEEE, 2020, pp. 220–231. DOI: 10.1109/SEC50012.2020.00035.
- [68] Andy Georges, Dries Buytaert, and Lieven Eeckhout. “Statistically Rigorous Java Performance Evaluation”. In: *Proc. of the Conference on Object-Oriented Programming, Systems, and Applications (OOPSLA '07)*. ACM, 2007, pp. 57–76. DOI: 10.1145/1297027.1297033.
- [69] Martin Grambow, Tim Dockenfuß, Trevor Schirmer, Nils Japke, and David Bermbach. “Efficiently Detecting Performance Changes in FaaS Application Releases”. In: *Proc. of the International Workshop on Serverless Computing (WoSC '23)*. ACM, 2023, pp. 13–17. DOI: 10.1145/3631295.3631395.
- [70] Martin Grambow, Jonathan Hasenburg, and David Bermbach. “Public Video Surveillance: Using the Fog to Increase Privacy”. In: *Proc. of the Workshop on Middleware and Applications for the Internet of Things (M4IoT '18)*. ACM, 2018, pp. 11–14. DOI: 10.1145/3286719.3286722.
- [71] Martin Grambow, Jonathan Hasenburg, Tobias Pfandzelter, and David Bermbach. “Is it Safe to Dockerize my Database Benchmark?” In: *Proc. of the Symposium on Applied Computing, Posters Track (SAC '19)*. ACM, 2019, pp. 341–344. DOI: 10.1145/3297280.3297545.
- [72] Martin Grambow, Denis Kovalev, Christoph Laaber, Philipp Leitner, and David Bermbach. “Using Microbenchmark Suites to Detect Application Performance Changes”. In: *Transactions on Cloud Computing* 11.3 (2023). IEEE, pp. 2575–2590. DOI: 10.1109/TCC.2022.3217947.
- [73] Martin Grambow, Christoph Laaber, Philipp Leitner, and David Bermbach. “Using application benchmark call graphs to quantify and improve the practical relevance of micro-

- benchmark suites”. In: *PeerJ Computer Science* (2021). PeerJ. DOI: 10.7717/peerj-cs.548.
- [74] Martin Grambow, Fabian Lehmann, and David Bermbach. “Continuous Benchmarking: Using System Benchmarking in Build Pipelines”. In: *Proc. of the Workshop on Service Quality and Quantitative Evaluation in new Emerging Technologies (SQUEET '19)*. IEEE, 2019, pp. 241–246. DOI: 10.1109/IC2E.2019.00039.
- [75] Martin Grambow, Lukas Meusel, Erik Wittern, and David Bermbach. “Benchmarking Microservice Performance: A Pattern-based Approach”. In: *Proc. of the Symposium on Applied Computing (SAC '20)*. ACM, 2020, pp. 232–241. DOI: 10.1145/3341105.3373875.
- [76] Martin Grambow, Tobias Pfandzelter, and David Bermbach. *Application-Centric Benchmarking of Distributed FaaS Platforms using BeFaaS*. Tech. rep. 3S.2024.1. TU Berlin, 2024.
- [77] Martin Grambow, Tobias Pfandzelter, Luk Burchard, Carsten Schubert, Max Zhao, and David Bermbach. “BeFaaS: An Application-Centric Benchmarking Framework for FaaS Platforms”. In: *Proc. of the International Conference on Cloud Engineering (IC2E '21)*. IEEE, 2021, pp. 1–8. DOI: 10.1109/IC2E52221.2021.00014.
- [78] Martin Grambow, Erik Wittern, and David Bermbach. “Benchmarking the Performance of Microservice Applications”. In: *SIGAPP Applied Computing Review* 20.3 (2020). ACM, pp. 20–34. DOI: 10.1145/3429204.3429206.
- [79] Xue Han and Tingting Yu. “An Empirical Study on Performance Bugs for Highly Configurable Software Systems”. In: *Proc. of the International Symposium on Empirical Software Engineering and Measurement (ESEM '16)*. ACM, 2016, pp. 1–10. DOI: 10.1145/2961111.2962602.
- [80] Jonathan Hasenburg, Martin Grambow, and David Bermbach. “Towards A Replication Service for Data-Intensive Fog Applications”. In: *Proc. of the Symposium on Applied Computing, Posters Track (SAC '20)*. ACM, 2020, pp. 267–270. DOI: 10.1145/3341105.3374060.
- [81] Jonathan Hasenburg, Martin Grambow, and David Bermbach. “MockFog 2.0: Automated Execution of Fog Application Experiments in the Cloud”. In: *Transactions on Cloud Computing* 11.1 (2023). IEEE, pp. 58–70. DOI: 10.1109/TCC.2021.3074988.
- [82] Jonathan Hasenburg, Martin Grambow, Elias Grunewald, Sascha Huk, and David Bermbach. “MockFog: Emulating Fog Computing Infrastructure in the Cloud”. In: *Proc. of the International Conference on Fog Computing (ICFC '19)*. IEEE, 2019, pp. 144–152. DOI: 10.1109/ICFC.2019.00026.
- [83] Wilhelm Hasselbring. “Benchmarking as Empirical Standard in Software Engineering Research”. In: *Evaluation and Assessment in Software Engineering (EASE '21)*. ACM, 2021, pp. 365–372. DOI: 10.1145/3463274.3463361.
- [84] Sen He, Glenna Manns, John Saunders, Wei Wang, Lori Pollock, and Mary Lou Soffa. “A Statistics-Based Performance Testing Methodology for Cloud Applications”. In: *Proc. of the Joint Meeting on European Software Engineering Conference and Symposium on the*

-
- Foundations of Software Engineering (ESEC/FSE '19)*. ACM, 2019, pp. 188–199. DOI: 10.1145/3338906.3338912.
- [85] Christoph Heger, Jens Happe, and Roozbeh Farahbod. “Automated Root Cause Isolation of Performance Regressions During Software Development”. In: *Proc. of the International Conference on Performance Engineering (ICPE '13)*. ACM, 2013, pp. 27–38. DOI: 10.1145/2479871.2479879.
- [86] Sören Henning and Wilhelm Hasselbring. “Theodolite: Scalability Benchmarking of Distributed Stream Processing Engines in Microservice Architectures”. In: *Big Data Research* 25 (2021). Elsevier. DOI: 10.1016/j.bdr.2021.100209.
- [87] Sören Henning and Wilhelm Hasselbring. “A configurable method for benchmarking scalability of cloud-native applications”. In: *Empirical Software Engineering* 27.6 (2022). Springer. DOI: 10.1007/s10664-022-10162-1.
- [88] Sören Henning, Benedikt Wetzels, and Wilhelm Hasselbring. “Reproducible Benchmarking of Cloud-Native Applications with the Kubernetes Operator Pattern”. In: *Proc. of the Symposium on Software Performance 2021 (SSP '21)*. CEUR, 2021.
- [89] Vojtěch Horký, Peter Libiř, Lukáš Marek, Antonín Steinhäuser, and Petr Tůma. “Utilizing Performance Unit Tests To Increase Performance Awareness”. In: *Proc. of the International Conference on Performance Engineering (ICPE '15)*. ACM, 2015, pp. 289–300. DOI: 10.1145/2668930.2688051.
- [90] Peng Huang, Xiao Ma, Dongcai Shen, and Yuanyuan Zhou. “Performance Regression Testing Target Prioritization via Performance Risk Analysis”. In: *Proc. of the International Conference on Software Engineering (ICSE '14)*. ACM, 2014, pp. 60–71. DOI: 10.1145/2568225.2568232.
- [91] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010. ISBN: 978-0-321-60191-9.
- [92] Karl Huppler. “The Art of Building a Good Benchmark”. In: *Proc. of the Technology Conference on Performance Evaluation and Benchmarking (TPCTC '09)*. Springer, 2009, pp. 18–30. DOI: 10.1007/978-3-642-10424-4_3.
- [93] InfluxData Inc. *InfluxDB 2.0*. 2024. URL: <https://github.com/influxdata/influxdb/tree/2.0>.
- [94] Victoria Metrics Inc. *VictoriaMetrics*. 2024. URL: <https://github.com/VictoriaMetrics/VictoriaMetrics>.
- [95] Henrik Ingo and David Daly. “Automated System Performance Testing at MongoDB”. In: *Proc. of the workshop on Testing Database Systems (DBTest '20)*. ACM, 2020, pp. 1–6. DOI: 10.1145/3395032.3395323.
- [96] Alexandru Iosup, Nezih Yigitbasi, and Dick Epema. “On the Performance Variability of Production Cloud Services”. In: *Proc. of the International Symposium on Cluster, Cloud and Grid Computing (CCGRID '11)*. IEEE, 2011, pp. 104–113. DOI: 10.1109/CCGrid.2011.22.

- [97] Ana Ivanchikj, Ilija Gjorgjiev, and Cesare Pautasso. “RESTalk Miner: Mining RESTful Conversations, Pattern Discovery and Matching”. In: *Proc. of the International Conference on Service-Oriented Computing - Workshops (ICSOC '18)*. Springer, 2018, pp. 470–475. DOI: 10.1007/978-3-030-17642-6_46.
- [98] Pooyan Jamshidi, Claus Pahl, Nabor C. Mendonça, James Lewis, and Stefan Tilkov. “Microservices: The Journey So Far and Challenges Ahead”. In: *IEEE Software* 35.3 (2018). IEEE, pp. 24–35. DOI: 10.1109/MS.2018.2141039.
- [99] Nils Japke, Christoph Witzko, Martin Grambow, and David Bermbach. “The Early Microbenchmark Catches the Bug – Studying Performance Issues Using Micro- and Application Benchmarks”. In: *Proc. of the International Conference on Utility and Cloud Computing (UCC '23)*. ACM, 2024, pp. 1–10. DOI: 10.1145/3603166.3632128.
- [100] Omar Javed, Joshua Heneage Dawes, Marta Han, Giovanni Franzoni, Andreas Pfeiffer, Giles Reger, and Walter Binder. “PerfCI: A Toolchain for Automated Performance Testing during Continuous Integration of Python Projects”. In: *Proc. of the International Conference on Automated Software Engineering (ASE '20)*. IEEE, 2020, pp. 1344–1348. DOI: 10.1145/3324884.3415288.
- [101] Zhen Ming Jiang and Ahmed E. Hassan. “A Survey on Load Testing of Large-Scale Software Systems”. In: *Transactions on Software Engineering* 41.11 (2015). IEEE, pp. 1091–1118. DOI: 10.1109/TSE.2015.2445340.
- [102] Tomas Kalibera and Richard Jones. *Quantifying Performance Changes with Effect Size Confidence Intervals*. 2020. URL: <https://arxiv.org/abs/2007.10899>.
- [103] Chia Hung Kao, Chun Cheng Lin, and Juei-Nan Chen. “Performance Testing Framework for REST-based Web Applications”. In: *Proc. of the International Conference on Quality Software (QSIC '13)*. IEEE, 2013, pp. 349–354. DOI: 10.1109/QSIC.2013.32.
- [104] Alexander Keller and Heiko Ludwig. “The WSLA Framework: Specifying and Monitoring Service Level Agreements for Web Services”. In: *Journal of Network and Systems Management* 11.1 (2003). Springer, pp. 57–81. DOI: 10.1023/A:1022445108617.
- [105] Jeongchul Kim and Kyungyong Lee. “FunctionBench: A Suite of Workloads for Serverless Cloud Function Service”. In: *Proc. of the International Conference on Cloud Computing (CLOUD '19)*. IEEE, 2019, pp. 502–504. DOI: 10.1109/CLOUD.2019.00091.
- [106] Jóakim v. Kistowski, Jeremy A. Arnold, Karl Huppler, Klaus-Dieter Lange, John L. Henning, and Paul Cao. “How to Build a Benchmark”. In: *Proc. of the International Conference on Performance Engineering (ICPE '15)*. ACM, 2015, pp. 333–336. DOI: 10.1145/2668930.2688819.
- [107] Markus Klems, David Bermbach, and René Weinert. “A Runtime Quality Measurement Framework for Cloud Database Service Systems”. In: *Proc. of the International Conference on the Quality of Information and Communications Technology (QUATIC '12)*. IEEE, 2012, pp. 38–46. DOI: 10.1109/QUATIC.2012.17.
- [108] Markus Klems, Michael Menzel, and Robin Fischer. “Consistency Benchmarking: Evaluating the Consistency Behavior of Middleware Services in the Cloud”. In: *Proc. of the*

-
- International Conference on Service-Oriented Computing (ICSOC '10)*. Springer, 2010, pp. 627–634. DOI: 10.1007/978-3-642-17358-5_48.
- [109] Jörn Kuhlenkamp, Markus Klems, and Oliver Röss. “Benchmarking Scalability and Elasticity of Distributed Database Systems”. In: *Proceedings of the VLDB Endowment* 7.12 (2014). VLDB Endowment, pp. 1219–1230. DOI: 10.14778/2732977.2732995.
- [110] Tobias Kurze, Markus Klems, David Bermbach, Alexander Lenk, Stefan Tai, and Marcel Kunze. “Cloud Federation”. In: *Proc. of the International Conference on Cloud Computing, GRIDs, and Virtualization*. IARIA, 2011, pp. 32–38.
- [111] Christoph Laaber, Mikael Basmaci, and Pasquale Salza. “Predicting unstable software benchmarks using static source code features”. In: *Empirical Software Engineering* 26.6 (2021). Springer, pp. 1–53. DOI: 10.1007/s10664-021-09996-y.
- [112] Christoph Laaber, Harald C Gall, and Philipp Leitner. “Applying test case prioritization to software microbenchmarks”. In: *Empirical Software Engineering* 26.6 (2021). Springer, pp. 1–48. DOI: 10.1007/s10664-021-10037-x.
- [113] Christoph Laaber and Philipp Leitner. “An Evaluation of Open-Source Software Microbenchmark Suites for Continuous Performance Assessment”. In: *Proc. of the International Conference on Mining Software Repositories (MSR '18)*. ACM, 2018, pp. 119–130. DOI: 10.1145/3196398.3196407.
- [114] Christoph Laaber, Joel Scheuner, and Philipp Leitner. “Software Microbenchmarking in the Cloud. How Bad is it Really?”. In: *Empirical Software Engineering* 24.4 (2019). Springer, pp. 2469–2508. DOI: 10.1007/s10664-019-09681-1.
- [115] Christoph Laaber, Stefan Würsten, Harald C. Gall, and Philipp Leitner. “Dynamically Reconfiguring Software Microbenchmarks: Reducing Execution Time without Sacrificing Result Quality”. In: *Proc. of the Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. ACM, 2020, pp. 989–1001. DOI: 10.1145/3368089.3409683.
- [116] Hyungro Lee, Kumar Satyam, and Geoffrey Fox. “Evaluation of Production Serverless Computing Environments”. In: *Proc. of the International Conference on Cloud Computing (CLOUD '18)*. IEEE, 2018, pp. 442–450. DOI: 10.1109/CLOUD.2018.00062.
- [117] Philipp Leitner and Cor-Paul Bezemer. “An Exploratory Study of the State of Practice of Performance Testing in Java-Based Open Source Projects”. In: *Proc. of the International Conference on Performance Engineering (ICPE '17)*. ACM, 2017, pp. 373–384. DOI: 10.1145/3030207.3030213.
- [118] Philipp Leitner and Jürgen Cito. “Patterns in the Chaos - A Study of Performance Variation and Predictability in Public IaaS Clouds”. In: *Transactions on Internet Technology* 16.3 (2016). ACM, pp. 1–23. DOI: 10.1145/2885497.
- [119] Alexander Lenk, Markus Klems, Jens Nimis, Stefan Tai, and Thomas Sandholm. “What’s Inside the Cloud? An Architectural Map of the Cloud Landscape”. In: *Proc. of the Workshop on Software Engineering Challenges of Cloud Computing (ICSE '09)*. IEEE, 2009, pp. 23–31. DOI: 10.1109/CLOUD.2009.5071529.

- [120] Alexander Lenk, Michael Menzel, Johannes Lipsky, Stefan Tai, and Philipp Offermann. “What Are You Paying For? Performance Benchmarking for Infrastructure-as-a-Service Offerings”. In: *Proc. of the International Conference on Cloud Computing (CLOUD '11)*. IEEE, 2011, pp. 484–491. DOI: 10.1109/CLOUD.2011.80.
- [121] James Lewis and Martin Fowler. *Microservices*. 2014. URL: <https://martinfowler.com/articles/microservices.html>.
- [122] JinJin Lin, Pengfei Chen, and Zibin Zheng. “Microscope: Pinpoint Performance Issues with Causal Graphs in Micro-service Environments”. In: *Proc. of the International Conference on Service-Oriented Computing (ICSOC '18)*. Springer, 2018, pp. 3–20. DOI: 10.1007/978-3-030-03596-9_1.
- [123] Heiko Ludwig, Alexander Keller, Asit Dan, Richard P King, and Richard Franck. “Web Service Level Agreement (WSLA) Language Specification”. In: *IBM Corporation* (2003).
- [124] Ivan Lujic, Vincenzo De Maio, Klaus Pollhammer, Ivan Bodrozic, Josip Lasic, and Ivona Brandic. “Increasing Traffic Safety with Real-Time Edge Analytics and 5G”. In: *Proc. of the International Workshop on Edge Systems, Analytics and Networking (EdgeSys '21)*. ACM, 2021, pp. 19–24. DOI: 10.1145/3434770.3459732.
- [125] Qi Luo, Kevin Moran, Lingming Zhang, and Denys Poshyvanyk. “How Do Static and Dynamic Test Case Prioritization Techniques Perform on Modern Software Systems? An Extensive Study on GitHub Projects”. In: *Transactions on Software Engineering* 45.11 (2019). IEEE, pp. 1054–1080. DOI: 10.1109/TSE.2018.2822270.
- [126] Vincenzo De Maio, David Bermbach, and Ivona Brandic. “TAROT: Spatio-Temporal Function Placement for Serverless Smart City Applications”. In: *Proc. of the International Conference on Utility and Cloud Computing (UCC '22)*. IEEE, 2022, pp. 21–30. DOI: 10.1109/UCC56403.2022.00013.
- [127] Pascal Maissen, Pascal Felber, Peter Kropf, and Valerio Schiavoni. “FaaSdom: A Benchmark Suite for Serverless Computing”. In: *Proc. of the International Conference on Distributed and Event-based Systems (DEBS '20)*. ACM, 2020, pp. 73–84. DOI: 10.1145/3401025.3401738.
- [128] Maciej Malawski, Kamil Figiela, Adam Gajek, and Adam Zima. “Benchmarking Heterogeneous Cloud Functions”. In: *Proc. of the European Conference on Parallel Processing (Euro-Par '17)*. Springer, 2017, pp. 415–426. DOI: 10.1007/978-3-319-75178-8_34.
- [129] Johannes Manner, Martin Endreß, Tobias Heckel, and Guido Wirtz. “Cold Start Influencing Factors in Function as a Service”. In: *Proc. of the International Conference on Utility and Cloud Computing (UCC '18)*. IEEE, 2018, pp. 181–188. DOI: 10.1109/UCC-Companion.2018.00054.
- [130] Aleksander Maricq, Dmitry Duplyakin, Ivo Jimenez, Carlos Maltzahn, Ryan Stutsman, and Robert Ricci. “Taming Performance Variability”. In: *Prof. of the USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*. USENIX, 2018, pp. 409–425.

-
- [131] Horácio Martins, Filipe Araujo, and Paulo Rupino da Cunha. “Benchmarking Serverless Computing Platforms”. In: *Journal of Grid Computing* 18.4 (2020). Springer, pp. 691–709. DOI: 10.1007/s10723-020-09523-1.
 - [132] David S. Matteson and Nicholas A. James. “A Nonparametric Approach for Multiple Change Point Analysis of Multivariate Data”. In: 109.505 (2014), pp. 334–345. DOI: 10.1080/01621459.2013.849605.
 - [133] Jonathan McChesney, Nan Wang, Ashish Tanwer, Eyal de Lara, and Blesson Varghese. “DeFog: Fog Computing Benchmarks”. In: *Proc. of the Symposium on Edge Computing (SEC '19)*. ACM, 2019, pp. 47–58. DOI: 10.1145/3318216.3363299.
 - [134] Nagy Mostafa and Chandra Krintz. “Tracking Performance Across Software Revisions”. In: *Proc. of the International Conference on Principles and Practice of Programming in Java (PPPJ '09)*. ACM, 2009, pp. 162–171. DOI: 10.1145/1596655.1596682.
 - [135] Shaikh Mostafa, Xiaoyin Wang, and Tao Xie. “PerfRanker: Prioritization of Performance Regression Tests for Collection-Intensive Software”. In: *Proc. of the International Symposium on Software Testing and Analysis (ISSTA '17)*. ACM, 2017, pp. 23–34. DOI: 10.1145/3092703.3092725.
 - [136] Stefan Mühlbauer, Sven Apel, and Norbert Siegmund. “Identifying software performance changes across variants and versions”. In: *Proc. of the International Conference on Automated Software Engineering (ASE '20)*. ACM, 2020, pp. 611–622. DOI: 10.1145/3324884.3416573.
 - [137] Steffen Müller, David Bermbach, Stefan Tai, and Frank Pallas. “Benchmarking the Performance Impact of Transport Layer Security in Cloud Database Systems”. In: *Proc. of the International Conference on Cloud Engineering (IC2E '14)*. IEEE, 2014, pp. 27–36. DOI: 10.1109/IC2E.2014.48.
 - [138] Gail C. Murphy, David Notkin, William G. Griswold, and Erica S. Lan. “An Empirical Study of Static Call Graph Extractors”. In: *Transactions on Software Engineering and Methodology* 7.2 (1998), pp. 158–191. DOI: 10.1145/279310.279314.
 - [139] Iulian Neamtiu and Tudor Dumitraş. “Cloud Software Upgrades: Challenges and Opportunities”. In: *Proc. of the International Workshop on the Maintenance and Evolution of Service-Oriented and Cloud-Based Systems (MESOCA '11)*. IEEE, 2011, pp. 1–10. DOI: 10.1109/MESOCA.2011.6049037.
 - [140] Sam Newman. *Building Microservices*. O’Reilly Media, 2021. ISBN: 978-1-4920-3402-5.
 - [141] Thanh H. D. Nguyen, Meiyappan Nagappan, Ahmed E. Hassan, Mohamed Nasser, and Parminder Flora. “An Industrial Case Study of Automatically Identifying Performance Regression-Causes”. In: *Proc. of the Working Conference on Mining Software Repositories (MSR '14)*. ACM, 2014, pp. 232–241. DOI: 10.1145/2597073.2597092.
 - [142] Jakob Nielsen. *Usability Engineering*. Morgan Kaufmann, 1994. ISBN: 978-0-08-052029-2.
 - [143] Adrian Nistor, Tian Jiang, and Lin Tan. “Discovering, Reporting, and Fixing Performance Bugs”. In: *Proc. of the Working Conference on Mining Software Repositories (MSR '13)*. IEEE, 2013, pp. 237–246. DOI: 10.1109/MSR.2013.6624035.

- [144] Augusto Born de Oliveira, Sebastian Fischmeister, Amer Diwan, Matthias Hauswirth, and Peter F Sweeney. “Perphecy: Performance Regression Test Selection Made Simple but Effective”. In: *Proc. of the International Conference on Software Testing, Verification and Validation (ICST '17)*. IEEE, 2017, pp. 103–113. DOI: 10.1109/ICST.2017.17.
- [145] Frank Pallas and Martin Grambow. “Three Tales of Disillusion: Benchmarking Property Preserving Encryption Schemes”. In: *Proc. of the International Conference on Trust, Privacy and Security in Digital Business (TrustBus '18)*. Springer, 2018, pp. 39–54. DOI: 10.1007/978-3-319-98385-1_4.
- [146] Frank Pallas, Philip Raschke, and David Bermbach. “Fog Computing as Privacy Enabler”. In: *Internet Computing 24.4* (2020). IEEE, pp. 15–21. DOI: 10.1109/MIC.2020.2979161.
- [147] Ioannis Papapanagiotou and Vinay Chella. *NDBench: Benchmarking Microservices at Scale*. 2018. URL: <https://arxiv.org/abs/1807.10792>.
- [148] Swapnil Patil, Milo Polte, Kai Ren, Wittawat Tantisiriroj, Lin Xiao, Julio López, Garth Gibson, Adam Fuchs, and Billie Rinaldi. “YCSB++: Benchmarking and Performance Debugging Advanced Features in Scalable Table Stores”. In: *Proc. of the Symposium on Cloud Computing (SOCC '11)*. ACM, 2011, pp. 1–14. DOI: 10.1145/2038916.2038925.
- [149] Tobias Pfandzelter and David Bermbach. “tinyFaaS: A Lightweight FaaS Platform for Edge Environments”. In: *Proc. of the International Conference on Fog Computing (ICFC '20)*. IEEE, 2020, pp. 17–24. DOI: 10.1109/ICFC49376.2020.00011.
- [150] Tobias Pfandzelter, Jonathan Hasenburg, and David Bermbach. “Towards a Computing Platform for the LEO Edge”. In: *Proc. of the International Workshop on Edge Systems, Analytics and Networking (EdgeSys '21)*. ACM, 2021, pp. 43–48. DOI: 10.1145/3434770.3459736.
- [151] Tobias Pfandzelter, Sören Henning, Trevor Schirmer, Wilhelm Hasselbring, and David Bermbach. “Streaming vs. Functions: A Cost Perspective on Cloud Event Processing”. In: *Proc. of the International Conference on Cloud Engineering (IC2E '22)*. IEEE, 2022, pp. 67–78. DOI: 10.1109/IC2E55432.2022.00015.
- [152] Michael Pradel, Markus Huggler, and Thomas R. Gross. “Performance Regression Testing of Concurrent Classes”. In: *Proc. of the International Symposium on Software Testing and Analysis (ISSTA '14)*. ACM, 2014, pp. 13–25. DOI: 10.1145/2610384.2610393.
- [153] Tilmann Rabl, Michael Frank, Hatem Mousselly Sergieh, and Harald Kosch. “A Data Generator for Cloud-Scale Benchmarking”. In: *Proc. of the Technology Conference on Performance Evaluation and Benchmarking (TPCTC '10)*. Springer, 2010, pp. 41–56. DOI: 10.1007/978-3-642-18206-8_4.
- [154] Brian Ramprasad, Joydeep Mukherjee, and Marin Litoiu. “A Smart Testing Framework for IoT Applications”. In: *Proc. of the International Conference on Utility and Cloud Computing (UCC '18)*. IEEE, 2018, pp. 252–257. DOI: 10.1109/UCC-Companion.2018.00064.
- [155] Kim-Thomas Rehmann, Changyun Seo, Dongwon Hwang, Binh Than Truong, Alexander Boehm, and Dong Hun Lee. “Performance Monitoring in SAP HANA’s Continuous Inte-

- gration Process”. In: *SIGMETRICS Performance Evaluation Review* 43.4 (2016). ACM, pp. 43–52. DOI: 10.1145/2897356.2897362.
- [156] David Georg Reichelt, Stefan Kühne, and Wilhelm Hasselbring. “Automated Identification of Performance Changes at Code Level”. In: *Proc. of the International Conference on Software Quality, Reliability and Security (QRS '22)*. IEEE, 2022, pp. 916–925. DOI: 10.1109/QRS57517.2022.00096.
 - [157] Alex Rodriguez. “Restful Web services: The basics”. In: *IBM developerWorks* (2008).
 - [158] Marcelino Rodriguez-Cancio, Benoit Combemale, and Benoit Baudry. “Automatic Microbenchmark Generation to Prevent Dead Code Elimination and Constant Folding”. In: *Proc. of the International Conference on Automated Software Engineering (ASE '16)*. ACM, 2016, pp. 132–143. DOI: 10.1145/2970276.2970346.
 - [159] Gregg Rothermel, Roland H Untch, Chengyun Chu, and Mary Jean Harrold. “Test Case Prioritization: An Empirical Study”. In: *Proc. of the International Conference on Software Maintenance (ICSM '10)*. IEEE, 1999, pp. 179–188. DOI: 10.1109/ICSM.1999.792604.
 - [160] Barbara G Ryder. “Constructing the Call Graph of a Program”. In: *IEEE Transactions on Software Engineering* 3 (1979). IEEE, pp. 216–226. DOI: 10.1109/TSE.1979.234183.
 - [161] Juan Pablo Sandoval Alcocer, Alexandre Bergel, and Marco Tulio Valente. “Learning from Source Code History to Identify Performance Failures”. In: *Proc. of the International Conference on Performance Engineering (ICPE '16)*. ACM, 2016, pp. 37–48. DOI: 10.1145/2851553.2851571.
 - [162] Juan Pablo Sandoval Alcocer, Alexandre Bergel, and Marco Tulio Valente. “Prioritizing versions for performance regression testing: The Pharo case”. In: *Science of Computer Programming* 191 (2020). Elsevier. DOI: 10.1016/j.scico.2020.102415.
 - [163] Jörg Schad, Jens Dittrich, and Jorge-Arnulfo Quiané-Ruiz. “Runtime Measurements in the Cloud: Observing, Analyzing, and Reducing Variance”. In: *Proceedings of the VLDB Endowment* 3.1–2 (2010), pp. 460–471. DOI: 10.14778/1920841.1920902.
 - [164] Dominik Scheinert, Alexander Acker, Lauritz Thamsen, Morgan K. Geldenhuys, and Odej Kao. “Learning Dependencies in Distributed Cloud Applications to Identify and Localize Anomalies”. In: *Proc. of International Workshop on Cloud Intelligence (CloudIntelligence '21)*. IEEE, 2021, pp. 7–12. DOI: 10.1109/CloudIntelligence52565.2021.00011.
 - [165] Gerald Schermann, Jürgen Cito, and Philipp Leitner. “Continuous Experimentation: Challenges, Implementation Techniques, and Current Research”. In: *IEEE Software* 35.2 (2018). IEEE, pp. 26–31. DOI: 10.1109/MS.2018.111094748.
 - [166] Gerald Schermann, Dominik Schöni, Philipp Leitner, and Harald C Gall. “Bifrost: Supporting Continuous Deployment with Automated Enactment of Multi-Phase Live Testing Strategies”. In: *Proc. of the International Middleware Conference (Middleware '16)*. ACM, 2016, pp. 1–14. DOI: 10.1145/2988336.2988348.
 - [167] Joel Scheuner, Marcus Bertilsson, Oskar Grönqvist, Henrik Tao, Henrik Lagergren, Jan-Philipp Steghöfer, and Philipp Leitner. “TriggerBench: A Performance Benchmark for

- Serverless Function Triggers”. In: *Proc. of the International Conference on Cloud Engineering (IC2E '22)*. IEEE, 2022, pp. 96–103. DOI: 10.1109/IC2E55432.2022.00018.
- [168] Joel Scheuner, Rui Deng, Jan-Philipp Steghöfer, and Philipp Leitner. “CrossFit: Fine-grained Benchmarking of Serverless Application Performance across Cloud Providers”. In: *Proc. of the International Conference on Utility and Cloud Computing (UCC '22)*. IEEE, 2022, pp. 51–60. DOI: 10.1109/UCC56403.2022.00016.
- [169] Joel Scheuner, Simon Eismann, Sacheendra Talluri, Erwin Van Eyk, Cristina Abad, Philipp Leitner, and Alexandru Iosup. *Let’s Trace It: Fine-Grained Serverless Benchmarking using Synchronous and Asynchronous Orchestrated Applications*. 2022. URL: <https://arxiv.org/abs/2205.07696>.
- [170] Joel Scheuner, Philipp Leitner, Jürgen Cito, and Harald Gall. “Cloud WorkBench – Infrastructure-as-Code Based Cloud Benchmarking”. In: *Proc. of the International Conference on Cloud Computing Technology and Science (CloudCom '14)*. IEEE, 2014, pp. 246–253. DOI: 10.1109/CloudCom.2014.98.
- [171] Trever Schirmer, Nils Japke, Sofia Greten, Tobias Pfandzelter, and David Bermbach. “The Night Shift: Understanding Performance Variability of Cloud Serverless Platforms”. In: *Proc. of the Workshop on SErverless Systems, Applications and MEthodologies (SESAME '23)*. ACM, 2023, pp. 27–33. DOI: 10.1145/3592533.3592808.
- [172] Mohammad Shahrads, Jonathan Balkind, and David Wentzlaff. “Architectural Implications of Function-as-a-Service Computing”. In: *Proc. of the International Symposium on Microarchitecture (MICRO '52)*. ACM, 2019, pp. 1063–1075. DOI: 10.1145/3352460.3358296.
- [173] Mohammad Shahrads, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. “Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider”. In: *Proc. of the USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX, 2020, pp. 205–218.
- [174] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Tech. rep. 2010-1. Google, 2010.
- [175] Marcio Silva, Michael R Hines, Diego Gallo, Qi Liu, Kyung Dong Ryu, and Dilma Da Silva. “Cloudbench: Experiment Automation for Cloud Environments”. In: *Proc. of the International Conference on Cloud Engineering (IC2E '13)*. IEEE, 2013, pp. 302–311. DOI: 10.1109/IC2E.2013.33.
- [176] Nikhila Somu, Nilanjan Daw, Umesh Bellur, and Purushottam Kulkarni. “PanOpticon: A Comprehensive Benchmarking Tool for Serverless Applications”. In: *Proc. of the International Conference on COMMunication Systems & NETworkS (COMSNETS '20)*. IEEE, 2020, pp. 144–151. DOI: 10.1109/COMSNETS48256.2020.9027346.
- [177] Petr Stefan, Vojtěch Horký, Lubomír Bulej, and Petr Tůma. “Unit Testing Performance in Java Projects: Are We There Yet?” In: *Proc. of the International Conference on Per-*

-
- formance Engineering (ICPE '17)*. ACM, 2017, pp. 401–412. DOI: 10.1145/3030207.3030226.
- [178] Chunqiang Tang, Thawan Kooburat, Pradeep Venkatachalam, Akshay Chander, Zhe Wen, Aravind Narayanan, Patrick Dowell, and Robert Karl. “Holistic Configuration Management at Facebook”. In: *Proc. of the Symposium on Operating Systems Principles (SOSP '15)*. ACM, 2015, pp. 328–343. DOI: 10.1145/2815400.2815401.
- [179] Mert Toslali, Srinivasan Parthasarathy, Fabio Oliveira, Hai Huang, and Ayse K. Coskun. “Iter8: Online Experimentation in the Cloud”. In: *Proc. of the Symposium on Cloud Computing (SoCC '21)*. ACM, 2021, pp. 289–304. DOI: 10.1145/3472883.3486984.
- [180] Dmitrii Ustiugov, Theodor Amariuca, and Boris Grot. “Analyzing Tail Latency in Serverless Clouds with STeLLAR”. In: *Proc. of the International Symposium on Workload Characterization (IISWC '21)*. IEEE, 2021, pp. 51–62. DOI: 10.1109/IISWC53511.2021.00016.
- [181] Alexandru Uta, Alexandru Custura, Dmitry Duplyakin, Ivo Jimenez, Jan Rellermeier, Carlos Maltzahn, Robert Ricci, and Alexandru Iosup. “Is Big Data Performance Reproducible in Modern Cloud Networks?” In: *USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*. USENIX, 2020, pp. 513–527.
- [182] André Van Hoorn, Jan Waller, and Wilhelm Hasselbring. “Kieker: A Framework for Application Performance Monitoring and Dynamic Software Analysis”. In: *Proc. of the International Conference on Performance Engineering (ICPE '12)*. ACM, 2012, pp. 247–248. DOI: 10.1145/2188286.2188326.
- [183] Jan Waller, Nils C. Ehmke, and Wilhelm Hasselbring. “Including Performance Benchmarks into Continuous Integration to Enable DevOps”. In: *Software Engineering Notes* 40.2 (2015). ACM, pp. 1–4. DOI: 10.1145/2735399.2735416.
- [184] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. “Peeking Behind the Curtains of Serverless Platforms”. In: *Proc. of the USENIX Annual Technical Conference (USENIX ATC '18)*. USENIX, 2018, pp. 133–145.
- [185] Elaine J. Weyuker and Filippos I. Vokolos. “Experience with Performance Testing of Software Systems: Issues, an Approach, and Case Study”. In: *Transactions on Software Engineering* 26.12 (2000). IEEE, pp. 1147–1156.
- [186] Li Wu, Jasmin Bogatinovski, Sasho Nedelkoski, Johan Tordsson, and Odej Kao. “Performance Diagnosis in Cloud Microservices Using Deep Learning”. In: *Proc. of the International Conference on Service-Oriented Computing (ICSOC '20)*. Springer, 2020, pp. 85–96. DOI: 10.1007/978-3-030-76352-7_13.
- [187] Li Wu, Johan Tordsson, Jasmin Bogatinovski, Erik Elmroth, and Odej Kao. “MicroDiag: Fine-grained Performance Diagnosis for Microservice Systems”. In: *Proc. of the International Workshop on Cloud Intelligence (CloudIntelligence '21)*. IEEE, 2021, pp. 31–36. DOI: 10.1109/CloudIntelligence52565.2021.00015.
- [188] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. “MicroRCA: Root Cause Localization of Performance Issues in Microservices”. In: *PROC. of the Network Operations and*

- Management Symposium (NOMS '20)*. IEEE, 2020, pp. 1–9. DOI: 10.1109/NOMS47738.2020.9110353.
- [189] Tianyi Yu, Qingyuan Liu, Dong Du, Yubin Xia, Binyu Zang, Ziqian Lu, Pingchao Yang, Chenggang Qin, and Haibo Chen. “Characterizing Serverless Platforms with Serverless-Bench”. In: *Proc. of the Symposium on Cloud Computing (SoCC '20)*. ACM, 2020, pp. 30–44. DOI: 10.1145/3419111.3421280.
- [190] Shahed Zaman, Bram Adams, and Ahmed E Hassan. “Security Versus Performance Bugs: A Case Study on Firefox”. In: *Proc. of the Working Conference on Mining Software Repositories (MSR '11)*. ACM, 2011, pp. 93–102. DOI: 10.1145/1985441.1985457.
- [191] Shahed Zaman, Bram Adams, and Ahmed E. Hassan. “A Qualitative Study on Performance Bugs”. In: *Proc. of the Working Conference on Mining Software Repositories (MSR '12)*. IEEE, 2012, pp. 199–208. DOI: 10.1109/MSR.2012.6224281.
- [192] Ben Zhang, Nitesh Mor, John Kolb, Douglas S. Chan, Ken Lutz, Eric Allman, John Wawrzynek, Edward Lee, and John Kubiawicz. “The Cloud is Not Enough: Saving IoT from the Cloud”. In: *Proc. of the USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '15)*. USENIX, 2015, pp. 1–7.
- [193] Qi Zhang, Lu Cheng, and Raouf Boutaba. “Cloud computing: state-of-the-art and research challenges”. In: *Journal of Internet Services and Applications* 1 (2010), pp. 7–18. DOI: 10.1007/s13174-010-0007-6.
- [194] Haidong Zhao, Zakaria Benomar, Tobias Pfandzelter, and Nikolaos Georgantas. “Supporting Multi-Cloud in Serverless Computing”. In: *Proc. of the International Conference on Utility and Cloud Computing (UCC '22)*. IEEE, 2022, pp. 285–290. DOI: 10.1109/UCC56403.2022.00051.
- [195] Qing Zheng, Haopeng Chen, Yaguang Wang, Jiangang Duan, and Zhiteng Huang. “COS-Bench: A Benchmark Tool for Cloud Object Storage Services”. In: *Proc. of the International Conference on Cloud Computing (CLOUD '12)*. IEEE, 2012, pp. 998–999. DOI: 10.1109/CLOUD.2012.52.